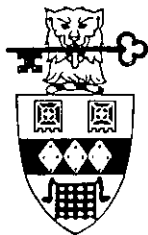


VECTOR

2nd Generation APLs on the PC

- * John Searle's Guide to APL Conferences
- * Election Results and AGM Report
- * 16-page Educational Supplement
- * Reviews of APL*PLUS II and Sax
- * David Selby on APL2 for the PC
- * David Piper on Parallel Computing and APL



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.5 No.4 April 1989

Contributions

All contributions to VECTOR may be sent to the Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (ideally with a photograph of the author and a brief biographical note). Unless otherwise specified, each item will be considered for publication as a personal statement by the author.

Please supply as much material as possible in machine-readable form, ideally on an IBM PC compatible (DSDD) diskette. APL code should be kept separate from the text, and can be accepted as camera-ready copy, or as APL*PLUS/PC workspaces or IBM .AIO files.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1989-90

Category	Fee	Vectors	Passes
Non-voting Student	£5	1	1
UK Private	£10	1	1
Overseas Private	£18	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£85	10+	5
Overseas Corporate	£140	10+	
Sustaining	£360	50+	5

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Treasurer on the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £100 for half-page or less (subject to layout constraints).

Deadlines for bookings and copy are given under the Quick-reference Diary

Advertisements should be booked with, and sent to, Alison Chatterton, whose address is given with the Index to Advertisers.

CONTENTS

		Page
EDITORIAL:		
In which Piglet meets a Heffalump	Adrian Smith	3
APL NEWS		
Quick Reference Diary		7
APL Course Diary		9
General Correspondence		10
APL89 and APL90 Information		12
A Guide to Conference Going	John Searle	17
British APL Association News	Graham Parkhouse	19
AGM and Election Results		18
Incoming Committee Statements		21
News from Sustaining Members	Alison Chatterton	22
The Education VECTOR	Alan Sykes	27
REVIEWS		
APL Product Guide	Alison Chatterton	43
2nd Generation APLs on Micros ...	Adrian Smith	53
APL*Plus II (technical)	Peter Cyriax	55
APL*Plus II (applications)	Ajay Askoolum	61
SAX	Michael Berry	67
RECENT MEETINGS		
APL: the Future Directions		
Dyadic Systems	Peter Donnelly	78
APL2 on the IBM PC	David Selby	82
GENERAL ARTICLES		
Standardised Test Scores	Jerry Brennan	86
TECHNICAL SECTION		
Prize Competition Results		
Caption Competition	David Ziemann	94
Hackers Corner (DOS envt variables)	Adrian Smith	95
Technical Correspondence		96
Logic in APL	Joseph L.F. de Kerf	99
Parallel Computing and APL	David Piper	113
British APL Association Software Library		
New Entries	Alison Chatterton	123
Provisional Contents of VECTOR 6.1		126
Index to Advertisers		127

Have you remembered to renew your
BAA subscription?

(For categories of membership
see inside front cover.)

If this is someone else's VECTOR, how
about taking the plunge and joining?
Old VECTORs are already collectors' items.
Start your collection now!!

Think what you miss by not joining - you won't hear
about meetings which cover topical issues of
importance to the APL user. You won't know about any
special events organised by the BAA, and
of course you won't receive VECTOR!

Why not spread the word - you must know other APLers who
are not members, so encourage them to join. Tell them what
they're missing by not taking part! If every existing member
persuaded just one other person to join, we could double the
membership of the BAA this year! Let's go for it!

The logo consists of the letters 'APL' in a bold, serif font. The letters are white with a thick black outline. They are set against a dark, rectangular background that has a horizontal gradient, being lighter at the top and darker at the bottom. The letters are slightly shadowed, giving them a three-dimensional appearance as if they are floating above or attached to the background.

Editorial: In which Piglet meets a Heffalump

by Adrian Smith

One day, when Christopher Robin and Winnie-the-Pooh and Piglet were all talking together, Christopher Robin finished the mouthful he was eating and said carelessly: "I saw a Heffalump today, Piglet."

"What was it doing?" asked Piglet.

"Just lumping along," said Christopher Robin. "I don't think it saw me."

"I saw one once," said Piglet. "At least I think I did," he said. "Only perhaps it wasn't."

"So did I," said Pooh, wondering what a Heffalump was like.

"You don't often see them," said Christopher Robin carelessly.

"Not now," said Piglet.

"Not at this time of year," said Pooh.

Avid readers of the works of A.A. Milne may like to follow up the reference! Of course the outcome was the construction of a Cunning Trap along the lines of a Very Deep Pit into which the Heffalump would fall. The outcome was rather different ...

So to the substance of this Editorial: why can't someone write a second-generation APL which is not littered with Heffalump traps? There are two particular pitfalls into which I still occasionally fall, and from which I have wasted many valuable hours extracting bemused colleagues. Here is a line of code to illustrate the worst:

```
DISPLAY 2 PLUS 2
```



How did you read this? I bet you assumed that PLUS was a dyadic function, which returned a result for neat output by monadic function DISPLAY?! You comprehended the line in two very different ways: firstly you used your knowledge of English to make sense of the context of the words, then you confirmed your supposition with a quick check that it did indeed correspond to a valid APL expression. In plain old VS APL the syntax of the language mapped rather well to the syntax of standard English, and all was well.

Just suppose for a minute that PLUS is actually monadic! The line of code is still valid, but DISPLAY now gets a 2-element vector to handle. This I can live with. Suppose we make DISPLAY a variable (or a niladic function) e.g. DISPLAY←3 ... PLUS will now add the vector 3 2 to 2 (if it was dyadic), or it will just return its right argument (if monadic), and the result of the whole expression will be the vector 3 2 2. Of course, we could imagine the case where PLUS←5 ...

This (apparently) simple APL expression has the scope for an enormous variety of misinterpretations! The problem is that the information you use to decode it will depend on the way you read the English language. In contrast the interpreter has access to a deal of privileged internal information (e.g. it can easily check whether an object is a function or a variable) which is not available on casual inspection of the code. The example which follows is quite real, and wasted more time than I care to admit:

```
SHOW 'JOURNEY,TIME,DEST' WHERE (TIME > 2.5) AND TIME < 8
```

In the SQL-style syntax of our in-house 4GL, this looks absolutely valid. AND is a dyadic cover for $\wedge$. WHERE does the obvious compression (returning its left argument to keep things going), and SHOW puts the result on the screen. Easy eh? Suppose I told you that the expression failed on a LENGTH ERROR in AND? However, when taken apart:

```
ZZ+(TIME > 2.5) AND TIME < 8
```

... it worked fine!! One clue was that the whole thing was taking place inside a function, and again it all worked as you would expect if you cleared the stack and ran it from APL. The mists slowly began to clear when I noticed that WHERE had actually been localised, and assigned to the boolean result from a previous expression. The consequence was that APL was seeing:

```
SHOW (( 'JOURNEY,TIME,DEST' ) (WHERE) (TIME > 2.5) ) AND TIME < 8
```

... let us be thankful that TIME did not contain exactly three elements, otherwise this hideous piece of nonsense would have 'worked' and caused even more confusion somewhere else!

"So at last he lifted up his head, jar and all, and made a loud roaring noise of Sadness and Despair
... and it was at that moment that Piglet looked down."

So much for Strands. These are the first and worst of the Heffalump traps. When I read in Peter's review of APL*PLUS-II that: "The single most important use for Nested Arrays is to pass multiple parameters into functions", I get very concerned. How on earth am I to tell which are the functions, and which are the arguments of which functions? "Bother" said Pooh!!

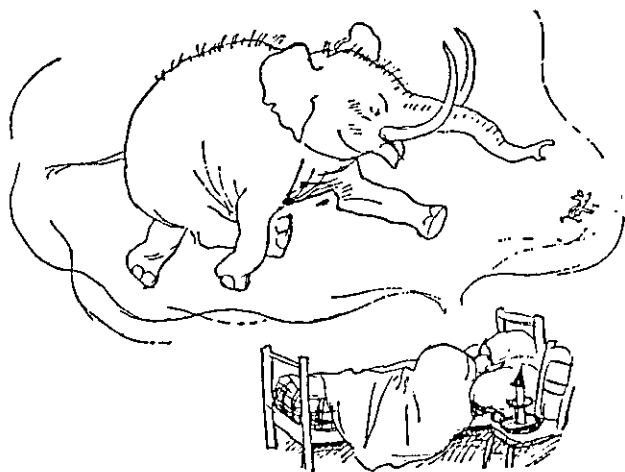
Trap No.2 is much more insidious, but equally nasty. It is called 'mixed' or 'heterogeneous' arrays. Here is how it works: you have a simple little print routine which starts with PAGE+66 132p ' ' and drops blocks of your report into the appropriate parts of the page. You make a minor change to this routine, which involves inserting a page number in the top left hand corner:

```
PAGE[1;1]+PGCT
```

... but unfortunately you forget to format PGCT. Any sensible language (I count VS APL among the sensible languages) would complain about this, but dear old APL*PLUS just soaks up a lot more storage, puts the lights out in computer room, and carries on.

Some little while later, in a different function (with good modular design you returned PAGE as an explicit result), you try to write this harmless-looking array to the SYSOUT queue via an AP. *** SPLAT *** it duly goes, typically with something helpful like SV SPACE QUOTA EXCEEDED. You check the shape of PAGE, you display it on the screen, you utter various carefully selected incantations, but can you see what is wrong? Even when you learn the trick `⌈/ ,PAGE⌈⌈AV` and find the rogue characters, you can still waste a surprising amount of time tracking down the missing `⌈`.

Maybe I'm getting even older and even crabbier, but I would find it much easier to stand up and justify plain old VS APL (with all its limitations) as a good language for serious commercial work, than I would to defend a language which regularly sets traps like this. As E.H. Shepard so aptly drew it ...



dyalog APL

C INTERFACE

```
#include "apl.h"
#include "support.h"

main()
{
    ARRAY *rslt, *larg, *rd;
    *getarray(), *frr;
    *idn_d(), *d_idn;
    int fncode;

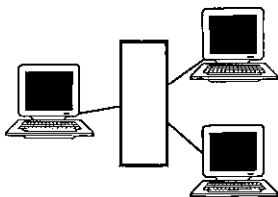
    startup(2);
    define("IDN_TO_DATE", RSI);
    define("DATE_TO_IDN", RSI);

    larg = rarg = rslt = 0;
    setjmp(errbuff);

    while(1)
    {
        larg = freearray;
        rarg = freearray;
    }
}
```

UNIX INTERFACE

MULTI-USER



CHOICE OF HARDWARE

IBM Ps/2 Model 80
Compaq 386
Altos 3068
Apollo
AT&T 382
Bull SPS7, SPS9
Dec Vax
Hewlett-Packard HP9000
IBM 6150
Masscomp
NCR Tower 32
Pyramid
Sun
Unisys Series 5000

FILE SYSTEMS

EDITORS

```
[0] Z←DISPLAY Z;
[1] □IO+0 □ SZ←*
[2] Z←' '(DFMT
[3] NSS:→(1 0/DZ
[4] NOR:Z←(1/pZ)p
[5] →(1=DZ)/SMP
[6] Z←' '(QTV~1
[7] SMP:QB+(2i0 '
[8] FMT:Z←[FMT Z
[9] BOX:R C←1+pZ
[10] C←(1+1/2 C)
[11]
```

FULL-SCREEN INPUT

Share Name _____

Number of Shares _____

Unit Cost _____

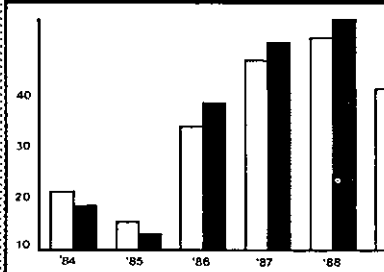
Buyer _____

SQL INTERFACE

EMPNO	ENAME	JOB	MGR
7329	SMITH	CLERK	7902
7499	ALLEN	SALESMAN	7698
7521	WARD	SALESMAN	7698
7566	JONES	MANAGER	7839
7654	MARTIN	SALESMAN	7698
7698	BLAKE	MANAGER	7830
7782	CLARK	MANAGER	7839
7788	SCOTT	ANALYST	7566
7839	KING	PRESIDENT	
7844	TURNER	SALESMAN	7698
7876	ADAMS	CLERK	7788

DEVICE SUPPORT

GRAPHICS INTERFACE



NESTED ARRAYS

DISPLAY REPORT

	North	South	East
Cakes	14	76	46
Buns	22	5	68

...gives you all this – and APL too!

For further information, contact
Dyalog Systems Limited
Park House, The High Street, Alton, Hampshire, GU34 1EN
Telephone: (0420) 87024. Telex: 858811

Quick Reference Diary

Date	Venue	Event
29 September 1989	London	BAA Meeting
27 October	London	BAA Meeting
24 November	London	BAA Meeting
12 January 1990	London	BAA Meeting
16 February	London	BAA Meeting
23 March	London	BAA Meeting
20 April	London	BAA Annual General Meeting

All British APL Association meetings are to be held at the Royal Over-Seas League, Park Place, near Green Park Tube station. Meetings begin at 2.00pm and close at 5.30pm.

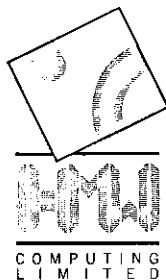
Provisional Topics for BAA Meetings

by Peter Branson

The following is a preliminary list of topics suggested at the AGM on 21st April:

- Applications of 2nd generation APLs. A selection of speakers demonstrating how the newer APL features can enhance productivity in substantial applications.
- Design of large APL systems. Considerations of workspace structure; function modularity; data structures; file systems; end-user screens etc. (Illustrated with practical applications.)
- Progress in APL standards work. A review of the standards committee work since the publication of the first draft standard. Progress to date, and where next?
- APL at British Airways. A review of some of their major applications using APL. (This meeting might be held at Heathrow where a full demonstration of the systems would be possible.)
- APL - another year on. The major APL suppliers to report on the past year, and on what goodies they have coming along for us. (This is likely to be the penultimate meeting of the BAA year.)

A number of other suggestions were made which are still being considered, and a firm programme will be drawn up and included in VECTOR 6.1 (July 1989). If any members who were unable to attend the AGM have further ideas for meeting topics, please let me know as soon as you can at the address / phone number on the inside back cover.



SOLUTIONS FOR THE CITY



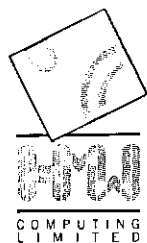
We are a specialist firm of consultants with a prestigious list of clients including major City institutions.



For one of these clients we have recently developed a foreign exchange dealing system with the following features:— deal entry, deal enquiry, multi-currency position keeping, basket currency calculators, arbitrage calculators and ticket production.



If you are interested in foreign exchange dealing, treasury operations, portfolio management, commodity broking or similar areas we would be delighted to discuss how we could help. Call Stan Wilkinson, HMW COMPUTING LIMITED Hamilton House 1 Temple Avenue London EC4Y 0HA Telephone 01-353 4212



EXCITING POSITIONS AT THE LEADING EDGE



With top City institutions and leading Blue chip companies amongst our client list we have a lot to offer experienced APL programmers.

Call Stan Wilkinson to discuss your future.

HMW COMPUTING LIMITED
Hamilton House 1 Temple Avenue
London EC4Y 0HA Telephone 01-353 4212

APL Training Courses

Dates shown cover the period from May to December 1989. For confirmation of dates and/or further details please contact the vendor directly.

Level	Company	Days	Dates
Beginners	Cocking & Drury	3	6 June, 20 June, 4 July, 18 July
	MicroAPL	1	29 June, 24 Aug, 19 Oct
	Mercia	3	17 July, 12 Sept
APL*PLUS/PC	MicroAPL	1	20 July, 14 Sept, 9 Nov
Intermediate	MicroAPL	1	6 July, 31 Aug, 26 Oct
	Mercia	2	6 June
Advanced	MicroAPL	1	13 July, 31 Aug, 14 Sept, 26 Oct
System Design	Cocking & Drury	4	
	Mercia	3	28 June, 3 August
Statgraphics	Cocking & Drury	2	13 June, 11 July
	Mercia	1	27 June, 2 August

Dates for future issues of VECTOR

	Vol.6 No.1	Vol.6 No.2	Vol.6 No.3
Copy date	1st June 89	1st Sept 89	30th Nov 89
Ad booking	12th June 89	10th Sept 89	8th Dec 89
Ad Copy	25th June 89	21st Sept 89	23rd Dec 89
Distribution	July 89	October 89	January 90

General Correspondence

The VECTOR working group welcomes correspondence on any topic affecting the APL community. All letters should be addressed to the Editor, and should indicate whether they are for the general or technical section. (Letters containing APL code will normally be classed as 'technical'.) The editor reserves the right to edit any letter unless the writer states that it is to be published in full or not at all.

Printer Support from QL-APL

From: Alan Angus

14th May 1989

Several months ago I bought MicroAPL's interpreter for the Sinclair QL and I have been using it to explore the possibility of using APL in my work as a maths teacher. I have also acquired a copy of the I-APL interpreter for use in school. Norman Thomson and Anthony Camacho have both been very helpful, answering enquiries and sorting out problems, and the articles by Alan Sykes in Education Vector have been most instructive.

It would be a great help to me, and to others learning APL and planning to introduce it into schools, if there was a pool of public domain programs available. Are there any simple APL programs or tutorials available on disk in the BAA library?

The I-APL interpreter has a built-in facility to enable the special APL character set to be printed out on an Epson-compatible dot matrix printer; I wish MicroAPL's QL interpreter had this feature. MicroAPL have been most helpful, but they obviously cannot afford to give much support to a machine that offers such a small, and shrinking, market. Are there any other users of QL APL who can help me? What I need is some way of printing the APL character set on an Epson-compatible printer (I use a Brother M1009). Other things that would be very useful are a screen dump and some way of transferring workspaces between I-APL and MicroAPL.

Hopefully by next term I will have enough knowledge of APL to enable me to exploit I-APL in my mathematics teaching. With APL, LOGO and symbolic algebra packages available on powerful microcomputers there are many exciting possibilities to be explored in school mathematics.

Yours sincerely,

Alan Angus,
1 Mitford Avenue,
Blyth,
Northumberland NE24 5SQ.

APL in Statistics Conference

From: Alan Sykes

10th May 1989

The British APL Association and the APL Statistics User Group are to hold a conference at the University of Wales Conference Centre in Gregynog, near Newtown, on 18-20 September 1989.

The aim of the conference is to:

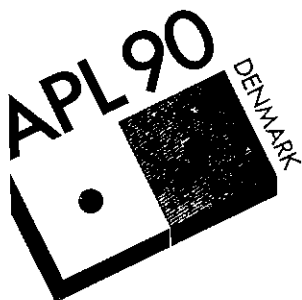
- (a) discuss areas of statistical research and teaching which are making use of APL at present;
- (b) propose draft standards for writing APL statistical code;
- (c) discuss the nature of functions which should be included in a collection of basic statistical functions;
- (d) review currently available APL routines.

This conference is part of a longer term project (which we hope will receive substantial financial support from the BAA, and perhaps also from SIGAPL) called ASL (APL Statistics Library) to produce a library of statistical functions written in APL which would be available as a public domain library for the academic community. Eventual release to commercial statistical users is planned, although probably on a chargeable basis.

We propose to schedule the conference to alternate presentations on areas of current statistical research which use APL, and discussion periods to cover the establishment of standards (and the library).

Academic and industrial statisticians, and other interested parties requiring further details of the conference (and the proposed project) should contact

Dr. A.M. Sykes
Department of Management Science and Statistics
University College of Swansea, Singleton Park, Swansea SA2 8PP
(Telephone: 0792-295296, or JANet: JAN@UK.AC.SWAN.MS).



FOR THE FUTURE



Dansk Dataforening

The International Conference on APL

13 - 17 August 1990

Hotel Marienlyst, Helsingør, Denmark

CALL FOR PAPERS

Organized by

The APL Special Interest Group of Danish Data Association

Co-sponsored by

The Association of Computing Machinery, Special Interest Group on APL,

ACM/SIGAPL

APL90 in Denmark

The APL90 conference will take place from 13th to 17th August 1990 at the Marienlyst Hotel in Elsinore (Helsingør), near Copenhagen.

This seaside hotel offers an excellent environment for APL90, the second APL conference to be arranged in Denmark. We are looking forward to a conference with many valuable papers on the progress of APL and its use in numerous applications areas.

We encourage you to submit a paper to the conference. You may choose between submitting a regular paper and submitting a proposal for a poster session. The schedule for submission is as follows:

- Please send your abstract before the end of September 1989 to the APL90 address shown at the end of this announcement, or to one of the members of the program committee.
- Then send a complete draft before the end of November 1989. The submitted papers will be reviewed, and an answer will be sent to all submitters before the end of January 1990.

Both abstracts and complete drafts may be sent by mail, telefax, PC readable diskette, or by any of the available mailboxes.

For accepted papers we expect to receive a camera ready copy before the end of April 1990. Detailed instructions on the preparation of the paper will be mailed with the acceptance. We intend to mail the conference proceedings to all participants before the conference.

Plans for the conference include a Software Exchange as well as an exhibition, showing many of the new offerings in APL systems.

A call for participation will be distributed early 1990. Please note that a wide range of accommodation has been reserved already, from four star hotels to family rooms and a youth hostel. Also, a program for the families will be offered during the conference.



ASSOCIATION FOR COMPUTING MACHINERY

APL90 Program Statement:

In line with tradition, APL90 will be a forum for exchange of experience and ideas within the world of APL. 'For the future' is the main theme of the conference, which will illustrate, through the presentations and discussions, why APL not only has been but will remain an excellent tool for Information Management.

Whenever new technologies emerged, APL was there to greet them. Think of SQL/DB2 interfacing and vector processing, to name a few. By its open and dynamic structure, APL is particularly suited for the development and implementation of new ideas and techniques. We encourage papers illustrating how such front-line concepts are delivered through the use of APL. Any bright ideas on Image Processing, Neural Networks ...?

Effective Information Management and Management Information will be increasingly important for the competitive ability of any organization in the future. We invite papers showing how APL can combine the creation and use of corporate-wide information systems with end user computing by professionals. We especially welcome papers which demonstrate how this can be done in an efficient, structured and well documented way.

We trust that APL90 will provide APL implementers and philosophers alike with the incentive and opportunity to present their current work and to discuss ideas and plans for the future. However, as much program development in other languages now takes place in environments that rival or surpass APL's once unique environment, we hope that users will take this opportunity to put demand on the implementers for modernizing APL's look and feel. We encourage papers that guide the implementers in the right direction, so that our language can remain competitive in the world of windows, CASE, object oriented programming systems and distributed databases.

APL was originally developed for educational purposes and has recently started to find its way back into schools due to its availability on small computers. It is an attractive educational tool because it enables teachers to use data with its natural structure, rather than in a traditional, scalar fashion. We encourage papers which focus on the advantages of this use of APL.

APL implementations continue to evolve, with extensions to the language itself, as well as interfaces to - and from - other environments; recently, encouraging signs have been noted with respect to standardization of 'enhanced' APL dialects. We would very much like to see papers presenting problems in and, hopefully, proposing solutions to these areas.

APL90 Addresses:

APL90 Conference
 c/o Kim S. Andreassen
 Snerlevej 3
 DK-3000 Helsingør, Denmark

Telefax: + 45 42 84 73 16 (I.P.Sharp Copenhagen), or
 + 45 42 10 05 32 (Beta-plan Helsingør)
 Mailbox: IPSA APL90
 IBM GJERLOV at CPHVM1
 EARN/BITNET APL90 at DKUCCC11
 BIX kandreasen

Conference Chair:
 Gitte Christensen, I. P. Sharp, Denmark
 IPSA Mailbox: GITTE

Program Committee:

Chair:
 Per Gjerlov, IBM A/S, Denmark
 Home address: Bakkevej 2A, DK-3460 Birkerød,
 Denmark
 IPSA mailbox: PER

Jan Bohman, Lunds University, Sweden
 Work address: Lunds Tekniska Högskola,
 Inst. for Datalogi och Numerisk Analys,
 P.O. Box 118, S-2210 Lund, Sverige

Hans J. Helms,
 The Council of the EEC, Bruxelles, Belgium
 Home address: 13 Avenue de l'Echevinage,
 B-1180 Bruxelles, Belgium

Ivan Jensen, NOVO Industri A/S, Denmark
 Home address: Eivindsvej 12, DK-2920
 Charlottenlund, Denmark
 IPSA mailbox: IJENSEN

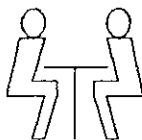
Maurice Jordan, British Airways
 Work address: British Airways,
 Comet House (S42),
 P.O Box 10, Heathrow Airport, Hounslow,
 Middlesex TW6 2JA, Great Britain

Andreas Werder, Credit Suisse
 Home address: Neunbrunnenstrasse 188,
 CH-8046 Zuerich, Schweiz

Conference Bureau:

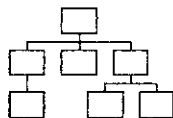
DIS Congress Service Copenhagen A/S
 Linde Alle 48
 DK-2720 Vanløse, Denmark
 Telephone + 45 31 71 22 44
 Telefax + 45 31 71 60 88
 Telex 15476 dis dk

A single source for APL



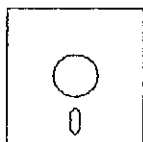
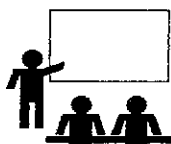
Advice

Design



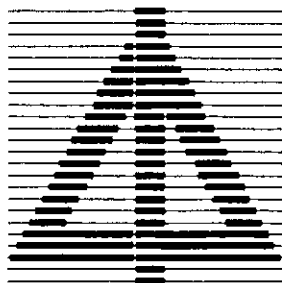
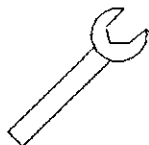
Application
Development

Education



APL software

Support



COCKING & DRURY LTD.

THE APL PROFESSIONALS

180 Tottenham Court Road,
London W1P 9LE

Tel: 01 - 436 9481

An APLer's Guide to Conference Going

by John Searle

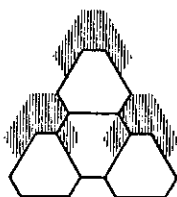
So you are thinking about attending an APL conference! This is a suggested guide to maximizing your enjoyment and value for money from attending one of these unique occasions.

1. Orientation. Upon arrival, consult all available maps and learn the fastest path from your accommodation to the conference and to the nearest bar.
2. Registration. If your name is A-K, deliberately go to the L-Z desk just to see how well the organizers are coping.
3. Proceedings. Desperately search through the proceedings for papers that cite your own previous papers as references. If appropriate, check that your own paper has been included and that it looks as good as everyone else's.
4. Programme. Which streams to attend? Which talks clash with the opening hours of the nearest bar? Is the new triadic transpose operator to be discussed at any of the talks? James Wheeler is sure to be announcing a new release of APL*PLUS.
5. Freebies. Look at, feel, squeeze and smell all of the conference giveaways. Develop a first assessment of where the bulk of your conference registration money has been spent. Look for true APLers who are still carrying / wearing / drinking out of last year's freebies.
6. The Banquet. This is the true test of the value of an APL conference. For a quiet evening, sit with people that you know. For a more challenging and memorable evening, sit with people that you don't know and who don't speak your native language. For an outrageous evening, sit with either the Finns or Phil Benkard. Revise your estimate of where your conference registration money has been spent.
7. Hospitality Suites. Check them all out, and frequent the ones with the best food and drink. Try to extract as much information as possible from the demonstrations, but when talking to sales people wear someone else's name badge.
8. Information Desk. Marvel at how the registration desk has been modified to an information desk. Always wait until the last day to buy extra conference freebies at cut-rate prices. Harass the overworked conference committee at every possible opportunity, just to see how they are still responding to stress.

9. On Reflection. On departure, shake everyone by the hand and promise to see them at $APL8x + 1$. Realise that the "gods" of APL are really just people after all.

Someone [1] said that the future of APL depended on the extent to which we are still ahead of our time. Let us hope that time proves this to be true.

- [1] Someone, J., "The Future of Programming - W(h)ither APL?", APL84 Conference Proceedings, Helsinki, June 1984.



APL PEOPLE

THE APL EMPLOYMENT AGENCY

- ★ Permanent and Contract Vacancies
- ★ UK and Overseas
- ★ Free Advice on career matters
- ★ Discreet, confidential service

We currently have many openings for programmers
with 1-3 years APL experience (any type).

Contact Jill Moss on 0225 462602 during office hours
or 0225 333618 eves & w/ends

or write to her at The Old Malthouse, Clarence St., Bath BA1 5NS

**Minutes of the 1989 AGM
of the British APL Association
21 April 1989 at the Royal Over-Seas League**

Chairman: Peter Donnelly
Minutes: David Eastwood

The AGM opened at 14.05.

1. Minutes of the 1988 AGM

These appeared in Vector 5.1 and were approved by those present.

2. Chairman's Report.

The past year has been one of mixed fortunes for the BAA. There has been a lot of good work done by committee members and others, but poor administration has let us down time and time again.

APL89 in Canterbury was very well received, and an overall success. It was especially good to see so many visitors from abroad. On behalf of the BAA I would like to thank the APL89 committee - Dick Bowman, Christine McCree, Mel Chapman, Dave Parker and Phil Goacher for their efforts.

VECTOR continues to be an interesting high-quality technical magazine, but is more often late than on schedule; sometimes disastrously so. After the sad case of Vol.5 No.2 (dated November 1988 but published on 17 March 1989), the committee has resolved to make the timely publication of VECTOR a major target for next year. For many people VECTOR is THE reason for membership of the BAA. If members don't receive their VECTORS it is hardly surprising that they fail to renew their subscriptions. Despite these problems, I would like to extend my thanks to Adrian Smith and the other members of the VECTOR sub-committee who I know put in many long hours on our behalf.

The meetings this year have been of a consistently high standard thanks to Jonathan Martin. For me the talks on Artificial Intelligence, Parallel Processing, SQL and Function-Point Analysis have been both interesting and instructive; and illustrate a healthy trend away from APL insularity. The meeting devoted to Future Plans from the Vendors was also, I understand, well received. If there was an award for the most humorous APL talk of the year it would surely go to John Searle for his highly entertaining A-Z of APL at the meeting in March.

In stark contrast to the quality of the meetings has been the poor publicity for them. Although it would be wrong of me to divert blame from the committee, we have on several occasions been badly let down by the BCS who do our mailing. Despite the additional cost it would entail, the committee is considering the idea of doing our own mailing next year.

One of the disappointments expressed by Dick Bowman last year was the total lack of applications from members for funding. The BAA has considerable financial resources, and is anxious to spend money on worthy projects. I am pleased to report that this year David Eastwood has been busy with the APL Statistics User Group with a scheme to establish the APL statistics library. The BAA committee has given outline approval for the project and is awaiting firm proposals from the Statistics group. I understand that a mini conference to define objectives and discuss methodology etc. is planned for later this year. Alan Sykes, who is chairing the conference committee, would welcome interest from commercial users.

On the education front, Alan Sykes has been very active as the editor of Education Vector, and in generally promoting APL to universities and schools. Among other things, Alan has submitted articles for Teaching Statistics and has delivered a talk at the 1989 Mathematics Association conference. Alan is to be congratulated for his efforts.

Alison Chatterton has now taken over many of the routine administrative tasks of the BAA. In particular she has assumed responsibility for the maintenance of the membership list and for the production of mailing lists. The committee hopes that Alison will be able to help towards the timely production of VECTOR in the future.

APL scored a number of PR successes last year, in particular the article by John Samson in Computing which was at least partly the result of the efforts made by members of the committee. We also managed to get a reporter from Computing to cover one of our technical meetings; surely a first for the BAA.

Finally I would like to extend my personal thanks to our energetic Secretary Graham Parkhouse, whose hard work and enthusiasm have been an example for us all.

3. Treasurer's Report

The Treasurer, Phil Goacher, presented draft accounts for the year. The accounts have yet to be audited and approved by the BCS, and will thus appear in full in a subsequent issue of Vector.

The draft accounts showed a reduced membership income for the year, and the Treasurer commented that the membership appeared to have reverted to pre-APL '86 levels. Actual members at April 1989 were:

	UK	Overseas
Ordinary	273	119
Corporate	24	10
Sustaining	10	
'Free'	6	16

'Free' memberships were those organisations to whom we sent copies of Vector, usually overseas APL groups.

In answer to a question from the floor, the Treasurer could not identify any membership enquiries resulting from press coverage such as the articles which had appeared in Computing.

Other areas of the accounts showed reductions in expenditure in recruitment and publicity, tied to the level of activity in these areas. Vector activities showed an approximate £1,880 loss (against a £800 profit in the previous year). This was mainly due to including the costs of 'Education Vector' under the main Vector heading. The costs of mailing Education Vector were estimated at £1,200 by the Treasurer.

The late issue of Vectors 5.2 and 5.3 had meant that the BAA debtors were significantly increased. The Treasurer stated that the invoices outstanding included some £2,000 from APLication which he was confident could be collected and some £800 from Vector advertising which had only just been issued.

Overall, the BAA funds had increased during the year, helped by investment income.

4. Election of the 1989/90 Committee

Only one post had been contested - that of chairman. The results of the election were read by Phil Goacher.

2 spoiled papers

Anthony Camacho 123
Peter Donnelly 146

Peter Donnelly was therefore declared elected as the 1989/90 Chairman.

Other members were elected unopposed:

Graham Parkhouse	Secretary
Adrian Smith	Journal
John Sullivan	Treasurer
Alan Sykes	Education
Peter Branson	Activities
Jonathan Martin	Publicity
John Searle	Projects
David Eastwood	Technical
Jill Moss	Recruitment

5. Any Other Business

Peter Donnelly announced that the 1989 Outstanding Achievement Award had been made to British Airways for their AIMS Management Information System. A formal presentation ceremony would be held at Heathrow. This ceremony would be open to interested members of the BAA, as well as the press.

The meeting closed at 14.45.

Incoming Committee Statements

The incoming BAA Committee were introduced to those present at the AGM. Graham Parkhouse and Adrian Smith were not present, but the other committee members made brief statements about their planned activities for the forthcoming year. Each Committee member asked for suggestions about what the members wanted. Extracts from these statements follow:

Peter Branson (Activities)

Peter canvassed attendees for suggestions and ideas on future activities and stressed the importance of the membership making suggestions for meetings and talks. Amongst the ideas mentioned were: another Vendors meeting; progress on APL standards; meetings for educationalists; review of second generation APLs.

David Eastwood (Technical)

David stated that he would continue to be involved with the planned statistics conference, and also that he saw the Technical Officer as filling a support role for the Activities and Journal Officer.

Alan Sykes (Education)

Alan said that he hoped to continue his activities as last year. A number of initiatives were only now bearing fruit - notably publication of articles covering APL. He would continue with Education Vector and was involved in organising the Statistics Conference.

Peter Donnelly (Chairman)

Peter saw a major part of his role in 1989/90 as ensuring that the BAA administration improved, and also that Vector appeared more regularly.

Jonathan Martin (Publicity)

Jonathan hoped to ensure that BAA meetings were included in the listing sections of the computer press and to try and recover lost membership.

John Sullivan (Treasurer)

John also saw the drive to regain lost membership as an important part of his function.

Jill Moss (Recruitment)

Jill endorsed the comments of Jonathan and John and stated that she hoped to turn around the recent trend in membership levels.

John Searle (Projects)

John hoped that his year of office would see the BAA funds halved and the members happy with the way the money had been spent.

News from Sustaining Members

Dyadic Systems Ltd

Dyadic is pleased to release more details of its forthcoming implementation of Dyalog APL for MS DOS.

This is a full implementation of Dyalog APL designed to run under MS-DOS 3.3 (or higher) on any 386-based PC or PS/2. It requires a minimum of 2Mb RAM, but supports optional demand paging so that the size of the workspace can exceed the physical memory installed. An 80387 maths co-processor is strongly recommended.

The new Dyalog APL Screen Manager based on `□SM` (Screen Map) and `□SR` (Screen Read) is included in the DOS version. It has been implemented using a direct memory map of the display, so screen updates are extremely fast.

One of the most important features of standard Dyalog APL is the ease with which Auxiliary Processors can be written in high-level languages. This feature has been extended to the DOS version so that existing Dyalog APL APs can be ported with little difficulty. The AP interface also includes additional facilities designed to improve performance and functionality. For example, you can now access workspace variables and even execute APL expressions from within an AP.

Naturally, the component file system supports standard MS DOS networking protocols, and there are facilities for accessing DOS native files. A built in APL terminal emulator is also included.

Graphics support is provided through an interface to GSS*CGI. Not only does this product provide a rich set of graphics primitives, but device drivers are available for a very wide range of displays, printers and plotters. The interface, which was originally developed for the Xenix version of Dyalog APL, allows you to draw several objects with one command using nested arrays.

Dyadic is designing a new development environment/editor which will appear first in the MS DOS version, but will later be available on all implementations of Dyalog APL. This will be a window-based debugger/editor that is fully integrated with the APL session manager.

Alpha-test copies of Dyalog APL for MS DOS will be distributed during May. The production version will be shipped in August. The software will not be copy-protected, and an affordable run-time license will be available. Final pricing will be announced later.

MicroAPL Ltd

MicroAPL is continuing to expand its consultancy activities, with an increasing amount of non-APL work. We have been actively recruiting new consultancy staff and still have vacancies in this area. APL work undertaken in recent months includes Securities analysis, local government contracts and real time management information systems. Non-APL work has included PC network consultancy and installation as well as the provision of systems written in C, Assembly Language and work with some of the modern database software for micros.

APL software sales continue to grow. MicroAPL's commitment to selling good APL software at fair prices continues to be supported by our clients. STSC's APL*PLUS II has proved successful, as well as the recent APL*PLUS PC version 8 upgrades. Sales of APL.68000 for the Apple Mac provide evidence of the growing inroads this computer is making into the academic and business communities - in some cases replacing PCs for established applications.

Following our release of APL.68000 version 7.20 for the Apple Macintosh version, we have upgraded a number of other APL.68000 implementations to version 7.20. Users of our Mirage-based machines such as the Spectrum and the Aurora will be pleased to hear that version 7.20 is available for their machines - with the most significant feature being the inclusion of the EDIT full screen editor into APL.68000 as a system function (and command). We have also started shipping version 7.20 for the Wicat MCS and Unix versions and will shortly be announcing a release date for the Commodore Amiga.

Cocking and Drury Ltd

Welcome to Richard Green and Adam Marengos who have recently joined the company. Richard is a statistics graduate from Swansea University who has just completed his PhD in Operational Research which involved graph theoretic modelling techniques for pallet packing and stock cutting. Adam joins as our new administrative assistant.

The success of APL*PLUS Mainframe and the APL*PLUS II System has made it necessary to expand the APL*PLUS support team, and we are now implementing our plan to recruit in this area.

Release 2 of the APL*PLUS II System for 80386 based machines is currently being beta-tested, and we expect to start delivering in August. The new release will include quad-NA and will now produce results when functions derived from the 'each' operator are applied to empties - two frequently requested features. Release 2 will also provide an easier migration path from APL*PLUS PC, as well as many further improvements.

Under the support agreement with our existing II System customers, all upgrades are automatic and free.

Run-time and corporate run-time deals are now available for the APL*PLUS II System.

New releases are also planned for the APL*PLUS Mainframe, Vax and Unix interpreters, as well as Release 9 of APL*PLUS PC later this year, demonstrating STSC's commitment to connectivity.

I P Sharp Associates

Gottfried Bach, I.P. Sharp Regional Manager for Germany, Austria and Switzerland, is promoted to the post of Manager, I.P. Sharp Europe, based in Frankfurt. He also becomes Manager, Global Limits Control System, with worldwide responsibility. Mike Butler, Product Manager, Global Limits Control System, is promoted to the post of Manager, I.P.Sharp UK, based at our Coventry office.

APL People Ltd

This past financial year has been our most successful yet, with a record number of people being placed in new jobs and a steady growth in consultancy.

Most of our existing contracts have been renewed and new work has been secured for the Company (in FOCUS now as well as APL). We currently have several large projects underway and are always interested to hear from APLers looking for contract work.

On the recruitment front, there has been a lot of activity recently, and we are expecting the momentum to increase as Summer approaches and more people contemplate a move. APL People offers free career advice and assistance to anyone in the APL community. We are also happy to advise 'first-time' contractors on such things as Tax and National Insurance and how to go about setting up a Limited Company.

Jill Moss has recently been elected to the post of Recruitment Officer for the BAA and should, by the time this is published, have instigated a number of campaigns for increasing the membership of the Association. We would like to take this opportunity to welcome any new members reading Vector for the first time.

Finally, our sister company, Applied Production Logic, is now marketing the completed multi-user version of its Production Control software and has already had orders from a number of prestigious clients in the engineering industry.

**THE
EDUCATION
VECTOR**

April 1989

Edited by Alan Sykes

Reprinted from VECTOR Vol.5 No.4. VECTOR is the Quarterly Journal of the British APL Association; for more information about the British APL Association, contact the Secretary: Graham Parkhouse, Dept. of Mech. Eng., University of Surrey, GUILDFORD.

Contents

Editorial	Alan Sykes	28
Puzzle Corner	Alan Sykes	29
Letters	Dave Piper, Les May	30
The Latest I-APL News	Anthony Camacho	31
Understanding Randomness using APL	Alan Sykes	34

Dr Alan Sykes,
Dept of Management Science and Statistics,
University College of Swansea,
Singleton Park, SWANSEA SA2 8PP

The Education Vector

by Alan Sykes

It is now nearly a year since I was elected to the committee of the British APL Association. What a busy year it has been. I suppose it is an axiom of present-day life that there is never enough time to do all the things one would like to do, and I'm sure that this is true of those of us in education who are interested in APL.

One consequence of this, as readers of Education Vector will have observed already, is that my articles tend to be biased towards the application of APL to statistical matters. There is no doubt in my mind that APL serves the statistician very well. It is also a marvellous language for use in Education, and my article this issue is very much an educational experiment - to try to present the essentials of Statistics both at the level of number crunching (data analysis) and also at a more theoretical level by presenting, in parallel, tools that aid the understanding of the models which underpin the data analysis.

This, in retrospect, is rather an ambitious thing to try and I can only hope that you, the reader, will give me feedback on the success or otherwise of this experiment. Ideally, it would be nice to think that universities were (as they used to be) places where academics are encouraged to try bold experiments in education - I could then justify a sabbatical to sit down and write a textbook! Sadly, if there is no financial benefit to the university involved, one can forget such pipe dreams.

At the Mathematical Association Conference at Sheffield shortly after Easter, fears were expressed that the advent of the national curriculum would stifle experiments in mathematical education from primary school upwards. Nevertheless, there was at the higher levels of mathematics teaching, an acceptance of the existence of powerful computing tools that must inevitably influence the teachers' approach. It was good to be able to give a live demonstration of the power of APL to a group of teachers who had not met APL before.

I also found it very instructive to listen to talks on the use of PROLOG and DERIVE. PROLOG is of course a 'logic crunching' language, and DERIVE allows the user to perform elementary algebraic manipulations including differential and integral calculus. I couldn't help feeling that it would be nice to have such tools in the APL environment and, though my colleague Allan Prys Williams can supply the former, I know of no computer algebra package in APL. Any offers?

In this Education Vector, I have proposed a problem for all readers, be they experienced APLers or relative newcomers. This is to the former by way of a gentle reminder that there is a whole class of computations that does not always fall easily into the APL

environment. I would like to see a more open discussion on ways of proceeding with such problems - suppression of such topics does APL a disservice in the long run.

Recently, I received a letter from Jaromir Antoch in the Department of Statistics at Charles University in Prague. I find it very encouraging and exciting to know that Education Vector finds its way to many different countries. I extend a warm invitation to all readers to submit notes, letters and articles for inclusion in forthcoming issues.

Puzzle/Problem Corner

In keeping with the theme of 'Understanding Randomness through APL' consider the age-old 'coupon-collecting' problem. I believe 'cigarette cards' in tea-packets etc. are coming back into fashion and so the topic has some relevance to everyday life!

Suppose there are six different cigarette cards in a set, and that each packet of tea contains one of the six cards chosen at random. How many packets (on average) do we need to buy to get the whole series? I know of course that there is a very neat theoretical solution to this problem that is easily computed in APL - the answer is $6 \times + / \div 16$ which equals 14.7.

The problem here is to apply APL to simulate this random process. Readers are asked to submit APL solutions that provide one simulation and then to consider how to provide a sufficiently large sample of simulations efficiently, so that an average value of about 14.7 can be derived. Submissions that present more than one method together with an appraisal of their relative merits are encouraged.

(Editorial Note: I was reminded of this problem at a workshop on simulation at the Sheffield conference - at the time I only had a BBC micro with BASIC at my disposal and it took me about an hour to write a program and produce 14.6 as the average of 1000 simulations!)

Letters

From: Dave Piper

18th April 1989

I was interested to read in today's Independent that the Soviet Union will be installing 3 million PCs over the next five years. Surely this represents an opportunity for APL that we cannot afford to miss. Of course, this would give the Soviet Union such a tremendous advantage in 10 or so years time that COCOM may have something to say about the export of such advanced technology. Frankly my dear, I don't give a damn!!

D.B. Piper

41 Sandown Drive, Rainham, Kent ME8 9DT

From: Les May

22nd March 1989

I have just received my first copy of VECTOR and would like to take the opportunity both to thank the APL community for giving I-APL to education and to support Anthony Camacho's comments about the choice of keyboard layout adopted by the project. As a total beginner with I-APL I have found much of the keyboard easy to remember without key stickers, using the same aids to memory that he lists.

Incidentally the problem for us beginners isn't the notation, it's learning to 'think' APL. Even passing acquaintance with procedural languages makes it difficult to unwind the mind from 'do it one at a time' and to start thinking 'do it all at once'. Children will not come to I-APL with any such preconceptions. Perhaps there is a gap in the market for a small book which assumes that the user knows what one does with a computer, e.g. input and verification of data, sorts, selects, etc., but explains how an APLer would do it.

What I would have found very useful with my first VECTOR was a short glossary of APL terms. You may know what APL2 and DIALOG are, and how they extend APL, but it was a mystery to me until I found someone to ask.

I would also be interested to read what people are using APL for. Because I'm involved in education, the examples I know are drawn from that field: an ethnographer using it to process Navaho text in about as much workspace as I-APL provides, a historian using it to convert £ s d to decimal currency, and biologists using it to study bird migration and connectivity of food webs. Given the current emphasis on links between education and industry, demonstrating the commercial importance of APL would seem worthwhile.

Les May

20 Crescent Road, Rochdale, Lancs

I-APL NEWS: The State of the Project

by Anthony Camacho

We must not be tempted, now that we have issued good BBC, RML Nimbus, PC and Archimedes ports, to rest on our laurels.

Implementations

For these machines and the PC we now have complete implementations which offer access to all the facilities of the machine, to native files, graphics, colour and sound. The PC version of I-APL now takes note of DOS environment settings such as 'SET IAPL=/H' for the Hercules adaptor. This saves remembering the correct startup options for your particular machine.

Workspaces

We also have a growing collection of useful workspaces. We are very conscious that these are of varying difficulty and that there is no adequate printed guidance to help a teacher to use them effectively. Since I-APL has not validated them, edited them or even checked the robustness of the code, they are offered through the British APL Association software library. Such software has no guarantees of suitability or quality, and without such guarantees is much less attractive to teachers.

Would You Like to be Software Editor?

The BBC and Archimedes machines cannot easily use workspaces which are issued on PC compatible disks. Either the software library must begin to handle non-PC disks or we need simple instructions how to transfer workspaces from a PC to a BBC or Archimedes (including how to wire a suitable cable!). Would anyone be willing to do this? Would anyone like to take on the job of editing, indexing and explaining the workspaces to a consistent standard? A page of clear explanation per workspace could make all the difference whether a teacher felt it worth the trouble of trying it out or not.

Books

There are now also the essential books. The second edition of the I-APL Instruction Manual is being issued. It covers the RML Nimbus, BBC 'B' and Master and Archimedes as well as PC clones. The first edition of 'An APL Tutorial' is exhausted and the second edition is being supplied: there are a few improvements but the second edition is substantially unchanged. There is still a small stock of 'An Encyclopedia of APL' and the

second edition is camera-ready and will go to press soon, so that orders will not be delayed for lack of it. First editions are now accompanied by a list of errata (only one side of A4) which may help to avoid difficulties. We now also have 'APL in Social Studies' by Tama Traberman - a new booklet based on Tama's experience teaching with APL in New Jersey.

The Road Winds Uphill All the Way

We have always said, and believed, that the hardest part of the I-APL project would be getting people to use APL in education and that overcoming the first hurdle of making it possible for teachers to afford an interpreter does not count as success. In fact we have not even overcome the first hurdle except in the UK and Denmark. I-APL is not yet available on the common home and school computers in the USA or Germany or France (but porters are working hard at the job and we hope and expect to be able to complete more versions this year).

What Help Do Teachers Need?

Without underestimating the work that still has to be done to get I-APL available on the main remaining school microcomputers, I feel we should now address the problem of persuading more teachers to get to know and use it. The main difficulty most of us have with this is that we don't really know what it is that would be most effective in helping and encouraging teachers to use it. We have to ask the teachers and provide what they ask for. And we probably have to get other teachers to help.

Let us Publish Teaching Material

Perhaps the best help and encouragement that can be provided is teaching material that has been used successfully, clearly explained and fully documented. It also needs to be cheap. For example Tama Traberman's 'APL in Social Studies' is clearly the product of practical teaching experience. The material is graded so that it does not demand too much of pupil or teacher at any stage, and the text is clear and well presented. It is also short. A teacher can read through it without having to give up a weekend.

We want more of this kind of material. We want more reports of use in the classroom like Bruce Dickson's. We want to establish, in every teacher's mind, that APL has a low threshold and a high ceiling - that it can be used at an elementary level and is easy to introduce and that there is practically no limit to the heights you can reach with it. So reports of use in higher education too will be valuable. Knowing that what is taught at an elementary level in school is likely to be built on and valuable to a student in an advanced course will certainly not discourage schools from introducing APL to their pupils.

Can YOU Help to Get Teaching Material into a Publishable Form

So, please, if you are a teacher and experiment with APL, write up your work and allow me to publish it. The British APL Association is keen to publish anything about the use of APL in schools or colleges in the Education Vector. I am keen to publish anything with sufficiently wide appeal to sell a thousand copies. This is not restricted to I-APL. Any variety of APL is acceptable, though, of course the less use is made of extensions beyond the ISO standard, the wider the potential appeal. If you are an APLer who knows a teacher, please help and encourage them to introduce APL to their pupils and offer to write up the experience (or the course) for publication in an article or a booklet.

I pay royalties, organise typesetting and printing and sell copies to authors at cost. Publishing your course this way may even be the cheapest way of getting the course notes copied in sufficient volume to supply a dozen classes.

For examples please see the I-APL Manual, the Tutorial in APL and APL in Social Studies. Anything from 24 to 96 pages can be produced economically in the same style.

Understanding Randomness Using APL

by Alan Sykes

Introduction

My aim in this article is to use APL and some of the tools introduced in previous articles to explore randomness. As I showed in the first of my articles, quite surprising patterns can occur where random phenomena are concerned, and in a sense that's what Statistics is all about - order from chaos.

Many people find Statistics very difficult to follow. One of the reasons for this is the need to think on two planes at the same time. First there is the statistical recipe - what you do with the data. But to understand why the recipe makes sense, the theoretical model for randomness also needs to be studied. Through the interplay between recipe and theoretical model sense can be made of the results of applying the recipe. In this article I hope to integrate both planes of thought through the use of APL.

Throwing a Fair Die

Let's start with throwing a die (singular of dice). APL allows us to simulate throwing a die using ? - e.g.

```
THROW+?120p6
.5 1 FREQTAB THROW
1 22
2 15
3 22
4 14
5 19
6 28
```

Can we make sense of these results? If the die is well-behaved, then on each throw, each of the six possible results are equally likely. This means that if we denote the outcomes by the vector $X+1\ 2\ 3\ 4\ 5\ 6$ then the associated probabilities must be given by the vector $P+(6p1)\div6$. The interpretation is that on one throw the probability of obtaining a 1 is one sixth, and the understanding is that in a large number of throws the proportion of 1's will be about one sixth. Equivalently, we can say that the expected frequency of each possible number in 120 throws is 20.

Some care must be taken to interpret the word 'expected' correctly. It does not mean that the observed number of 1's equal to 22 is wrong - what it does mean is that if we

repeatedly took samples of 120 throws of a die, then the number of 1's in each sample would be a random number with an average value (in the long run) of 20.

Can we make any other observations concerning expected frequencies? If each of the six possible outcomes is equally likely, then we would expect one half of all observations to be even, i.e. 60 even numbers out of the 120 total. The observed number of even numbers in THROW can be calculated from the frequency table or directly by the expression

$$+/T = LT + THROW \div 2$$

57

(This simple expression uses $\downarrow$ to round down to the next smallest integer and also employs T as a temporary variable - a useful trick.)

To proceed further, we need to be able to calculate the average value or sample mean. The function MEAN is probably the most often quoted APL function:

```

      VR←MEAN DATA
[1]  R←(+/DATA)÷ρDATA
      V

```

Alternatively using the direct definition, we write:

$$MEAN: (+/\omega) \div \rho \omega$$

Applying MEAN to THROW we have:

```

      MEAN THROW
3.6416667

```

Corresponding to this value is the expected value for the throw of a die which can be calculated using the vectors X and P previously defined. Remembering that P contains the expected proportion of the throws that equal the values 1 2 3 4 5 6, it is clear that the expected value for a throw of a die is given by the sum of the probabilities times the values, namely $+/P \times X$ or $P +. \times X$, giving in this case the value 3.5. This idea is embodied in the expected value function given by:

```

      VR←P E X
[1]  R←(,P)+.×,X
      V

```

or

$$E: (, \alpha) +. \times, \omega$$

This makes it possible to emulate the mathematical notation very closely. For example, using a pair of redundant brackets, we may write:

P E(X) 3.5

Try convincing yourself that this calculation tells you what would happen in the long run by statements such as MEAN ?100006.

Let's consolidate the discussion so far by another simple example. Suppose our die was not fair, and that a six was three times as likely as the other values. At the theoretical level the probabilities P must be calculated using $P+1 \ 1 \ 1 \ 1 \ 1 \ 3 \div 8$. Using our expected value function E , we now get the expected value equal to 4.125:

P E(X)
4.125

This calculation tells us to expect a sample mean of around 4.125 when such a die is used. Let's see if this is the case. To simulate throws of the new die, let the vector $A = 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 6 \ 6$. It is clear that if we sample at random from A , then we will have three times the chance of selecting a 6 as any other number.

[illegible]

The bar chart shows us that the simulation works, giving an excess of 6's over the other values. Calculating the mean of THROW:

MEAN THROW
4.29

... gives a value quite close to that expected.

Simulating a Biased Die

The simulation of the biased die poses a general problem: given a set of probabilities P associated with a set of values X , how can we choose numbers from X according to the probabilities specified by P ? One possibility is to represent the probabilities in P correctly, say, two decimal places, and then construct a vector A which contains the values in X repeated $100 \times$ times; i.e. an extension of the idea used already.

The more correct procedure involves using numbers which are randomly chosen between 0 and 1 - the so-called Uniform Distribution. Consider the APL function RND:

```

      V←RND N;T
[1]  R←(¯0.5+?NpT)÷T+100000
      V

```

or

```
RND:({¯0.5+?ωpT)÷T+100000
```

Then RND 3 produces 3 numbers chosen at random between 0 and 1

```

      RND 3
0.845315
0.351765
0.945705

      BAR 0 .1 FREQTAB RND 100
0.05 | ██████████
0.15 | ██████████
0.25 | ████
0.35 | ████████████████████
0.45 | ████████████████████
0.55 | ████████████████████
0.65 | ████████
0.75 | ████████
0.85 | ██████
0.95 | ████████████████

```

To choose an element of X according to the probabilities in P, we proceed as follows:

- select a random number r between 0 and 1;
- determine n such that there are at most n of the elements of $+ \setminus 0, P$ less than r;
- choose $X[n]$.

For example, suppose $P \leftarrow (1 \ 2 \ 1 \ 1 \ 1 \ 4) \div 10$, so that if $X \leftarrow 1 \ 2 \ 3 \ 4 \ 5 \ 6$, then a six is twice as likely as a 2 which in turn is twice as likely as a 1, 3, 4 or 5. To choose a value of X according to the probabilities in P we can write:

```

      X[+/(RND 1)>+ \0,P]
3

```

This does not demonstrate that the probabilities are being adhered to. To do this we need to repeat this procedure many times. The easiest way to do this is to use the 'jot-dot' construction we used in my first article on counting.

though of course they could both equal the same numeric value. However the distribution for each throw is the same, and specified by $X1\ 2\ 3\ 4\ 5\ 6$ and $P(61)6$.

What of the distribution of both scores? Here, APL fails us (at least at the first generation level) because the possible outcomes can be displayed in the form

```

1 1 1 2 1 3 1 4 1 5 1 6
2 1 2 2 2 3 2 4 2 5 2 6
3 1 3 2 3 3 3 4 3 5 3 6
4 1 4 2 4 3 4 4 4 5 4 6
5 1 5 2 5 3 5 4 5 5 5 6
6 1 6 2 6 3 6 4 6 5 6 6

```

which ought to be achieved by $X \circ ., X$ (because 1,2 catenates 1 and 2 together to form the vector 1 2 for example).

The probabilities associated with each of these outcomes is however calculated with the expression $P \circ . \times P$ (assuming that the two throws are 'independent'), consisting of a 6 by 6 matrix of values equal to one thirty-sixth.

However, if we wish to look at the distribution of an actual numerical value derived from the two throws, then APL provides us with very elegant and satisfactory tools to perform the distribution arithmetic. As a first example suppose S is the sum of the two throws. Then the set of possible values (with repetitions) is given by $X \circ . + X$ and the associated probabilities are given by $P \circ . \times P$ as before. The distribution of S is then given by summing up all the probabilities associated with equal values in the array $X \circ . + X$.

To help us in this section, we need two further functions. First $PBAR$ is a simple alteration of BAR so that it will display a bar chart of probabilities rather than frequencies, obtained from BAR by multiplying the probabilities by 80 (for an 80 character screen). Secondly we need an equivalent of $FREQTAB$ that accumulates probabilities associated with the same numerical values. The functions are as follows:

```

▽ B+PBAR F
[1] B←( ¯F[F[;1]),' |', ' □'[1+F[;2]∘.≥1[ ¯F[;2]+1.5+80×F[;2]]
    ▽

▽ R+P PROBTAB X;Z;V;PR;I
[1] Z←,X ⋄ Z←Z[Z] ⋄ V←(1+Z×¯1ϕZ)/Z ⋄ R←1,1+Z ⋄ +(0=1+I)/0
[2] PR←+/(V∘.=,X)×((ϕV),ϕ,X)ϕ,P
[3] R←V,[1.5]PR
    ▽

```

Now we can investigate the distribution of the sum of two throws of a die with X and P as last defined.

```
⊞PP÷4
```

```
(P°.×P) PROBTAB X°.+X
```

```
2  0.02778
3  0.05556
4  0.08333
5  0.11111
6  0.1389
7  0.1667
8  0.1389
9  0.11111
10 0.08333
11 0.05556
12 0.02778
```

Note that the probability that the sum is equal to 7 is one sixth, as out of the 36 possible equally likely values for the outcomes of throwing a die twice, there are six of them such that the sum of the two scores is 7.

Further,

```
PBAR DIST←(P°.×P) PROBTAB X°.+X
```

```
2 | 
3 | 
4 | 
5 | 
6 | 
7 | 
8 | 
9 | 
10| 
11| 
12|
```

... and we see the pattern of increasing and decreasing probabilities that we derived from simulation in the first of my articles on Counting in APL. Alternatively we can produce a simulation of this distribution using:

```
+/?100 2p60
```

and so

```

      BAR 1.5 1 FREQTAB SUM+/?100 2p6
2 | 0000
3 | 0000
4 | 0000000000
5 | 000000000000
6 | 000000000000000000000000
7 | 0000000000000000
8 | 00000000000000000000
9 | 00000000
10 | 00000000
11 | 00000000
12 | 0

```

Note the similarity in the two bar charts. Further note that `MEAN SUM` gives the answer 6.84, quite close to the value of `DIST[;2] E DIST[;1]` which of course gives the answer 7. The fact that the distribution of the sum of two throws peaks about the middle value is due to a very important mathematical theorem, called the Central Limit Theorem, a consequence of which is that the distribution of the sample mean is nearly always more closely centred around the theoretical mean than the distribution of just one observation. For example, compare the distribution of one throw, displayed by

```

      PBAR P PROBTAB X
1 | 0000000000000000
2 | 0000000000000000
3 | 0000000000000000
4 | 0000000000000000
5 | 0000000000000000
6 | 0000000000000000

```

and the distribution of the mean of three throws, given by

```

      PBAR (P°.xP°.xP) PROBTAB (X°.+X°.+X)÷3
1
1.333 | 0
1.667 | 00
2
2.333 | 000000
2.667 | 00000000
3
3.333 | 0000000000
3.667 | 0000000000
4
4.333 | 00000000
4.667 | 00000000
5
5.333 | 00
5.667 | 0
6

```

The Central Limit Theorem tells us that, barring a few pathological examples, no matter what shape the distribution of one observation is, the bell-shaped curve of the Normal or

Gaussian distribution will appear as you look at the distribution of the sample mean, providing the size of the sample is large enough. Let's take one more example of the calculus of distributions. Suppose, instead of calculating the sum of the two scores, we calculate the maximum.

```
PBAR  $\square$ +DISTMAX+(P $\circ$ .xP) PROBTAB X $\circ$ .fX
1 0.02778
2 0.08333
3 0.1389
4 0.1944
5 0.25
6 0.3056
```

```
1 |  $\square\square\square$ 
2 |  $\square\square\square\square\square\square$ 
3 |  $\square\square\square\square\square\square\square\square\square\square$ 
4 |  $\square\square\square\square\square\square\square\square\square\square\square\square$ 
5 |  $\square\square\square\square\square\square\square\square\square\square\square\square\square\square$ 
6 |  $\square\square\square\square\square\square\square\square\square\square\square\square\square\square\square\square\square\square\square\square$ 
```

Not surprisingly, larger probabilities are given to higher values.

The expected value is given by `DISTMAX[;2] E DISTMAX[;1]` and equals 4.472. If we simulate this example with the expression:

```
MAX+f/?100 2p6
BAR .5 1 FREQTAB MAX
1 |  $\square\square\square\square$ 
2 |  $\square\square\square\square\square\square\square$ 
3 |  $\square\square\square\square\square\square\square\square\square\square$ 
4 |  $\square\square\square\square\square\square\square\square\square\square\square\square\square$ 
5 |  $\square\square\square\square\square\square\square\square\square\square\square\square\square\square\square$ 
6 |  $\square\square\square\square\square\square\square\square\square\square\square\square\square\square\square\square$ 
```

we see that again, the correspondence between theoretical model and simulation is close. Further `MEAN MAX` gives the sample mean of 4.32.

So, to round off, we have demonstrated some of the patterns that can occur when randomness is explored, and in particular shown how APL provides superb tools for the calculations involved with the theoretical models. However I must add one slight word of caution: all the simulation results shown here were to some extent chosen because they were close to the results predicted. If you try them for yourself, you will find that randomness may sometimes hide the patterns. For example, in the distribution of the maximum of two throws of a die the probabilities of 1 2 3 4 5 or 6 are increasing. This pattern may well be absent from the simulation results! So perhaps Lady Luck has the last word after all.

APL Product Guide

Compiled by Alison Chatterton

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Timesharing Services
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to:

BAA Administration,
c/o Alison Chatterton,

at the address shown on the inside back cover

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Dyadic	APL Coprocessor	3,500+	32-bit coprocessor board for IBM PC. NS32000 cpu with FPP, up to 4Mb RAM, 16Mb virtual memory. Software includes Unix V.2, Dyalog APL, graphics support, DOS interface. Provides multi-user Unix/DOS environment.
	IBM 6150	15,000+	Multi-user Dyalog APL system with Fast 32-bit RISC processor, FPP, up to 8Mb RAM, 210Mb Disk, 16 users. Interface to SQL, graphics and APL support for standard IBM peripherals.
	Altos 3068	25,000+	Multi-user Dyalog APL system with MC68020 cpu & MC68881 FPP. Also features a LAN which supports IBM PCs as Dyalog APL terminals.
	Sun 3	15,000+	Multi-user Dyalog APL systems which can be configured as a network of workstations and or a traditional time-sharing cpu. With its 25MHZ 68020 cpu, the Sun 3/200 is the fastest APL microcomputer on the market.
Inner Product	IBM PC	2,000-6,000	IBM PCs supplied for turnkey applications
M.B.T.	MBT Series 10	poa	UNIX/68010 based multi-user APL system
	TORCH	poa	68000/280 multiprocessor
MetaTechnics	-	poa	Details on application -- IBM PC compatible
MicroAPL	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	Spectrum	7,000+	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	Spectrum/386	2,000+	80386-based PC to run APL*PLUS II. Fully expandable.
	Atari 1040ST	799	1 Mb Mono/Colour System, includes 1 Mb disc drive & mains transformer built into Console.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES (£)	DETAILS
Mercia	APL*PLUS	poa	STSC's Mainframe Service - MAILBOX etc.
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.
Uniware	APL*PLUS	call	STSC's mainframe service

APL INTERPRETERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Software	Dyalog APL	1,000-8,000	See Dyadic Systems entry
Cocking/Drury	APL*PLUS/PC Rel 8	475	STSC's full featured APL for IBM PC, PC/AT and PS/2. Now with virtual workspace, quad-MF, GSS graphics, Network and VGA support. Includes session ring editor.
	Run-time	poa	Closed version of APL*PLUS/PC which prevents user exposure to APL
	APL*PLUS II	2,000	includes 1st year maintenance
	runtime	250	
	site licence	poa	
	APL*PLUS UNX	poa	STSC's 2nd gen hi-performance APL for IBM PC/RT, Sun, DEC, HP and major Unix computers and workstations.
	APL*PLUS VMS	poa	STSC's 2nd generation APL for DEC VAX computers running under VMS. Comparable in feature to APL*PLUS II system, APL*PLUS Unix, and APL*PLUS mainframe.
	APL*PLUS mainframe	poa	STSC's 2nd gen APL interpreter. Enhances VS APL with component files, nested arrays, formatting, user command processor, fast/ins and other productivity features. For VM/CMS, MVS/TSO: offers simple upgrade from VS APL.

Dyadic	Dyalog APL	795-10,000	2nd gen. APL for UNIX systems, e.g. IBM 6150, Sun, Vax, NCR, HP9000, AT&T, Altos, Apollo, Whitechapel, Sperry, etc.
Gen. Software	APL*MYRIADE	poa	Runs on Texas Instruments TI990 range.
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411. From all IBM dealers.
	Mainframe APL2	poa	APL2 release 3
	APL2 Appl. Env't	poa	
Inner Product	VIZ::APL	250-350	8-bit Zilog Z-80 CP/M
	APL*PLUS/PC	600	See under Cocking & Drury
M.B.T.	Dyalog APL	poa	See Dyadic Systems entry
	MBTAPL	poa	Enhanced Dyalog APL for MBT hardware.
	VIZ::APL	poa	Customized for TORCH hardware
MetaTechnics	APL*PLUS Rel 6	475	Discount on quantity.
MicroAPL	APL.68000	1,000-2,000	Full implementation with component files, error trapping etc. for SPECTRUM, HP300, SUN, NCR etc.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	APL.68000 Apple Macintosh	87	
	APL.68000 Apple Mac II	260	
	APL.68000 Amiga	87	Full APL interpreters with support for windows, mouse, graphics etc.
	APL.68000 for Atari ST	87	
	APL.68000 for IBM-PC	995	
	APL*PLUS/PC - REL 8	425	
	APL*PLUS II	1,395	
Portable	PortAPL	\$195	IBM PC Software
		\$275	Mackintosh
		\$2,995	DEC VAX
I.P. Sharp	APL*PLUS/PC	480	STSC APL
	Upgrade V7 to V8	100	
	Upgrade any to V8	200	
	Sharp APL/PCX V1	1,650	For IBM XT/AT 370
	Sharp APL/PC V2	40	shareware for IBM PC/XT
	Sharp APL	poa	for IBM mainframes
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers.
	PortaAPL	280	PORTABLE SOFTWARE's APL for APPLE MACINTOSH.
		2,995	PORTABLE SOFTWARE's APL for the DEC VAX.

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	Lynwood J300	1,560	Monochrome ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	Lynwood J500	2,295	Colour ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	IBM 3163	791	Low-cost Monochrome APL vdu. Supports downloaded Dyalog APL font.
	IBM 3164	1,093	Low-cost Colour APL vdu. Supports downloaded Dyalog APL font.
Farnell	Tandberg TDV 2221	995	Ergonomic design APL terminal, 50-19200 baud, 15lfif5 anti-reex screen, low profile keyboard
	Tandberg TDV 2271	1,195	Combined APL/ANSI ergonomic terminal as above.
Gen. Software	Mellordata	400	Second-hand Elite 3045A
M.B.T.	various		Contact MBT for details

Meta Technics	IBM EGA compatible	299	Emulates EGA & Hercules, Half Card
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1,450	With monochrome graphics
	Concept201	1,295	APL VDU with 8 page memory
	Concept 201G	1,650	Graphics VDU
Shandell	HDS2010	1,215	ANSI/3.64 DEC VT52/100/220 compatible, 15" tilt/swivel screen, low profile keyboard 8 page memory, windows, viewports, 80/132 columns, full overstrike, 2 or 3 comms, ports, 55 PF keys, NVM storage.
	HDS2010G/GX	1495+	As above plus Tektronix 4014, Retrographics VT640/D0640 and Visual 500 compatible. 1024 x 390 or 1024 x 780 resolution.
Tektronix	4114B	13,500+	19" D.V.S.T.:Graphics: 3120 x 4096 displayable; Intelligent: up to 800K memory; APL keyboard (option 4E)
	4125	21,550+	19" 2D colour graphics; Workstation (1280 x 1024);Intelligent: up to 800K memory; APL keyboard (mod AP)
	4128	26,822+	As 4125 plus 3D wireframe

APL PRINTERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Datatrade	Datasouth DS180+	1,295	180 cps matrix printer with 4K buffer, 9 x 7 dot matrix and APL option.
	Datasouth DS220	1,695	Letter quality; graphics capability, APL option (both available with IBM Twinex or Coax interface).
Dyadic	IBM 4201 Proprinter	poa	100, 200, 40(nlq) cps, matrix printer, with graphics. Supports downloaded Dyalog APL font.
	Toshiba P351	poa	24 pin high-quality matrix printer 100 cps letter quality, 192 cps draft.
Inner Product	Epson FX80	500	Soft char. set, 160 cps, 80 column
	Anadex 9620	1,150	200 cps., 132 col., tractor feed
	Siemens PT88	620	180 cps., 80 col., silent
	TGC Starwriter	1,180	40 cps., letter quality
M.B.T.	Facit 4565	poa	40 cps letter-quality
	Facit 4510/11/12	poa	Matrix printers
MetaTechnics	Queen-data	295	Low-cost APL Daisy-wheel printer
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Philips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL-385	APL-385	50(PC),125(mf)	including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	AFM/AP	11,035	Interprocess Software for VM/CMS & MVS/TSO.
	- Keyed Access	2,650	Component File Management System (VSAPL/APL2)
	- Interactive Link	1,325	
	- Mail Exchange	2,650	
	CALL/AP	4,030	Non-APL program execution (VSAPL/APL2)
	APLPRINT	2,205	Output to high speed line printer or 328x devices (VSAPL/APL2)

	ENHANCED FORMAT	2,205	Extends Format operator to full "Quad-FMT" status (VSAPL/APL2)
	ISP	750	Input and Output Stack Processors for manipulating terminal I/O
	OSP	2,205	with facilities for Error Trapping (VSAPL)
	DISPLAY CAPTURE	poa	Allows terminal output to be collected and held for retrieval by an APL function (APL2)
	UCF	poa	User Communication Facility for data transfer between users (APL2)
	RDS	poa	Relation Data Base System
	PANEL	poa	Fullscreen management system
	PFS	poa	Program File System - APL Systems development aid
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beta-plan	BETA-FONT	poa	Multiple font PC character generator. Dealers required for non-Scandinavian countries.
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Butel	Merlin	5,000	Mainframe APL spreadsheet runs under VM/CMS, TSO, VSPC
	Merlin/PC	poa	Version for APL*PLUS/PC
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for IBM VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	First APL compiler. Available with APL*PLUS enhancements and Sharefile under VM/CMS & MVS/TSO
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Convert non-APL files to APL files
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Powerful Information Centre spreadsheet incorporating data exchange between APL and FOCUS, IFPS, SAS, APL/DI, ADRSII, LOTUS123, VISICALC, MULTIPLAN, DIF files
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(microcomputer)	STATGRAPHICS Rel 2.6	645	Integrated Statistics and graphics on IBM PC, AT, PS/2 and compatibles
	Update 2 to 2.6	140	
	Update 1 to 2.6	330	
	APL*PLUS PC Tools	325	Utilities including: RAM disk, full screen data entry, menu input, report generation, file documentor, exception handling and games.
	IRMA Module	120	327x IRMA support.
	Fin & Stat. Library	350	Financial & statistical routines
	SPREADSHEET MGR	195	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII, LOTUS
E&S	PROTOPAK		Packages for prototyping management information systems - consisting of: PC & mainframe modules
	RMS		Relational databases.
	AMS		Multi-dimensional arrays
	RAMS		Combined RMS & AMS.

	BMS		Dynamic nancial modelling & forecasting
	FMS		Full-screen handler for APL*PLUS/PC. (AP124-based)
	CMS		Communications package.
	SOS	poa	Scheduled ordering and stock control.
FASTCODE	FASTCODE, MONITOR	\$239	Assembler written monitor for APL applications in APL*PLUS/PC
Gen. Software	PROPS	500+	Spreadsheet system for Product and/or Project Planning.
H.M.W.	INPUT	poa	Matrix manipulation package for data entry & report generation
	PRINTPAK	poa	Block printing for VM/CMS
	VIEWPAK	poa	AP124 Protocol emulator for IBM/PC
	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
Inner Product	Viewcom	150	Control Viewdata from APL
	APL/dBASE II	150	Interface APL with dBase II
	APL/dBASE III	150	Interface APL with dBase III
	APL/LOTUS	150	Interface APL with Lotus
	APL/WORDSTAR	150	Interface APL with Wordstar
	APL/MULTIPLAN	150	Interface APL with spreadsheet
	CEMAS	3,500	EEC monetary and agrimonetary analysis.
M.B.T.	RHOMBUS	poa	Integrated Ofsu3suc System
	HASLEMERE	poa	Hotel Accounting System
Mercia	STATGRAPHICS 3.0	685	Integrated stat. graphic system for PCs.
	Upgrade 2.1 to 3.0	395	
	Upgrade 2.6 to 3.0	175	
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC. Forecasting, Inventory Control, Scheduling, Distribution, etc
MetaTechnics	MetaScreen	99	Full-screen handler for APL*PLUS/PC, based on VSAPL AP124
	MetaPack	495	Comprehensive utilities package for APL*PLUS/PC. Includes: MetaScreen, MetaWS, Browse, Toolbox, Numeric Editor.
	APL-IEEE488	99	Controls IEEE488/GPIB Bus from APL*PLUS/PC.
	PLOT/PC	99	2D & 3D Graphics package. Includes interactive diagram Editors.
	Browse	99	Scrolling of DOS files, large APL variables.
	ADAPTA DLS	poa	Production & purchasing scheduling for process manufacturing.
	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	For APL*PLUS/PC product
	PC TOOLS Vol 1	295	
	STATGRAPHICS Rel 1	495	
	STATGRAPHICS Rel 2	535	
Parallax	ExecuCalc	\$5,000	Mainframe-based electronic spreadsheet for VM/CMS & MVS/TSO with links to micro products.

I.P. Sharp	ExecuPlot	\$5,000	Mainframe-based colour graphics with micro links.
	GLOBAL LIMITS	poa	Exposure management for banks
	IPSA/CONNECT	poa	Mainframe to micro link
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	65	Full screen front end to IPSA mailbox
	upgrade to V.2	15	
	NEWSFLASH	poa	Real time message exchange
Sugar Mill	VIEWPOINT	poa	4GL - Info centre product
	Stat 1	\$129.95	Statistical toolbox, menu driven
Uniware (mainframe)	STSC's ENHANCEMENTS	10,715	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	10,715	Component files for IBM VSAPL under VM/CMS and MVS/TSO and for IBM APL2
	PROGRAMMER TOOLS & UTILITIES	5,715	
	FILEPRINT	5,715	
	FILESORT	5,715	
	FILECONVERT	5,715	
	FILEMANAGER (EMMA)	5,715	STSC's database package.
	APL*PLUS COMPILER	21,430	First APL compiler. Complements APL*PLUS enhancements and Sharefile under VM/CMS and MVS/TSO.
	EXECUCALC	3,995	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL (VM or TSO).
	STATGRAPHICS	725	Statistics & Graphics for PCs.
	STATGRAPHICS FCA	140	An add-on module to STATGRAPHICS: Factorial Correspondence Analysis.
	APL*PLUS/PC TOOLS - VOL1	325	Incl. 327 x IRMA support, RAM disk, full screen data entry, menu input, report generation, games.
	- VOL2	125	Incl. File documentor, screen editor, exception handling.
	SPREADSHEET MNGR	250	APL spreadsheet with built-in ASCII, LOTUS and SYMPHONY interfaces.
(microcomputer)	APL*PLUS/PC FIN. & STAT.LIBRARY	350	Collection of financial and statistical utilities.
	POCKET APL	140	Smaller version of APL*PLUS/PC.
	UNIASM	275	Collection of assembler routines for APL*PLUS/PC users.
	UNITAB	240	APL*PLUS/PC spreadsheet-like data entry and validation system.
	The APL DEBUGGER	105	First released APL*PLUS/PC debugger.
	OVERLAYS	250	Fast assembler routines to handle overlays in APL*PLUS/PC.
	R:BRIDGE	380	Interface between APL*PLUS/PC & R:BASE 5000.
	DMA	380	A version of EMMA (APL database manager) for APL*PLUS/PC users.
	APL2C	295	Interface between APL*PLUS/PC and DATALIGHT C language.
	ADAPTA/DLS	33,333	Production & purchasing scheduling for process manufacturing.
Warwick University	BATS	500	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to 35 for academic institutions.
	FAB	free	Training program for the above.

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Consulting	Consultancy	180-550	Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation.
APL People	Consultancy	from 120	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL68000, C/Unix, TSO/MVS, VM/CMS, graphics. Additional skills available: OR, statistics, financial, commerce, retail, engineering.
APL Software Technology	Consultancy	poa	Technical & business systems, micros, networking & communications a speciality.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry. Designing, programming and implementing applications of O.R. and statistics.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-250 250-350 300-450 350-600 400-750	Junior consultant Consultant Senior consultant Principal Consultant Managing consultant
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.
Dyadic	Consultancy	poa	APL system design, consultancy, programming & training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.
E & S	Consultancy	150-250	System prototyping: all types of information system.
FASTCODE	Consultancy	poa	Specialise in improving performance of APL applications on micros & mainframes.
Gen. Software	Consultancy	100+	
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Inner Product	Consultancy	200	On-site micro-mainframe APL, PC/DOS & Assembler
Lloyd Savage	Consultancy	poa	Decision support, particularly specialising in Sales & Marketing systems.
M.B.T.	Consultancy	poa	
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.
MetaTechnics	Consultancy	poa	Management Information & Production. Engineering, APL - C/Assembler custom programming
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.	Consultancy	poa	Specialise in Maintenance and development of existing APL systems
Parallax	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	200	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.
Uniware	Consultancy	call	Junior to managing consultancy in APL.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APL Consulting	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
	Tim Perry	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT. Tel: 01-577 3541
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Alis	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT Tel: 01-577 3541
APL Software Technology	John Hagger	14 Rosewood Avenue, Alveston, Bristol BS12 2PP Tel: 0454415737
	Per Hultin	46 Vicarage Road, South Benfleet, Essex SS7 1PB Tel: 0374550501
Beta-plan APS	Kim Andreasen	Stengrade 75, DK-3000 Helsingør, Denmark. Tel: 45 2 21 48 48
Buckland Management Systems	John Buckland	Westwood, 9 Grove Road, Camberley, Surrey, GU15 2DN Tel: 0276 684327
Butel Technology Ltd.	Mike Munro	Butel House, 3 Great West Rd., London W4 5QJ Tel: 01-995-1433
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 60130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 01-767 4254
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE16 Tel: 01436 9481
Datatrade Ltd.	Tony Checkseld	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire. Tel: 0420-87024
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
Farnell International Instruments Ltd.	R. Fairbairn	Jubilee House, Sandbeck Way, Wetherby, W. Yorks. Tel: 0937-61961
	Roger Attard	Davenport House, Bowers Way, Harpenden, Herts. Tel: 05827-69071
FASTCODE	Andrew Dickey	P.O. Box 281, Croton-on-Hudson, New York 10520, U.S.A. Tel: (914) 271-3200
General Software Ltd.	M.E. Martin	22 Russell Road, Northolt, Middx. UB5 4QS. Tel: 01-864 9537
H.M.W. Computing Ltd.	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 01-353 4212
IBM UK Ltd	Nat.Enq. Centre	414 Chiswick High Rd, London W4 5TF Tel: 01-747 0747
Inner Product Ltd.	Dominic Murphy	Eagle House, 73 Clapham Common Southside, London SW4 9DG. Tel: 01-673 3354
Lloyd Savage Ltd	Philip Johnson	Cambridge House, Oxford Road, Uxbridge, Middx. UB8 2UD. Tel: 0895-59826
Mercia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5095

MetaTechnics Systems	John Stenbridge	Unit 216, 62 Tritton Road, London, SE21 8DE. Tel: 01-670 7959
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 01-922 8866
M.T.I.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: 09327 80848
Parallax Systems Inc.	Kevin Weaver	60 West 9th Street, New York, New York 10011, U.S.A. Tel: 212-475-4001
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW6 3ER Tel: 01-624 7013 (Answerphone) 0860-377953 (Mobile)
Portable Software	Richard Smith	60 Aberdeen Ave, Cambridge, Mass. U.S.A. 02138. Tel: 617-547-2918
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. Tel: 0276-20789
The Rochester Group	Robert Pullman	164 Pinnacle Rd, Rochester NY 14620 Tel: 716-461-3169
Shandell Systems Ltd.	Maurice Shanahan	12 High Street, Chalfont St. Giles, Bucks HP8 4QA. Tel: 02407-2027
I.P.S.A. Ltd.	Mike Butler	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA Tel: 0203 256562
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Tektronix UK Ltd.	Paul Morgan	Fourth Avenue, Globe Park, Marlow, Bucks SL7 1YD. Tel: 06284-6000
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France Tel: (1) 45-27-20-61 Telax: 648348F UNIWARE
Warwick Univ.	Dr Andy Pole	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523066 x2158

Books Received for Review

We have two books which we hoped to be able to review in the next VECTOR. The first of these is called 'Mathematical Experiments on the Computer'. It is by Ulf Grenander, has been around for several years, and is solid with really useful, well presented APL. I look forward to getting the full review of this one into 6.1 ... I only wish the author had mentioned 'APL' in the title so that we would have tripped over it sooner!

The second book is called 'APL: a Problem Oriented Approach'. It is a translation from a German original, and is published by Ellis Horwood. Unfortunately the typesetting of the APL is so impossibly bad that we felt we had to contact the publishers immediately in an attempt to get the book withdrawn before it did the APL community any more damage. I am glad to say that Ellis Horwood have been most co-operative, and are withdrawing the current edition from sale.

*** if you see this book for sale: do not under any circumstances buy it! There will at some point be a corrected version available. We shall then feel able to review it.

IBM Personal Computing in Schools Edited by Paul Barnettson and published by Kogan Page Ltd 1988

Short Notice by Anthony Camacho

This is a collection of 24 articles on using PCs in schools. Many of them are reports on the worthy use made of some 200 PCs installed with joint educational and IBM funding. As a survey of the use of IBM PCs in schools between about 1984 and 1987 it is of general interest. There are two articles that mention APL: "APL in Schools on the IBM PC" by Norman Thomson and "Extra-curricular Activities with an IBM PC" by Paul Barnettson himself. The latter is a much fuller description (but without any of the APL code) of the Charlton Chase system on which there were three pages in Vector 4.1.

Norman Thomson's article is a 15 page introduction to APL. It is a justification of his assertion that APL is the best way to do mathematics on a computer. He gives functions for: odd numbers, arithmetic progressions, geometric progressions, square and triangular numbers, multiplication and other tables, prime numbers, factorising, perfect numbers, Gregory's and Euler's series, sums of reciprocals of Nth powers of integers, average, mean, variance, standard deviation, median frequency distribution and simulation of statistical experiments. It is clear and persuasive - it would be an excellent leaflet to circulate to school maths teachers. I would like to see the BAA reprint it and circulate it.

Second Generation APLs on the PC

Introductory notes by Adrian Smith

The Technical Editorial in VECTOR 5.3 made the point that microcomputer hardware is finally catching up with the demands made of it by APL. We can at last have decent workspaces (I get over 3Mbyte of $\square$ WA in IBM's APL2/PC on my PS/2 Model-70), and the arithmetic is back to the mainframe standard with proper long integers and real bitty booleans. What's more these beasties go rather fast! Accordingly we have three (going on four) 386-specific APL systems to look at:

1. STSC's APL*PLUS II system. This is currently at the top end of the scale in terms of both performance and price. Peter Cyriax has taken a technical perspective, and Ajay Askoolum (of the Legal & General Assurance Society Ltd.) has looked at the product from the application side.
2. SAX from I.P. Sharp Associates. SAX was implemented as a joint development project between I.P. Sharp and STSC. The software is based on the STSC Unix APL, but the language follows SHARP APL, not the STSC product. Please check the note at the end of the review for a statement from IPSA on the availability of this product.
3. IBM's APL2/PC (32 bit). Dick Bowman is reviewing this for us for VECTOR 6.1. In price performance terms it really is a rather remarkable product. For £348 you get 32-bit arithmetic, workspace up to 15Mbyte (with no restrictions on object size), an APL2 interpreter (with almost nothing that matters left out) and a rather faster system than STSC's APL*PLUS/PC release 8. It requires an 80387 maths co-processor. David Selby gives a brief summary of this APL (or should we say his APL) on page 82.
4. Dyalog APL for MS DOS. The α -test version of this becomes available at the end of May, and we have every hope of getting a 'first impressions' into VECTOR 6.1. Some idea of its features can be gleaned from Peter Donnelly's notes on page 81 of this VECTOR.

I think we are moving into interesting times! We shall soon be looking towards the 80486 (some 5 times the speed of the 386), and towards I-APL on machines like NEXT and the transputer boards. Watch this space!

APL*PLUS II

STSC's Second Generation APL System for 80386 Machines

reviewed by Peter Cyriax

Early last Autumn STSC released APL*PLUS II, a new generation of APL running on '386' type IBM PCs. It offers mainframe power on your desk for a total of four or five thousand pounds; so whether you need to grade up from older PCs, or want to decentralise mainframe based systems, this is something you should know about.

In a nutshell, APL*PLUS II combines the environment of the APL*PLUS/PC with the APL language from the APL*PLUS/1000 mainframe systems. If you have never used either product, that leaves you none the wiser, but at least you have a treat in store.

There are several other versions of APL that will run on 386 machines - Dyalog, Sax and APL2, which I have heard good things about; but this is not a comparative review, it's an attempt to tell you about APL*PLUS II, and relate it to the products it was developed from.

First, what does it run on, and what do you get?

APL*PLUS II runs on an IBM PC or compatible under DOS 3.3 or later; and requires a 386 CPU with at least 2MB of memory. That gives you an APL that seems to run 4 or 5 times faster than APL*PLUS/PC on the same hardware, with a Workspace size of around 800K. For the moment, at least, it's a single user system, though it will run on a network.

If you want a bigger workspace you buy more memory (up to 16MB). There are many flavours of 386 PC - but a good 20Mhz machine with 3 or 4MB of memory is now commonplace, and costs a few thousand pounds. That makes an interesting comparison with a typical mainframe - it will put an APL system on your desk with a 2 to 3MB Workspace, and processing speed somewhat better than the old IBM 3031. The wire from the screen to the CPU is rather shorter on the PC, of course, so the PC is much more responsive - a couple of seconds faster than the mainframe running under light load.

However, most PCs don't have a floating point processor, so engineering, statistics, CAD-CAM or graphics applications can be very slow. If you have a lot of heavy calculations to get through, then a floating point processor should speed them up by as much as ten times, and the WIETEK chip is several times faster again - a real transformation, but still a couple of times slower than a mainframe with otherwise similar performance. You certainly can offload these sorts of applications from the mainframe to a 386 PC, but you have to think about it.

Floating point is not an issue for most people, so what it all comes down to is this:

A modern, single-user APL with mainframe size workspaces running at about one tenth the speed of the fastest CPU that IBM sell.

APL*PLUS II System

The main components of APL*PLUS II are:

1. APL Nested Array Interpreter
2. APL Component Shared File System
3. The Editor, and Access to the rest of the Environment which I will discuss in turn.

As it happens, I still have a Beta test version of APL*PLUS II, not the production version, but so far as I know there isn't a great deal of difference. I have been using the system for development, not trying to crash it; and the main reason I haven't upgraded yet is that I haven't found any bugs. APL*PLUS II seems reliable.

APL Interpreter

APL*PLUS II is a second generation APL supporting Generalized (Mixed and Nested) Arrays with the extra functions and operators needed to handle them. It also has the earlier developments, such as Grade on Characters and on Matrices, plus the extension of Compress to Replicate, of course.

There has been a lot of criticism of Nested Arrays - mainly that they produce incomprehensible, highly inefficient code. This criticism was also made of first generation APL - and with just as much validity. Like any powerful tool, Nested Arrays can be abused; and abused they certainly were, particularly when we were trying to learn how to use them to begin with. My own first attempts are no exception, but even so I find no real difficulty in reading code I wrote five years ago; whereas in some cases traditional APL would have needed a nightmare of globals, making it quite impossible to maintain. The single most important use for Nested Arrays is to pass multiple parameters into Functions, which helps to make them modular and clarifies their interface - hard to argue with.

Neither is efficiency an issue. This is particularly true of APL*PLUS II, which, in common with other modern APLs, was designed from the ground up to support Nested Arrays. The Monitor (OMF) facility is quite up to the task of finding out where CPU is going, and it shows that Nested Arrays can lead to simpler and faster code when they represent well how your data is actually organised. I would not be without them.

In language compatibility terms, APL*PLUS II is almost identical to the APL*PLUS/1000 mainframe system, which is a (major) development of VS/APL. The only differences I have found are that Replicate Foreach (/") does not work because of the conflict between Replicate (Compress) and Reduction - IVERSON ERROR; and also `⌊SIZE` gives the answer you always wanted anyway i.e. how big the object actually is. Dyalog APL is also very similar - it was modeled on STSC's experimental system that preceded APL*PLUS/1000; and IBM's APL2, though different, shares the same basic definition of Arrays plus many of the extensions. Finally APL*PLUS/PC is a small, non-nested version of APL*PLUS/1000; so altogether APL*PLUS II is generally upward compatible with, or at least similar to most other major APL systems.

The implementation of APL*PLUS II is also impressive, with only one obvious weakness - Index and Replicate are both very slow. Index has always been a problem, because in general it is quite difficult to spot the cases that can be optimised; but Replicate is surprising, though it may have been improved by now. This is a minor quibble for such a new product: overall the Interpreter seems fast - it runs in true 386 chip 32 Bit mode; and like the mainframe product it uses 1 Bit Booleans (faster, and saves a lot of space), 32 Bit Integers (no more floating-point dates and array indices), and 64 Bit Floating-point Numbers. Also, it passes APL values around with internal pointers as much as possible, rather than copying them, so it doesn't waste CPU or use up Workspace.

APL File System

APL*PLUS II supports the Component Shared File System originally developed by Sharp and STSC in the early 70s, and still going strong.

APL*PLUS/1000 and APL*PLUS/PC also have this same Component File System, implemented using the `⌊F...` family of System Functions.

An APL Component File is essentially a sequence of Components, each of which is an APL value of whatever shape, rank, depth, type and size you can dream up (or fit into your workspace). It is simple, but it fits APL well, which is why it is so successful.

The mainframe version of the File System is a shared system, which means that different people can use it simultaneously; even updating the same file at the same time in a controlled way. At first sight this feature appears irrelevant on a single user PC system; but it is implemented on both APL*PLUS/PC and APL*PLUS II. This means that both products can be used to share data on a network of PCs, and in due course APL*PLUS II should also work on a multi-user 386 system running under either CDOS or OS/2.

The performance of any File System depends predominantly on disk hardware and buffering, but the APL*PLUS II version is noticeably faster than APL*PLUS/PC. Also it

suffers far less from the fragmentation which can occur when Components keep changing their size, particularly if they are large.

APL PC Environment

This I love, and hate. The good news is there is nothing you cannot do, and probably with only one keystroke: the bad news is that, like me, you may find it quite impossible to remember what that keystroke is. To be fair, this problem seems to be endemic to all PC systems, not just PC APL. One of the features that makes APL attractive is the way it fits together to form a coherent whole - it seems a shame to spoil it with a myriad of un-coordinated facilities, however useful each one might be on its own. This is a very personal view, but I do feel that too much use has been made of the freedom that the PC provides. However these facilities are indeed useful, and I must admit that APL*PLUS II is a more productive development environment than either APL*PLUS/PC or any mainframe system.

The principal facilities of the APL PC Environment are:

1. APL Character Set Support
2. Session Manager
3. The Editor
4. Access to DOS Native Files
5. Screen Control
6. Graphics
7. Communication with other Systems

Support for the APL Character Set is available for almost any screen configuration and a large selection of printers, without any hardware changes, using software defined Character Sets. The old, low resolution mono screens needed a special Character Set ROM, and that is still available, but I cannot see anybody putting a screen like that on a 386 machine. I tested APL*PLUS II on a number of systems before buying one, and they all worked fine. Over the last few years screens and printers have become steadily more flexible, as well as better and cheaper; so Character Set problems should be a thing of the past.

The Session Manager is similar to IBM's Session Manager for VS/APL: it allows you to scroll back over your session log, and to change and re-execute statements; but it also contains other facilities e.g. string search, printer control, cut and paste, change

character sets, undo changes and deletions, talk to the mainframe... obtain help on all of these facilities etc., etc.

The Editor is a development of the `EDIT` full-screen editor available on earlier versions of `APL*PLUS/PC`. It is not that different to use, but much more powerful, and has been made available in Release 8 of `APL*PLUS/PC` as well. You can stack up edit sessions just as you can with mainframe `APL XEDIT`. The Session Manager itself is integrated into the Edit Stack - the system thinks you are editing the session log - which explains why some of the more 'advanced' features are there at all, but it all works cleanly, and it means that you can easily move code between functions or between a function and the session log, or even between workspaces as the edit sessions survives `LOAD` - surprising, but very handy. I also like the facility to break lines and glue them back together again. This Editor is also much better suited to text data than `XEDIT` is - almost as good as a word processor. Overall I like it, though one final facility does seem to be missing - it would be most useful to split the screen and have two sessions available simultaneously, particularly during de-bugging.

Access to DOS native files is via the `INF...` (Native Files) family of System Functions. I haven't used them much, but when I tried they did what I expected and ran quite quickly.

Screen Control centres around `WIN` and other `W...` System Functions. `WIN` defines the fields on the screen using a format matrix similar to the one in `GDDM`'s `ASMFMT` call. This is the same interface that `APL*PLUS/PC` offers, and I find it somewhat inelegant and a bit slow; though compared with using `GDDM` via `API26` it is a paragon of simplicity. Unlike `GDDM` this interface does not provide any graphics capability - that is what the `G...` family of System Functions are for. These, again, are the same as the `APL*PLUS/PC` offering.

However, `APL*PLUS II` also provides an interface to third-party graphics software (included in the product) that implements the Graphics Standard Interface - `GSI` - and supports an impressive list of devices. `GSI` is an ISO and ANSI device independent graphics standard, which means that you can decide what your picture looks like without having to worry what device you will use to produce it; and the definition should work on a wide range of hardware. This has to be the right approach. At the moment you cannot mix Text and Graphics on screen using either `GSI` or the `G...` functions in conjunction with `WIN`; but under `OS/2` Presentation Manager all screen handling will be in graphics mode - otherwise you cannot scale panels or lay them over each other, so I am sure that `STSC` will fix that restriction when they produce an `OS/2` version.

As far as Communications with other systems are concerned there are really three issues. If you want to drive local devices using the serial or parallel ports - touch screens, for instance, recently reared their heads - `PARBIN` is well up to the task. If you want to log on to a mainframe, you need an `IRMA` board (or something similar) if it's an IBM bi-synch

system, otherwise the built in terminal emulation or `PARBIN` should do the trick. Finally if you want to call programs in other languages, possibly running in other CPUs on a network, that is a bit harder. This is what IBM's new Systems Applications Architecture - SAA - is all about; and the central APL facility here is `QNA`, which isn't available, yet. However, all the signs are that STSC intend to provide the necessary features in this area; and my guess is that they will be available before most people actually need them.

Summary

I'm glad I bought a copy.

MicroAPL for ...

APL on the widest range of micros

STSC's APL*PLUS Release 8

£425

STSC's APL*PLUS II PC

£1395

APL.68000

APL.68000 for the Mac, ST, Amiga

£86.91

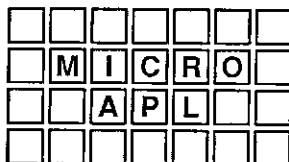
APL.68000 for the Mac II, SE30

£260.82

APL.68000 runs on many multi-user computers, including:

Sun, NCR, Wicat, HP200/300, Spectrum, Aurora, Stride £1200 to £2000

All prices quoted exclude VAT



MicroAPL Ltd.

South Bank Technopark

LONDON, SE1 6LN

Telephone: 01 922 8866

Access and Barclaycard

Orders accepted

APL*PLUS II for 80386 PCs

reviewed by Ajay Askoolum (Legal & General Assurance Society Ltd)

The Product

APL*PLUS II is a second generation APL interpreter for 80386 based PCs (including ATs with an Intel INBOARD/AT), with or without 80387 coprocessor, running DOS 3.3 or later, with RAM size of 2MB or more. The version reviewed is 1.00.07 dated 16th August 1988.

This interpreter, unlike the APL*PLUS/PC interpreters, is protected. Whilst a single copy of the software may be installed on as many PCs as desirable, only one such installation may be used at any time. The interpreter polls a 'dongle' - between the parallel printer port and cable - at random intervals throughout any active session. It cannot be invoked in the absence of this device and if removed during a session, the user is prompted to re-insert it and press ENTER. If ENTER is pressed without the device in place, the session is terminated with the ensuing loss of the workspace.

The software costs £2,000 plus VAT per copy and the run-time system costs £250 plus VAT per copy. An annual maintenance charge equal to 25% of the original purchase price is payable after the first twelve months. The maintenance charge includes all upgrades and (telephone) support provided by the appointed distributors Cocking & Drury (Software) Ltd. Discounts are negotiable on purchases of multiple copies at the same time.

The same package is available from MicroAPL for £1,395 plus VAT - presumably without any maintenance or upgrades.

An APL character generator ROM will be required if the display adapter in use is not EGA or VGA and will be supplied free of charge on request. This ROM is identical to the one which provides APL character generation with the APL*PLUS/PC interpreter. Most 80386 based PCs provide a soft character set facility and will not require this ROM.

APL*PLUS II is a new product from STSC - not intended as a replacement for APL*PLUS/PC - which is closely modelled on IBM's APL2. However, most of the file and $\square$ G-style graphics functions in APL*PLUS/PC are available in an enhanced form together with generic information to install and use Graphics Software Systems Computer Graphics Interface (GSS*CGI); this is a device-independent interface to graphics devices for which cover functions are provided in the VDI (Virtual Device Interface) workspace provided as part of the package. The editor, reminiscent of)XEDIT, is a new feature: it allows multiple objects - one of which is automatically the active workspace and which may include native files - to be edited simultaneously. Another

new feature is the session manager which is invoked by CTRL / and allows session bound features and parameters to be set via pull-down menus - this is an easier means than keyboard input of the relevant commands to achieve the customisation sought.

APL*PLUS/PC workspaces and files can be migrated into the APL*PLUS II environment and may not require code changes to run - workspaces cannot be loaded directly nor can files be tied as component files. The migration facility is not provided within the interpreter - as in the guise of)MCOPY in APL2. Instead, a number of base functions are supplied with adequate documentation on the syntax and procedure involved in the migration process. The reverse process is not documented - however, there would be little need for this when the APL*PLUS II specific features are used. Instead of the familiar AWS and ASF extensions, WS and SF are used to differentiate workspaces and files. The APL*PLUS/PC subset of this new interpreter is not totally compatible with APL*PLUS/PC - just functionally so, with the aid of the utilities provided.

The packaging, style and feel of this new product shares the same hallmarks as the existing APL*PLUS/PC product: the manuals (three volumes plus a quick reference guide) are precise and the usual range of complementary utility/tutorial workspaces - including printer support, assembler and APL character generation functions - are included on the three 1.2MB diskettes supplied (some GSS device drivers are not included because of lack of space, but are available upon request). Only 5.25 inch diskettes are supplied; however, the software may be supplied upon request and free of charge on 3.5 inch diskettes in lieu.

There are some omissions: there is no SYSHELP.SF file and therefore no help facility on the interpreter itself is available by default. The help facility provided is sparse and barely adequate. The system does not automatically use coprocessor support if available: the invocation must specify whether the version supporting the 80387 chip or the version with software emulation of the chip is used. The WEITEK coprocessor is not supported.

Moreover, the customisation of any APL session comes from the configuration stated in a native file which is CONFIG.APL by default, unlike the APL*PLUS PC method where the desired configuration is achieved by supplying the relevant parameters when invoking the interpreter and an initial workspace to load and an APLLIBS.ASF file.

Upgrading to APL*PLUS II

The interpreter is very powerful and provides a very much more versatile environment than APL*PLUS/PC - the obvious one being the facility for workspace sizes between 1-15MB with APL2 features.

This makes it a rival contender to the mainframe environment for the type of applications developed on the mainframe. However, while there is a definite cost advantage in favour of APL*PLUS II, there will be some loss of flexibility.

APL*PLUS II cannot be seen as a substitute for the APL*PLUS/PC applications currently in use: the cost of the equipment and the interpreter itself is far too prohibitive and does not necessarily offer any obvious critical advantages.

There are alternatives to APL*PLUS II which should also complicate the decision. There is DIALOG APL running under UNIX, with the facility to run up to four terminals and the flexibility to transfer workspaces to and from the mainframe APL2 environment, and with access security which is important in the maintenance of the application run. Also, IBM have announced an APL2 PC interpreter in the same league, at around 20% of the cost of APL*PLUS II with the possibility of free run time systems.

The APL*PLUS II package would be an ideal means for the development of personal productivity applications. These are the development of prototype solutions for use by a small team, probably within the same department - the styling and management of applications are irrelevant but the accuracy and speed of development are vital. In such a situation, the development is undertaken by the people who fully understand the processes involved without recourse to any systems analysis. In other words, APL is simply a well understood personal productivity tool rather than an art form accessible only via third parties.

The effectiveness of APL*PLUS II within this context is limited by at least two factors:

1. There is no support for full screen applications other than in the standard functions (□WIN, □WGET, □WPUT etc.). The available functions are extremely powerful and probably much better than the GDDM equivalent on the mainframe, but are not easy to use effectively. There is no reason why the Society could not write and document standard cover functions so as to simplify the task of developing full screen applications.
2. There is no provision of help facilities on the system features of APL*PLUS II which are unique to STSC. A system level help facility similar to the one provided with APL*PLUS PC would have been sufficient. Although the same help facility could be invoked from the APL*PLUS II interpreter, this would be deficient in those things which are characteristic of second generation interpreters and may be misleading not only because the two interpreters are not fully compatible but also because certain features are exclusive. Thus training courses would be necessary for those new to the PC environment.

Also, not enough information and utilities are available to assess the effectiveness of APL*PLUS II in a networked environment. The intention would be to share files and

workspaces among users - possibly on an alternate basis so that the dongle could be used by say two people - and with the mainframe APL2 environment.

On the other hand, the APL*PLUS II offers an APL2 style interpreter with enhanced file and graphics facilities which are very well documented in the manuals supplied.

I evaluated the package on a COMPAQ 386/20 with 5MB RAM, 80387 coprocessor, BROTHER HL 8 laser printer and colour VGA monitor. The cost of this type of configuration provides a good estimate of the cost of making a transition to APL*PLUS II on a PC from the current APL2 on the mainframe. Also, an assessment of the training needs and the level of support should be made.

The complexity of the task to migrate mainframe applications to APL*PLUS II must not be underestimated. GDDM or auxiliary processor 126 is not available in the APL*PLUS II environment, and file handling is very different on the PC - thus major changes will be required to the mainframe workspaces using GDDM and file facilities before they would run. Even those applications which are workspace based and do not use files may need scrutiny as APL2 and APL*PLUS II are not 100% compatible.

A Personal Assessment of APL*PLUS II

I am familiar with VSAPL under VSPC, VSAPL under TSO, APL2 under TSO and APL*PLUS/PC.

With a good knowledge of APL, APL*PLUS II is a familiar environment from the outset for me, particularly as it retains most of the characteristics of APL*PLUS/PC. A notable difference is the session manager and the new editor. The session manager is very unlike the session manager for APL2 in that it can be relegated to a dormant state. Its obvious advantage being the facility to set session bound parameters within a workspace via a series of pull down menus rather than by keystrokes.

The editor is reminiscent of)XEDIT in VSAPL under TSO. Once within the editor, it responds to the usual ALT/key combinations in the same way as the APL*PLUS/PC editor. The ability to edit and update native files from within a workspace simplifies the procedure a great deal - although when APL*PLUS II writes back an existing native file, called up by the editor but not altered, the new file is larger by a matter of a few hundred bytes. The specification of a workspace name using the DOS path as part of the specification rather than a library number appeared somewhat alien to an APL environment - but the usual convention of library numbers may be selected by modifying the CONFIG.APL file appropriately.

Some irritations with the APL*PLUS PC interpreter still exist - for instance it is still not possible to edit numeric objects in the new editor. Others do not: □TRACE and □STOP

survive the editor, there is no need to wrestle with the setting up of virtual workspaces. The usual convention for ambivalent functions is retained. There are some practical enhancements such as)PSAVE which prevents accidental overwriting of a workspace on disk. The)GROUP facility does not exist - in the VSAPL environment this allowed any number of objects which had been grouped to be copied from a saved workspace by simply specifying the group name. It is possible to copy any number of objects from a saved workspace but the individual object names must be known and specified in a character array and the array specified for the)COPY operation.

Given the size of the workspaces, and the fact there is no warning that there may be multiple editor sessions pending when logging off, I would have liked to have a facility within the interpreter which saved the active workspace at random intervals providing something in the guise of the CONTINUE workspaces in APL2. Such a facility would permit most of the work during any session to be partially safeguarded automatically.

The equivalent of □TF in IBM APL2 is absent, as is)MCOPY. Both of these omissions are noted by STSC; a way round the problems created is suggested in the workspace and file migration procedures detailed and the provision of the SLT (Source Level Transfer) workspace which uses the WIC (Workspace Interchange Convention level 1). An)MCOPY facility would be valuable in that developments are very likely to be continued in the APL*PLUS/PC environment and subsequently ported into the (expensive) APL*PLUS II environment.

The migration procedure advocated does work, although it is quite contrived to be contemplated as a matter of routine, as long as the few incompatibilities of the APL*PLUS/PC and APL*PLUS II are remembered. The □CT is different, □KEYW does not exist, system functions are not absorbent, library number convention must be used within the CONFIG.APL file when an application uses library numbers, the help file is defined in the CONFIG.APL file and the few incompatible □POKEs are sorted out.

For the types of applications I have developed, the omission of the □HNAMES system function causes unnecessary problems. It means that it is not possible to change the file to hand when pressing ALT F6 from an APL session. The file accessed is specified in the CONFIG.APL file and cannot be specified in a □LX function as is my current practice. Also, the fact that system functions are no longer absorbent and cause an error condition is very irritating to me - it's a style that I am very fond of and which I use to avoid yet another entry in the Symbol Table - space is at a premium when the maximum workspace size is 440KB. However, this is a minor problem since the APL*PLUS II workspace sizes are appreciably larger and the Symbol Table is dynamic and allows 32767 entries.

The interpreter is fast, the loading and saving of workspaces also appear appreciably faster. My personal preference would be to develop fresh applications from scratch using

wherever possible the enhancements of this second generation interpreter. The first objectives would be to develop a workspace of cover functions for full screen applications and another workspace of standard library routines of the type I have developed in APL*PLUS/PC.

Songs of Innocence & Experience

APL*PLUS II offers an unrivalled environment as a personal productivity tool, requiring only a good knowledge of the language for the development of any application without compromise. The power of the language is that it allows its user total control over expression - it allows all its characteristics to be interrogated, offers very good error trapping facilities as well as reporting errors in a constructive way, has operators and primitives which are capable of reducing concepts to a few symbols (think of nested arrays, for instance), offers excellent filing and graphics facilities and allows progress from concept to program code without any intermediate steps with great ease. It has no equal in the absence of automatic code generators.

Unfortunately, the APL environment is marred by its propensity for a variety of programmer abuse, and it requires a run time system - at extra cost - for the distribution of applications. The APL*PLUS II system suffers in a number of respects: the enormity of its workspaces makes a disciplined approach to documentation and maintenance mandatory, the cost of its run time system is high and the cost of putting together an APL*PLUS II work station is very high at between £7,000 to £12,000.

Given that APL*PLUS II is highly unlikely to be considered for application development in a green field situation, I would have liked such utilities as workspace migration from the mainframe and the ability to read data from industry standard applications, such as LOTUS and SMART, bundled with the package. A facility to use APL*PLUS II under Microsoft Windows/386 such that multiple APL*PLUS II sessions were possible might have its uses too - I was unable to use APL*PLUS II under the Windows environment.

The utility workspaces are the ones packaged with APL*PLUS/PC Release 6.0 onwards - that is they are not optimised for the enhanced second generation environment.

Nevertheless, I would have no hesitation in recommending APL*PLUS II as a strategic product for PC applications development.

SAX

reviewed by Michael J.A. Berry

I love APL, and at one time I hoped that the qualities which endear it to me would cause it to sweep the world leaving the likes of Fortran and Basic rocking in its wake.

I no longer harbour that hope. My new, more modest, desire is that APL continue to be available as an alternative to those of us who would like to use it. For even this goal to be achieved, two things must happen:

- [1] APL must continue to be the best tool available for expressing algorithms in a form which is both readable and executable.
- [2] APL must learn to coexist happily with the rest of the software world.

SAX makes significant progress in both of these areas.

The Language

SAX is a partial implementation of the language defined by Dr. Kenneth Iverson in 'A Dictionary of APL' (APL Quote Quad Vol. 18, no. 1, September 1987). Readers who are unfamiliar with the dialect of APL described by Dr. Iverson, may wish to consult it for amplification of the examples in this review. The SAX implementation also includes most, if not all, of the 'bells and whistles' which have accreted to the Sharp mainframe product over the years. □TRAP, □FMT, □PACK and so on are all available.

SAX was implemented as a joint development project between I.P. Sharp associates and STSC. The software is based on the STSC Unix APL, but the language follows SHARP APL, not the STSC product.

For reasons which I will examine in the second part of this review, APL programmers are sometimes forced to use other languages in the course of their work. It is often a painful experience. All those declarations! All those loops! After using SAX for a few months, I now get much the same feeling about the other APL's I have available (STSC's APL*PLUS/PC and SHARP APL/PC). The old familiar APL idioms now seem like clumsy circumlocutions filled with semi-colons and explicit iteration. I return to SAX with a feeling of liberation.

Much of the feeling can be attributed to the differences between any 'advanced' APL and the APL's I have available for comparison which are embellished versions of ISO Standard APL. The most important difference between advanced APL dialects and standard APL is that in the former, operators may take derived and defined functions as

arguments. Without that extension, available for several years now in APL2 and its derivatives, other common extensions such as arrays of arrays are robbed of much of their power and appeal.

SAX differs from other advanced APL dialects both in the set of primitive operators provided, and in the centrality of a concept called 'function rank'. The rest of this section examines the consequences of these differences. The information on APL2 comes from 'APL2 Programming: Language Reference' (IBM 1984) and 'APL2 at a Glance' (James Brown, Sandra Pakin, Raymond Polivka, Prentice Hall 1988).

Major steps in the evolution of APL have often come about through the recognition that something already extant in the language as a special case could be described in a general way and then extended. Thus $\wedge.$ became part of APL because Iverson recognized that the inner product used in mathematics was just one member of a useful family of operations. Similarly, reductions and inner products existed in the language long before a theory of operators existed to explain what the $/$ and $.$ were doing there. Once these few special cases were recognized as members of a whole class of operators which produce functional results, the stage was set for adding new ones and extending the old ones to new domains. The resulting proliferation of operators, especially operators for functional composition, has made possible a much more functional style of APL programming. Things formerly accomplished through modification of the control flow through branching may now be accomplished by modifying the way functions are applied. The result is not only more aesthetically pleasing programs, but also programs which preserve the essentially parallel nature of many algorithms rather than obscuring it with branch points.

Function rank is another idea which has always been in the language but has only recently been formalized and generalized. A function's rank is defined by the rank of its arguments at the fundamental level of its definition. All the scalar functions are said to have rank zero. They are defined in terms of scalars, and applied to arrays of higher rank according to rules well known to readers of Vector. In SAX, these rules are extended to cover what are often called the 'mixed' functions. They, too, have default ranks. Monadic ϕ for example, is of rank one. Its fundamental definition is in terms of vectors. When faced with an argument of rank 2 or higher, ϕ partitions the array into cells of rank 1 and then applies its fundamental (rank 1) definition to each of them. This is precisely what scalar (rank 0) functions have always done, except that a scalar function partitions its arguments into cells of rank 0 before applying its fundamental definition.

Dyadic functions may have differing left and right ranks. Again, ϕ makes a good example. It is said to have left rank 0 and right rank 1. In the expression $(15)\phi\ 5$ 50AV each scalar cell of the left argument controls the rotation of a vector cell of the right argument. Many functions have infinite rank by default. That is, they apply to the

entirety of their array argument without partitioning it into cells. Ravel is such a function, as is `enclose`.

Once the concept of function rank is recognized, it is natural to want an operator to control it. The `∘` operator of SAX, with a functional left argument and a data right argument, does just that. The following examples give a feel for its use.

```
m ← 3 4 5 p65 ↓ ⍱ av
m
```

```
ABCDE
FGHIJ
KLMNO
PQRST
```

```
UVWXY
Z[ \ ] ^
_ ! abc
defgh
```

```
ijklm
nopqr
stuvw
xyz{;
```

```
, 32 m
ABCDEFGHIJKLMN OPQRST
UVWXYZ[ \ ] ^ _ ! abcdefgh
ijklmnopqrstuvw xyz{;
```

```
⊗ 32 m
AFKP
BGLQ
CHMR
DINS
EJOT
```

```
UZ_d
Vl | e
W\af
Xlbg
Y^ch
```

```
insx
joty
kpuz
lqv(
mrw;
```

```

      pm
3 4 5
      pö2 m
4 5
4 5
4 5

```

```

      n+4 5p120
      n
0 1 2 3 4
5 6 7 8 9
10 11 12 13 14
15 16 17 18 19

```

```

      (5p100) +*1 n      #Add a vector to the rows
100 101 102 103 104
105 106 107 108 109
110 111 112 113 114
115 116 117 118 119

```

```

      (4p100) +*0 1 n    #add a vector to the columns
100 101 102 103 104
105 106 107 108 109
110 111 112 113 114
115 116 117 118 119

```

```

      <m                  #the default rank of enclose is infinite
|-----|
|ABCDE|
|FGHIJ|
|KLMNO|
|PQRST|
|      |
|UVWXY|
|Z[\]^|
|_|abc|
|defgh|
|      |
|ijklm|
|nopqr|
|stuvw|
|xyz{|
|-----|

```

```

      <*2 m              #Enclose the 2d cells
|-----|
|ABCDE||UVWXY||ijklm|
|FGHIJ||Z[\]^||nopqr|
|KLMNO||_|abc||stuvw|
|PQRST||defgh||xyz{|
|-----|

```


operate. This approach is more flexible because each function may partition the data as appropriate to its own algorithm.

The prevalence of enclosed arrays in APL2 may also explain the decision by its designers to make some functions 'pervasive' (operating inside what is nominally a scalar) and others not. Decisions such as this are best left in the hands of the programmer. In SAX, no function is pervasive, but `FOO">` produces a version of `foo` which pervades one level. `FOO">` is read 'foo dual disclose.' `F"G` derives a function which, for each argument cell, applies `g`, followed by `f`, followed by the inverse of `g`. The particular case `F">` is closely related to APL2's `F"` (read 'f each') for pervasive functions. The difference between the two flavours of 'each' in SAX (`F*0` and `F">`) is nicely illustrated using monadic `l`:

```

      (l*0) 1 3 5 7 9
0 0 0 0 0 0 0 0 0
0 1 2 0 0 0 0 0 0
0 1 2 3 4 0 0 0 0
0 1 2 3 4 5 6 0 0
0 1 2 3 4 5 6 7 8

      l"> 1 3 5 7 9
| | | | | | | | | | | | | | | | | | | | | | | | |
| 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| | | | | | | | |

```

SAX follows dictionary APL in cleaning up many of the messier parts of ISO APL. In SAX, one may dispense with square brackets and semi-colons completely. It is now easy to write functions which work correctly on arguments of unknown rank. The `{FROM|` function `f` selects cells from its right argument. The definitions of `+` and `+` have been extended so that it is not necessary to know the rank of the right argument in order to construct a meaningful left argument. Indexed assignment is made obsolete by the merge operator `}` which given a selection function, derives a dyadic function which selects elements from both its arguments to form a new result:

```

      n
0 1 2 3 4
5 6 7 8 9
10 11 12 13 14
15 16 17 18 19

      3↑n
0 1 2 3 4
5 6 7 8 9
10 11 12 13 14

      0{n
0 1 2 3 4
2{0{n
2

```

```

      'HELLO' 2''() n      aMerge character vector into n
0  1  2  3  4
5  6  7  8  9
H  E  L  L  O
15 16 17 18 19

```

The SAX Environment

The term 'environment' is used in two ways. The first refers to the programming environment and user interface. This includes editors, debuggers, cross-referencers, and the like. The second refers to the operating system environment of linkers, run-time libraries, etc. APL needs to do a better job of fitting into both of these environments. Twenty years ago, when commercial APL systems first became available, The APL work environment was a great leap forward. The interactive interpretive time-shared environment took much of the tedium out of programming. Because there was no existing programming environment as rich as the one which came built into APL/360, there was no reason for APL to run as part of a standard environment.

The world has come a long way in the last two decades. There are now programming environments which offer much more than any APL session manager. Unfortunately, it is not possible to run APL under most of these programming environments because APL interpreters usually come with an inseparable session manager which attempts to do everything the user might want. Even if the built-in full-screen editors and so on were not inferior to general purpose ones such as Emacs, they are different.

Differences are annoying. Before SAX, I could use GNU Emacs to program in C, Fortran, and Lisp, and to read my mail, look at usenet articles, and manage multiple shell windows, but I had to shut it all down to bring up APL. SAX properly separates the language from the environment. I can now run APL like any other UNIX process taking its input from STDIN and writing to STDOUT. This has allowed me to build an APL-Interaction mode for GNU Emacs which is modeled on the Lisp Interaction mode that comes bundled with the (free) GNU software.

I would like to donate this APL mode to the Free Software Foundation to make it part of the GNU software shipped everywhere. Unfortunately, it is not much use without a public domain screen font and printer font. It is time the owners of APL fonts realized that they are more valuable if given away. The first vendor who supplies a free X font for APL will be rewarded by having their $\square$ AV become a de facto standard.

Of course, many users may not be inclined to write their own session manager! SAX may also be run under its own session manager which includes the STSC full-screen editor. The 'union' keyboard layout used by the session manager preserves the traditional keyboard locations of ascii characters. This is important in any environment where the same keyboard device is used for more than one purpose.

Another welcome step towards integration with the rest of the computer world is that $ASCII \subseteq \square AV$. This means that ordinary files look right when read into APL, and that APL files which avoid the second half of $\square AV$ are acceptable to any other program. Now that standard screen tools such as SunView and X support 8-bit (256 character fonts) natively, APL's large character set is no longer a problem.

The documentation came packaged as 3 sets of 3-hole punched paper suitable for inserting in the standard North American ring binder. (Try buying one of these in the UK and see how far you get! Ed.) The three volumes are titled 'Language', 'Environment', and 'System'. Although I usually dislike multi-volume documentation sets, in this case I found that my first guess as to which volume I needed to consult was almost always correct. The Language volume is the one I consulted most often. It is well written and easy to understand. The System volume covers all the $\square NAMES$, system commands, shared variables, etc. The Environment volume covers installation, printing, and how to write your own AP's.

SAX is less successful at integrating with the operating system environment. Communication with Unix is through the traditional APL shared variables. In order to install the shared variable processor, one must recompile the Unix kernel; an operation fraught with peril. Once installed, the shared variable processor did not prove to be very robust. Hiccoughs in the local area network which only slow down other systems, seem to cause shared variable transactions to hang forever. The postscript file printing facility which was packaged with SAX had to be altered in order to run in the SUN OS (Berkeley) Unix system as it contained system 5 commands.

Large systems today are often made up of modules written in several different programming languages. In my own shop, modules written in C, Fortran, and assembler are freely intermixed. Systems written in Lisp, while not as promiscuous as these other languages, may at least call routines written in them. SAX still generally refuses to call or be called. One of the most impressive features of APL2 is the ability to write $\circ.FOO$ where FOO is a Fortran program.

The ultimate goal should be an APL processor which generates .o files which can be linked into an application in the same manner as the output of compilers for other languages. Only when that day comes will I feel that my right to continue using APL in my work is safe.

Experience

I used SAX to build a moderately large application which generates Fortran code from test specification records. I have used this application to generate many hundreds of thousands of lines of Fortran code. Because of the nature of the application, I put great stress on the interface between SAX and Unix. I use both component files (to store the

test records which include APL objects), and native files (for the Fortran output). I made a conscious effort to program in a 'new' style, using the rank operator in place of iteration and using functional composition of various kinds to derive just the function which would do what I wanted with a single application, rather than using intermediate results. At times this meant I spent longer writing the programs than I would have had I stuck to the APL vocabulary I have been using for years.

In the end, I was glad I had made the effort to master the new ideas. SAX is a far more powerful language than old-style APL. I would happily recommend this product to anyone. My understanding, however, is that it is not generally available except in quantity due to customer support considerations. My suggestion to IPSA is that they drastically lower the price and make it available with no support.

Availability

Subsequent to receiving Michael's review, VECTOR was asked to add the following statement from I.P. Sharp Associates giving the official position on the availability of the SAX product:

How can you get a copy of SAX? I.P.Sharp Associates is basically a wholesaler of the SAX technology. This means that only qualified educational institutions, potential large volume commercial purchasers, and prospective OEMs should contact I.P.S.A. directly. However, you can expect some announcements from selected computer manufacturers in the near future. Such third party suppliers will be making SAX available to the general public.

From: David Smith (SAX Product Manager), 4th April 1989.

RENAISSANCE DATA SYSTEMS

BOOK DIVISION

*** Enlightenment Thru Information Processing ***

1989 Catalog Excerpts (as of January 1, 1989)

+++ All APL Books in Print +++

	Quantity Total	Price Ordered Cost
Important for the Professional APL Programmer:		
=====		
THE APL IDIOM LIST, Perlis and Rugaber - Funded by Mobil. Written at Yale. Very terse, but includes nuggets of gold. 1977, 50 p.	\$ 6.50.	
APL - A DESIGN HANDBOOK FOR COMMERCIAL SYSTEMS, Smith - Readable, technical, thorough. Includes a small collection of good idioms. 1982, 180 p.	\$44.00.	
APL - ADVANCED TECHNIQUES AND UTILITIES, Bergquist. Good discussion of alternative approaches to a wide range of programming tasks - efficiency, searching, dates, workspace documentation, file design, boolean techniques, financial utilities. Assumes knowledge of the basics. Many idioms. 1986, 450 p.	\$44.95.	
Diskette for above. Contains all functions described in text. Format is for STSC APL*PLUS/PC but is uploadable to Sharp APL, VS/APL, APL2.	\$15.00.	
APL HANDBOOK OF TECHNIQUES, IBM - Utilities. Many comments. 1978, 109 p.	\$18.00.	
A Bit of APL History:		
=====		
***** New Printing! *****		
A PROGRAMMING LANGUAGE, Iverson - The book that started it all. A computer science classic. APL's roots. Reprinted by ROS on acid free paper in a high quality, soft cover edition. Only 25 copies available! 1962, 286 p.	\$59.95.	
A SOURCE BOOK IN APL, Falkoff & Iverson - By the famous duo. A must for serious APLers. The whys and the wherefores. Includes "Notation as a Tool of Thought," Iverson's 1979 ACM Turing Award Lecture. 1981, 139 p.	\$15.00.	
So you want to learn APL?		
=====		
APL2 AT A GLANCE, Brown, Pakin, Polivka. Solid, unintimidating, introduction to APL2. Clear illustrations, modern exercises in each chapter. Gets you started. Three sample applications. 1988, 444 p.	\$26.00.	
APL - AN INTRODUCTION, Peele - Especially for the computer-shy. 1986, 459 p.	\$29.95.	
----- Proceedings from APL Conferences -----		
APL AS A TOOL OF THOUGHT, PROFESSIONAL DEVELOPMENT SEMINARS, NY/SIGAPL. They cover a wide range of topics in business and education.		
1987 - Logic, insurance, statistics, A.I., accounting, fractals, linked lists.		
... 1984, 1985, 1986, 1987, 120 to 150 pages each	each \$15.00.	
1983-1987 as a package.	\$55.00.	

Sub-total: \$

Postage and handling - U.S. and Canada (\$2.50 each item over \$25, \$2.50 minimum.):

Postage and handling - International (please add 15% surface mail, 50% airmail):

NYC Residents please add 8 1/4% sales tax, NY State, 4%:

Total: \$

Please send complete catalog: _____

Date: _____

Name: _____

Please make check or money order payable in \$US and send with this form to:

Renaissance Data Systems
P.O. Box 20023
Park West Finance Station
New York, New York 10023-1510
U.S.A.
(212) 864-3078

Street: _____

City, etc.: _____

Postal

Code: _____

Country: _____

RECENT MEETINGS:

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

We are dependent on the willingness of speakers to provide a written version of their talk, ideally with copies of diagrams or slides used.

If you would like to speak at one of the regular British APL Association seminars, please ring the Activities Officer (address on inside back cover) who will respond enthusiastically to your offer.

Contents

APL: Future Plans	February 17th 1989	
Dyalog APL	Peter Donnelly	78
APL2 on the IBM PC	David Selby	82

Future Plans for Dyalog APL

by Peter Donnelly (Dyadic Systems Ltd.)

New Screen Manager (□SM/□SR)

Dyadic's new screen manager which went out on Beta Test before Christmas is now nearing final release. Although the basic design is unchanged, feedback from users has resulted in numerous improvements being made. In outline, the screen manager works as follows:

□SM (Screen Map) is a new system variable that represents the screen. It is used in conjunction with □SR (Screen Read) which lets the user move about the screen and edit its contents.

□SM is a nested matrix which defines a form. Its rows represent fields, and its columns attributes. A field is a rectangular area with:

- contents(data)
- window position and size
- type
- behaviour
- video characteristics (inactive and active)
- position of window relative to contents
- scrolling groups (horizontal and vertical)

Field types include character, numeric, and date. Naturally numeric fields contain true numbers; there is no need to convert to and from text. Indeed formatting is handled automatically and includes fixed and variable number of decimals, triple separators, and continental formats. Dates are represented as numbers (number of days since 1st. January 1900), but are displayed and edited as formatted character strings. Date formats are defined in a file which allows any conceivable date representation and includes the facility to define the names of the days and months.

Field behaviour includes autotab, exit if modified, exit if invalid number, exit on entry to field, pocket calculator input (for numbers), and everything else you need to control user interaction in a sophisticated and efficient manner.

Data that is larger than a window is automatically scrolled both vertically and horizontally. Fields may be linked horizontally and/or vertically in groups that scroll together. Finally, the definition of `□SM` is recursive so that a field can contain a form (i.e. a collection of sub-fields) and so on. This feature has two uses. Firstly you can define a form which is larger than your screen but which is viewed through a window the size of it. Sub-fields are then automatically scrolled into and out of view as the user TABs or otherwise moves around the form. The second use is for spreadsheet applications where the screen contains perhaps 5 forms (data, row titles, column headings, row totals, columns totals) all linked together so that they scroll or remain fixed relative to one another depending upon the direction of scrolling.

Assignment to `□SM`

When you make an assignment to `□SM` the change is instantly reflected on the screen. For example:

```
□SM[32;1]+←'Please input a valid product code'
```

would immediately display that message in the 32nd. field on the screen.

Similarly,

```
□SM[2;4 5 8]+24 80, RED ON BLUE
```

would instantly blow up field 2 to the size of the screen (24 by 80) and change the colour of its contents to be red on blue.

One of the most exciting features of `□SM` is its behaviour when it is localised. Dyalog APL has so-called 'pass-through' localisation. This means that a localised system variable is not undefined when you enter a function or operator. Instead it inherits the value from the calling environment until it is changed. When you enter a function in which `□SM` is localised, the screen remains unchanged. However, as soon as `□SM` is reassigned the new form is displayed on top of the existing one. Just like any other localised variable, shadowed values of `□SM` are inaccessible to your program, but they are visible on the screen. When the function in which `□SM` is localised exits, the local form disappears and the screen reverts to its original state. Localisation thus provides a simple and elegant means to do pop-up menus, help, etc.

Oracle Interface

The IBM Vendor-logo version of Dyalog APL for the IBM 6150 has an interface to SQL/6150, a special 'cut-down' implementation of the Oracle relational database. Dyadic has recently supplemented this popular feature by developing an interface to the full

Oracle product. This interface is available not just on 6150, but on all systems that support Dyalog APL.

New APL Development Environment

Dyadic is in the early stages of designing a new development environment in which the APL session, the editor, and the stack are all integrated in a seamless fashion. It will fully exploit the workstation environment of a bit-mapped colour screen, mouse, etc., but will nevertheless be fully compatible with existing APL conventions.

Currently most APLs let you type an expression then press a special key (normally Enter) to EXECUTE it. We plan to extend this idea by adding keys that mean EDIT and TRACE. Furthermore the EXECUTE, EDIT, and TRACE keys can be mapped to buttons to give you a 'point and click' mouse interface.

If you type FOO and then press <TRACE>, the function FOO will pop-up in a window on your screen, and your cursor will point to the first executable line which will be highlighted. If you now press <EXECUTE> that line will be executed and the cursor moved to the next line. If instead you pressed <TRACE>, the line would be executed with trace on any defined functions and operators used on that line. For example, if the line traced in FOO said Z←AVERAGE Y, a new window containing the function AVERAGE would pop-up overlapping the FOO window. You could then single-step through AVERAGE or trace deeper into your application as you wish. Execution can be restarted at any arbitrary line by simply moving the cursor to that line and pressing <EXECUTE> or <TRACE>.

The function on top of the stack is always available for editing. Other functions and variables can be edited by pointing at their name and pressing <EDIT>. At any time you can have the SESSION window, the stack in several overlapping windows, and one or more edit windows active. You can move between windows with a single keystroke or by moving the mouse, and data can be cut from one window and pasted in another.

If an APL event occurs, the event message will be displayed, the stack automatically displayed with the top function in an editable window, and the cursor placed on the line at which execution halted.

AIX on PS/2s and Mainframes

Dyalog APL will shortly be available for AIX (IBM's implementation of Unix) on PS/2s and mainframes in addition to 6150.

AIX on PS/2s will offer a similar environment to the IBM 6150, but with the emphasis on multi-tasking rather than multi-user facilities. AIX/370 will extend IBM's Unix offering

to a single operating environment spanning PS/2 workstations, departmental 6150 computers, and the corporate mainframe. As it runs under VM, AIX/370 will provide the means to integrate AIX-based distributed processing with mainstream DP applications and data.

IBM is putting more expenditure and effort into developing AIX than it is into PS/2s and OS/2 put together, and despite the apparent conflict with SAA, Dyadic expects AIX to capture a significant share of the commercial computing market in the future.

Dyalog APL for MS-DOS

Dyadic is currently working on an implementation of Dyalog APL that runs under MS-DOS on 386-based PCs such as the IBM PS/2 Model 80 and Compaq 386. The DOS version is based on the standard Unix implementation of Dyalog APL, but with changes necessary to move from a multi-tasking to a single task environment. The bulk of the interpreter, the component file system, line-editor, and even π SM, already work and are demonstratable. However several tasks (including the completion of π SM, and the new development environment referred to above) will be completed before the product is released. Performance of the DOS version is similar to that of Dyalog APL under Xenix. Under DOS the component file system is slower than it is under Unix, but highly competitive compared with other DOS APLs. Like Dyalog APL under Xenix, graphics will be supported by an interface to GSS*CGI.

APL2 for the IBM PC and PS/2

by David A. Selby

APL2 is IBM's strategic interactive programming language.

The announcement and availability of a second generation APL2 system for IBM PC and Personal System/2 Machines is a significant step forward for the APL programming language. APL2/PC offers a high degree of connectivity with other PC DOS based systems and 370 APL2 systems. Functions, operators and data may be migrated via the)IN &)OUT import / export commands and the use of an upload / download capable terminal emulator program.

Deviations

We distribute the APL2 language reference manual from the 370 product to demonstrate our level of compatibility. In our 300 page PC-specific guide we list the deviations from that document. There currently remain just 3 major areas of difference, these are:

- Complex arithmetic
- Selective specification
- $\square$ NA.

Our plan for the future is to reduce this list as further releases permit. It might be worth examining briefly the shipped alternatives to the above. Instead of $\square$ NA, the AP2 auxiliary processor permits the execution of external programs in other languages.

An external program provides a subset of the Complex arithmetic function. Work is still in progress on the definition of selective specification and is planned to be addressed in the future.

APL2/PC 16 bit

This is the smallest second generation APL system available in the market place today. It offers excellent performance, better than many first generation APL interpreters. Due to its economy of size, it also yields a generous 400 Kilobyte workspace in a 640 Kilobyte machine giving the user extra flexibility.

The Packager

Packaging an APL2 written application, leaves you with a single ".EXE" file which may be freely distributed without further licensing requirements to IBM. This for the first time makes APL2 the same as any other application generating language.

Once an APL2 application is packaged then all the "APLness" is removed - it no longer has the APL2 keyboard handler, soft character set, and if the application fails then it is immediately terminated, to the PC DOS prompt line, making it no different from any other programming language. Packaging is currently only available with the 16 bit interpreter.

APL2/PC 32 bit

For those users owning IBM Personal System/2 Model 70 and 80 machines wishing to exploit the power of the 80386 processor installed in these machines, coupled with an 80387, then this system provides a powerful APL2 interpreter which breaks through the memory barrier providing up to 15.7 megabyte workspaces on suitable machines. This is made possible by IBM's own extended memory manager. Coupled with an improvement in performance over its smaller 16 bit version, this interpreter is not large at just 188K bytes.

Port Time

APL2/PC is derived from our own high level IL language which represents the definition of an APL2 system. This facility significantly reduces port time with obvious benefits of cost and a wider variety of machine offerings to our customers.

Price Performance

APL2 provides excellent price performance, in the three systems discussed above, APL2/PC 16 bit, APL2/PC 32 bit, and the Packager, are offered complete with 18 device drivers, 29 workspaces (or 925 functions and operators) for £348 (plus VAT).

Conclusion

APL2 provides a write-once programming environment, giving second generation language power from the smallest IBM PC to the largest IBM Super Computer - the IBM 3090 with Vector Feature.



APL DEVELOPMENT PROGRAMMERS

British Airways use APL extensively to develop decision support systems for the airline. We are urgently seeking development programmers to work on a number of projects at Heathrow, providing the opportunity to get involved in both rewarding and interesting developments.

Applicants should have at least one year's experience of development using APL and preferably more. Comprehensive training will be provided to candidates where required.

In return, you will receive a competitive salary as well as an excellent benefits package which includes holiday bonus, profit sharing and favourable holiday travel opportunities. We offer a performance related pay scheme that will fully reward good performers.

For further details about the above vacancies, please telephone Nick Jones on 01-751 8150.

Alternatively, send a full c.v. to: Recruitment & Selection, Ref: TN/1143/NJ/VECTOR, British Airways Plc, "Meadowbank", PO Box 59, Hounslow TW5 9QX.



**BRITISH
AIRWAYS**

GENERAL ARTICLES

This section of **VECTOR** is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers

SCORE: An APL Program for the Scoring of Tests

by Jerry M. Brennan and Jennifer E. Daly

Introduction

Score is a general purpose program for the scoring of tests. The program scores data according to a key that is customised by the user, and options exist for keyed, reversed, and weighted scoring of data.

Detailed instructions for the program (a printout of these is included as an Appendix) can be accessed anytime by the user. Two methods of input are allowed:

1. Interactive input, where the user is queried for up to 512 (including blanks) elements of data.
2. Batch input, which allows for a virtually unlimited amount of data.

Output includes the scoring key(s), subject identification, input vector, total number of elements entered, and scores for all the factors (i.e. each scoring scale).

Score has three advantages:

- the program is flexible, with uncomplicated, customised keys that allow scoring of an unlimited number of items and factors.
- data can be scored in a variety of ways by changing the keys and rescoreing.
- large sets of scoring keys for different tests (e.g. batteries) can be compactly stored and easily retrieved for future use.

Score in APL

SCORE is a dyadic function with both arguments containing segmented character strings. The right argument KEY contains the scoring key. This argument requires a major segment for each scale and optional minor segments for special scoring features of each scale. The optional left argument DATA contains subject identification and item responses for each subject.

If DATA contains no elements, SCORE reads data interactively and the user is prompted to enter subject identification and item responses. In this mode the major segments of the KEY argument (i.e. each scoring key) as well as each scale in KEY and DATA (or, if interactive, each subject entry) are separated by a carriage return. Thus, for input, each

subject's data and each scoring key for that data is entered on a new line. Each minor segment within the scoring key and within the data set is separated by '/'.

Two main functions are used for scoring: SCORE and RECODE. SCORE is the primary function that calls RECODE if recoding of data is done. In addition, two one-line utility functions, SSINDEX and SSSHAPE, are also used. For all functions, the variable DATA contains all subjects' data, and SCORE loops through DATA for each subject. Within SCORE, the variable D contains data for one subject and F is a counter indicating which factor is being calculated. Similarly NF is the number of factors contained in KEY, S is the subject loop counter, and NS contains the number of subjects stored in DATA.

```
DATA SCORE KEY;INT;D;NS;NF;F;S;T;V;VALS;□ELX
-----
```

```
[1]  A Key is segmented string (e.g., 'ONE/1 2/1/0'). Each scale
[2]  A is separated by □TCNL & "/" separates fields in a scale.
[3]  A Fields: 1=scale name 2=items 3=multiplier 4=add.
[4]  A DATA is the optional left argument of a character vector
[5]  A with Subject ID, "/", and the item values. Each on newline.
[6]  A If DATA does not exist, the program asks for Subj & data.

[7]  0 0 p1 □POKE 96
[8]  'Scoring key:' □ NF+SSSHAPE □TCNL,□+KEY
[9]  →LOOP×LINT+0=□NC 'DATA'
[10]  NS+SSSHAPE DATA+□TCNL,DATA □ S+0
[11]  NXTS:→0×lNS+S+1 □ D+1+DATA SSINDEX S
[12]  '-----> Subject ID: ',_1+(T+Dl'/')↑D □ 'Data :',D+T+D
[13]  +nUM
[14]  LOOP:□+ '-----> Subject ID: ' □ →0×l0=p□ □ □+ 'Data : '
[15]  →0×l0=pD+□
[16]  nUM:→OK×l' 'εD □ D+((2×pD)p 1 0)\D
[17]  OK:D+εD □ (↑pD), ' numbers entered.' □ F+0
[18]  NXTf:→PAU×lNF+F+1 □ V+ '/' , (1+(□TCNL,KEY)SSINDEX F), '/1/0'
[19]  VALS+D RECODE 1+V SSINDEX 2
[20]  (1+V SSINDEX 1), '= ', ↑+/(ε1+V SSINDEX 3)×VALS+
    (ε1+V SSINDEX 4)
[21]  →NXTf
[22]  PAU:→LOOP×LINT □ →NXTS
    V
```

Lines 1-6 are comments that describe essential features of the program.

The □POKE on line 7 discards the character prompt utilized in the interactive mode.

Line 8 displays the scoring key(s) and determines the number of factors (i.e. the number of major segments) in KEY. The results of this line are then stored in NF, the loop factor counter.

Line 9 skips to LOOP (i.e. the interactive mode) if DATA does not exist. The program then reads data interactively (lines 14-16).

Line 10 catenates a carriage return to DATA, and calls SSSHAPE to determine the shape (i.e. the number of subjects) in DATA. It stores the result in Ns and initialises the subject counter S.

SSSHAPE, the function used to count the number of arguments in the segmented strings, returns the number of carriage returns present in the string. A carriage return is thus added to KEY (and also to DATA) so that the first element is included in the count.

```
R+SSSHAPE SS
-----
[1]  AReturns the shape of a segmented string
[2]  R++/SS=1+SS
    ▽
```

Similarly SSINDEX returns selected delimited segments from a longer string, the delimiter for the segmented string is the first character (i.e. carriage return).

```
R+SS SSINDEX I
-----
[1]  AFunction for the indexing of segmented strings
[2]  R+((+\SS=1+SS)€I)/SS
    ▽
```

Line 11 of SCORE increments the subject counter and exits when all the data has been processed. Otherwise the next subject is selected and the data stored in D.

Line 16 checks if there are blanks in D. If not, spaces are automatically inserted between the single digit integers; this facilitates entry of single digit data.

Line 18 goes to PAU when all of the factors have been calculated. This line also catenates the default multiplier, 1, and the default adder, 0, to the segmented string, and stores the result in V. These default options eliminate the need for user input of values when items are added with equal weight.

Line 19 passes the 'recode' subsegment to the RECODE function. If this subsegment is not empty, the data is recoded:

```
R←D RECODE C;T;NR;N;V;M;I
-----
```

```
[1]  A Recode data D from S e.g 1:1=4 3=7$3:1=4 2=9
[2]  A IF D→1 2 2 ^4 then R→4 2 9 ^4 [2nd value not recoded]

[3]  +rECx1': 'eC ◇ R+(xT)xD[|T+±C] ◇ +0
[4]  rEC:NR←SSSHAPE C+'$',C ◇ N+0 ◇ R+0
[5]  L1:→0×1NR←N+1 ◇ V←1+C SSINDEX N
[6]  V[(Vε':=')/lpV]←' ' ◇ I←1+V+±V
[7]  M+{(0.5×^1+pV),2)p1+|V
[8]  R←R,(×I)×1+{(M[;1]εD[|I])/M[;2]},100
[9]  +L1
V
```

Line 20 of SCORE computes the scale score by multiplying and then adding the possibly recoded values.

Line 21 returns to score the next factor. Finally if the program is being run interactively, line 22 goes to LOOP to query the user for additional data. Otherwise the program returns to nXT for the next subject.

Jerry M Brennan
University of Hawaii at Manoa
Department of Sociology, Porteus Hall
2424 Maile Way, Honolulu
Hawaii 96822

Appendix: Screen Hardcopy of Program Notes

To PRINT these NOTES, toggle CTRL-PRNTSC on or off.

The APL workspace name is TESTSCOR. Load this workspace with one of the following:

```
)LOAD TESTSCOR      [default drive (same as APL system)]
)LOAD 0 TESTSCOR     [drive (e.g., 0=A 1=B 2=C)]
```

To SEE the KEYS & DATA in the workspace TESTSCOR, type:

```
)VARS
```

SCORING KEY:

1. To CREATE the scoring key for the test type)EDIT and some name - probably the name of the test.

For example:

```
)EDIT SHORTMEMORY [Up to 77 characters with no blanks.]
```

2. ENTER the scoring key for each factor on one line.

For example:

```
FACTOR NAME/ITEMS ON FACTOR/MULTIPLIER/AMOUNT TO ADD
FACTOR NAME/ITEMS/MULTIPLIER/AMOUNT TO ADD
...etc
```

WHEN YOU ARE FINISHED CREATING OR MODIFYING THIS KEY, LEAVE THE EDITOR WITH Ctrl-E.

To SAVE the scoring key for use on another day, save the workspace with:
)SAVE

KEY: EXAMPLE 1: (Note that each factor begins on a new line)

```
ONE/1 2/1 3/~2
TWO/1 3 4/1/0
THREE/4 ~5/1/0
FOUR/2:1=10 2=20$5:1=30$~6:1=5 2=7
```

This test has three factors labeled ONE, TWO, THREE, and FOUR.

The first factor, ONE, contains Items one and two. The multiplier for Item one is 1 and for Item two the multiplier is 3. Minus 2 is added to all Items. The second factor, TWO,

contains three Items and all are multiplied by 1 and have 0 added. The third factor, THREE, contains Items four and five. Item four is added and Item five is subtracted.

Finally, factor FOUR has three Items: Two, five, and six. Items 2 and 5 are added and 6 is subtracted.

If Item 2 has a value of 1, it is scored 10; if it has a value of 2, it is scored 20. All other values of Item two are scored 0.

Similarly, if Item 5 has a value of 1, it is scored 30 otherwise, it is scored 0.

If Item 6 has a value of 1, it is scored 5. A value of 2 on Item 6 is scored seven. Otherwise, this item is scored 0. Note: Since Item 6 has a minus, these values are subtracted.

NOTE:':' delimits item numbers from their scoring weights. In the above example, a score of 1 equals 10 for Item 2. 'n' delimits items and their weights from each other. Also, the high minus is used to subtract items. This minus is Alt-2 on the keyboard and may look like a small two on your screen. For all computations, addition is done first, then multiplication and, finally, the sign is changed.

KEY: EXAMPLE 2:

FACTOR ONE/1 2
FACTOR TWO/3 4 5

EXAMPLE 2 has two factors: The first has two items and the second has three items. By default, nothing will be added to the items and there is no multiplier.

KEY: EXAMPLE 3:

This last example illustrates a scoring situation in which some items are REVERSED. The scoring for one REVERSED factor might be as follows:

Item 1 is punched 1, 2 ,3 and is to be scored 1, 2, 3.
Item 2 is punched 1, 2, 3 BUT is to be scored 3, 2, 1.

This might be keyed as follows:

FACTOR ONE/1=3\$2=2\$3=1

DATA ENTRY:

1. CALL the program with the following command:

```
SCORE SHORTMEMORY    [The name of the program is always SCORE,  
                      you insert the name of your key--in this case  
                      SHORTMEMORY.]
```

The program will query for a "Subject ID". This identification can be any length and any characters--including blanks. After Subject ID is entered, the program queries for the items. Data consisting of all single digit numbers may be entered as a string (i.e., without blanks between each single digit). Data consisting of double (or more) digits must be entered with blanks separating each number. To QUIT data entry, strike RETURN when queried for "Subject ID".

Note: Continue typing at the end of each line. The program will wrap automatically. The limit is 512 characters, including blanks.

2. Another way to ENTER DATA is to use the editor as you did for the scoring keys. However, this time a variable will be created that contains the data.

For example:

```
)EDIT SUBS           [Up to 77 characters with no blanks]
```

Enter the Subjects' ID followed by a "/". Next, enter the data. (See above for information about data entry). Begin each new subject on a new line. For example, the data for two subjects might look like this:

```
Sally Brown female age 42/12321  
Bill Douglas male age unknown/22213
```

When data entry is complete, quit with CTRL-E and save with)SAVE.

TEST SCORING:

1. To RUN the scoring program (using the key and data created in the last two examples):

```
SUBS SCORE SHORTMEMORY
```

Many keys and subject data can be saved in this workspace. In fact, you can save over 500 separate variables.

2. To ERASE a key or a data file, do the following:

```
)ERASE SHORTMEMORY    [Erase working copy in memory]  
)SAVE                 [save the erased copy onto the disk]
```

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Prize Caption Competition (Result)



We think we ought to run a subsidiary competition for anyone who can explain this entry from Tony Caporn:

"Question: what is the shape of the enclose of Mike Kingston ... pc 'MIKE KINGSTON' "

"Answer: An empty vase"

We can also give an honorable mention to Edwin Ashworth for the following:

"This lower-case 'i' is a little large for your keyboard Son."

However the prize has to go to John Searle for:

L: Kingston upon Canterbury (Not upon Thames)

R: (Not Kingston upon Thames) upon Canterbury

L: An APL developer with prototype function symbol

R: New function symbol with prototype user community.

... working from Right to Left ...

BEAM ME DOWN SCOTTY

A cheque for £75 is on its way. We await Mike Kingston's riposte with interest ...

Hacker's Corner: DOS Environment Variables

by Adrian Smith

Passing bits of information from DOS into your APL application can be done in many devious ways. The least useful is probably the X,Y,Z startup options, which give you the possibility of just three integers! Another approach is to have a standard ASCII file (not a .ASF like APLLIBS) called something like CONFIG.APL with various settings written into it. This is fine for specific applications, but of course it is one more thing to lose, or put in the wrong directory, or whatever.

If you want something relatively small and simple, why not use the excellent DOS facility which effectively lets you define global variables in the DOS environment? This method has the advantage that it is easy for any non-APL user to change the settings (probably with a quick edit on AUTOEXEC.BAT) and the values are always available to your application. The only snag is tracking down the settings from within APL ... for the code fragment below you must thank M. Philippe Denoyelle of the French group AFCET:

```
R←DOSΔSYMB REQ;AV;S;V;X;⊞IO;⊞SEG
```

```
-----
[1]      ⑆ Read any DOS SET xxx=yyy commands in the environment
[2]      ⑆ Note that '@' is 'Δ' and '$' is 'Δ' and '=' is '+'
[3]      ⑆
[4]      ⑆ REQ is the prefix we want, e.g. SYMB 'ΔP' will collect
[5]      ⑆ any vars beginning with '@p' in DOS.

[6]      ⊞IO←0

[7]      ⑆ Find "Program Segment Prefix" ....

[8]      ⊞SEG←0 ⋄ S←~16+256⊞PEEK 130 129
[9]      ⊞SEG←S ⋄ S←256⊞PEEK 45 44
[10]     V←0
[11]     Retr:⊞SEG←S ⋄ V←V,⊞PEEK1256 ⋄ X← 0 0 ⊞V ⋄ →(V/X)↑Done
[12]     S←S+16 ⋄ →Retr

[13]     ⑆ Translate from numeric, and erect into a table.

[14]     Done:AV←⊞AV ⋄ AV[64 36 61]←'ΔΔ+'
[15]     V←AV[(X11)↑V] ⋄ R←⊞TCNUL ΔTAB V
[16]     REQ←,REQ ⋄ →(⊞REQ)↑0
[17]     R←(R[;⊞REQ]⋄.=REQ)≠R
V
```

```
DOSΔSYMB 'C'
COMSPEC←D:\COMMAND.COM
```

Technical Correspondence

An IBM Bug to watch for, when moving to APL2

From: John Warden

16 May 1989

I would like to draw your attention to a major programming bug in some of the public workspaces distributed by IBM with VSAPL, which only becomes apparent when systems developed using certain of IBM's utility functions are migrated from VSAPL to APL2.

The problem is in the function OPEN, which is found in all of the utility workspaces for handling AP126. On line 3 of this function is a conditional branch to LO. However, there are two lines labelled LO, lines 6 and 18. In VSAPL, if there are duplicated labels, branching is to the first occurrence, in this case to line 6. This is the correct path through the function so no error occurs. However a problem arises when workspaces which use this function are migrated to APL2. In APL2 the action taken when there are duplicated line labels is to branch to the highest line number. In this case the branch would be to line 18. This line issues an error message and exits the function.

The label LO on line 18 appears to be redundant as there is no apparent reason for a branch to this line. The solution is to check all workspaces migrated to APL2 from VSAPL which contain the IBM utility functions for communicating with AP126 for the existence of the OPEN function and edit it to remove the label on line 18.

Yours faithfully,

John E Warden,
15 Uplands,
Hungerford,
Berkshire.

An Ambivalent Operator

From: David Piper

24th February 1989

Here is an APL2 operator of use in ambivalent functions. One use of ambivalent functions is to elide the left argument, and thereby force it to take a default value. The methods normally used to achieve this are, for example:

```
[...] →(2=⊖NC 'LEFT')/dyadic  
[...] LEFT←default  
[...] dyadic:
```

or perhaps ...

```
[...] ⍺(0=⍵NC 'LEFT')/'LEFT+default'
```

Neither of these solutions is ideal - the first embodies too much apparent complexity to achieve such a simple effect - code in which errors can be made. The second uses execute, which is bad for all the normal reasons!

The defined operator listed below allows the effect of the second example to be achieved without using execute:

```
VR←DEFAULT(FUNC ΔMONADIC CLASS) RIGHT
[1] A OP: default func right if class is not 2
[2] R←0                                A ASSUME DYADIC CALL
[3] +(2=CLASS)/0                      A EXIT IF CALL WAS DYADIC
[4] R+1(DEFAULT FUNC RIGHT)          A EXEC WITH DFLT ARGUMENT
▽
```

The operator is used as follows:

```
VAVE←SIZE AVERAGE NUMS;RUN
[1] A FN: MOVING OR NORMAL AVERAGE DEPENDING ON ⍵NC 'SIZE'
[2] →(↑(RUN AVE)+(ρNUMS){AVERAGE ΔMONADIC ⍵NC 'SIZE'}NUMS)/0
[3] AVE←(SIZE+NUMS)÷SIZE
▽
```

I trust the above is of interest, and look forward to any feedback.

Yours sincerely

D.B. Piper
41 Sandown Drive
Rainham, Kent

Setting PF keys and Other Initialization

From: Nicholas Small

April 1989

I have always found it very convenient for program development to have such frequently used strings as 'EDIT' and 'SAVE' on programmed function keys. With APL*PLUS/PC these strings can be incorporated in the interpreter so that they are there when one starts up APL. Consequently, I was somewhat disappointed when I discovered that it is not possible to have preset PF keys with APL*PLUS II.

We have a number of PC based APL applications, all run via DOS batch files, each of which calls the batch file which runs APL, passing to it the name of the desired workspace. The obvious thing to do to get values on PF keys, without having to edit every

workspace, is to use an initializing workspace which, having done its work, loads the application workspace. The difficulty is: how to pass the name of the workspace? The technique I found makes use of the output redirection facility in DOS. My batch file to call APL*PLUS II has the following lines in it:

```
SET TEMP= %1 %2
IF X%1==X SET TEMP=?
ECHO %TEMP% > C:\TEMP\APLWS.NAM
SET TEMP=
D:\APLII\APLSM J=D:\APLII\XAPL387 Q='8 INIT'
```

The third line writes the name of the required workspace, or '?' if none was specified, to a temporary file. The last line invokes APL, requesting a quiet load of the initializing workspace. (If you need to have parameters other than the workspace name passed to the APL-calling batch file, then the latter will have to be correspondingly more sophisticated.)

The latent expression in INIT performs the initialization, which clearly need not be restricted to setting up PF keys, and then calls the following function:

```
▽ GETWS;X
[1]  A Retrieve the name of the ws the user wanted, and load it
[2]  A If nothing requested, or error, then CLEAR
[3]  □SA+ 'CLEAR'
[4]  X+~2+NCREAD 'C:\TEMP\APLWS.NAM'
[5]  +( '?'=1pX)/0
[6]  ⚡'□LOAD''',X, ''''
▽
```

The function NCREAD reads the contents of a native file as a character string (excluding any end-of-file marker); the carriage-return and line-feed, which DOS adds after the workspace name, are dropped. The loading is done via an execute so that, if an error occurs during the □LOAD, the name of the requested workspace will be displayed (rather than just the name of the variable, X).

Symbolic Logic and the Algebra of Propositions

a survey by Joseph L.F. De Kerf

Abstract

A survey is given of symbolic logic and the algebra of propositions. It is shown how this algebra is implemented in APL 1 and may be simulated with the relational and maximum/minimum functions, the latter being especially useful for the manipulation of fuzzy sets. Implementations and standardisation of extended dyadic $\vee$ and $\wedge$ in APL 2 - the gcd and lcm of the arguments - are discussed.

Symbolic Logic and the Algebra of Propositions - A Survey

In symbolic logic and the algebra of propositions, propositions are generally represented by letters - for instance P, Q, R, etc. If a proposition is true, it's given the value 1 (or T for TRUE), and if the proposition is false, it's given the value 0 (or F for FALSE). From any proposition, or set of propositions, other propositions may be formed. Let us define the following basic functions:

(1) The Negation (NOT)

The negation of the proposition P is the proposition $\sim P$, being false when P is true, and being true when P is false.

Other notations are e.g.: P' P^* $\neg P$

(2) The Disjunction (OR)

The disjunction of the propositions P and Q is the composite proposition $P \vee Q$, which is true when P or Q or both P and Q are true, and which is false when P and Q are false.

Other notations are e.g.: $P \cup Q$ $P + Q$

(3) The Conjunction (AND)

The conjunction of the propositions P and Q is the composite proposition $P \wedge Q$, which is true when both P and Q are true, and which is false when P or Q or both P and Q are false.

Other notations are e.g.: $P \cap Q$ $P \& Q$ $P \times Q$

As $P \vee Q$ is true when both P and Q are true, the disjunction is also called the inclusive disjunction.

P	$\sim P$
1	0
0	1

P	Q	$P \vee Q$
1	1	1
1	0	1
0	1	1
0	0	0

P	Q	$P \wedge Q$
1	1	1
1	0	0
0	1	0
0	0	0

Table 1

Truth tables of the negation and of the disjunction $\vee$ and conjunction $\wedge$ are given in Table 1. With these truth tables it may be shown that the functions defined, the disjunction $\vee$ and the conjunction $\wedge$, satisfy the postulates of Huntington(1):

P1. The functions $\vee$ and $\wedge$ are commutative:

$$\begin{aligned} P \vee Q &\leftrightarrow Q \vee P \\ P \wedge Q &\leftrightarrow Q \wedge P \end{aligned}$$

P2. The values 0 and 1 are identity elements relative to the functions $\vee$ and $\wedge$ respectively:

$$\begin{aligned} R \vee 0 &\leftrightarrow 0 \vee R \leftrightarrow R \\ R \wedge 1 &\leftrightarrow 1 \wedge R \leftrightarrow R \end{aligned}$$

P3. Each function $\vee$ and $\wedge$ is distributive over the other:

$$\begin{aligned} P \vee (Q \wedge R) &\leftrightarrow (P \vee Q) \wedge (P \vee R) \\ (P \wedge Q) \vee R &\leftrightarrow (P \vee R) \wedge (Q \vee R) \\ P \wedge (Q \vee R) &\leftrightarrow (P \wedge Q) \vee (P \wedge R) \\ (P \vee Q) \wedge R &\leftrightarrow (P \wedge R) \vee (Q \wedge R) \end{aligned}$$

P4. The negation satisfies the identities:

$$\begin{aligned} R \vee (\sim R) &\leftrightarrow (\sim R) \vee R \leftrightarrow 1 \\ R \wedge (\sim R) &\leftrightarrow (\sim R) \wedge R \leftrightarrow 0 \end{aligned}$$

This means that the algebraic structure $\{0,1\}, \vee, \wedge$ is a boolean algebra (2), whereby:

- the disjunction $\vee$ is the boolean addition;
- the conjunction $\wedge$ is the boolean multiplication;
- 0 and 1 are the identity elements versus the boolean addition and multiplication respectively;

- and the negation $\sim$ is the boolean complementation.

The algebraic structure $\{0,1\}, \vee, \wedge$ being a boolean algebra, the following identities may be derived from the postulates:

$$T1. R \vee R \leftrightarrow R \text{ and } R \wedge R \leftrightarrow R$$

$$T2. R \vee 1 \leftrightarrow 1 \vee R \leftrightarrow 1 \text{ and } R \wedge 0 \leftrightarrow 0 \wedge R \leftrightarrow 0$$

$$T3. P \vee (P \wedge Q) \leftrightarrow (P \wedge Q) \vee P \leftrightarrow P \text{ and } P \wedge (P \vee Q) \leftrightarrow (P \vee Q) \wedge P \leftrightarrow P$$

$$T4. P \vee (Q \vee R) \leftrightarrow (P \vee Q) \vee R \text{ and } P \wedge (Q \wedge R) \leftrightarrow (P \wedge Q) \wedge R$$

$$T5. \sim(\sim R) \leftrightarrow R$$

$$T6. \sim(P \vee Q) \leftrightarrow (\sim P) \wedge (\sim Q) \text{ and } \sim(P \wedge Q) \leftrightarrow (\sim P) \vee (\sim Q)$$

Of course these identities may be derived directly from the truth tables. Deriving the identities from the postulates, however, proves that those identities are valid for every boolean algebra.

The Complete Set of Dyadic Logical Functions

Fundamentally, $2 \times 2 \times 2$ or 16 dyadic logical functions may be defined. Truth tables of these 16 dyadic logical functions F1...F16 are given in Table 2.

P	Q	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0
1	0	1	1	1	1	0	0	0	0	1	1	1	1	0	0	0	0
0	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0
0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0

Table 2

F1 and F16 are trivial in that they have no dependence on the arguments P and Q: $F1 \leftrightarrow 1$ and $F16 \leftrightarrow 0$. F4 and F6, and F11 and F13 are special cases since they depend on only one of the arguments. F4 and F6 return P and Q respectively. F11 and F13 return $\sim Q$ and $\sim P$ respectively, and as such define the negation $\sim$ of one of the arguments - which actually is a monadic function.

From the 10 remaining nontrivial dyadic functions, the definition of two of them - the disjunction F2 and the conjunction F8 - has been given in the previous section. The 8 remaining nontrivial dyadic functions may be defined as follows:

(4) The Non-Disjunction (NOR) - F15

The non disjunction $P \nabla Q$ of the propositions P and Q is the negation $\sim$ of the disjunction $P \vee Q$. The composite proposition is true when both P and Q are false, and is false when P or Q or both P and Q are true.

$$P \nabla Q \leftrightarrow \sim(P \vee Q) \leftrightarrow (\sim P) \wedge (\sim Q)$$

Other notations are e.g.: $P \wedge Q$ $P \cap Q$ $P | Q$ $P + Q$ $P \div Q$

(5) The Non-Conjunction (NAND) - F9

The non-conjunction $P \star Q$ of the propositions P and Q is the negation $\sim$ of the conjunction $P \wedge Q$. The composite proposition is true when P or Q or both P and Q are false, and is false when both P and Q are true.

$$P \star Q \leftrightarrow \sim(P \wedge Q) \leftrightarrow (\sim P) \vee (\sim Q)$$

Other notations are e.g.: $P \uparrow Q$ $P \div Q$

(6) The Implication or Conditional - F5

The implication $P \supset Q$ is the composite proposition which is false when and only when P is true and Q is false.

$$P \supset Q \leftrightarrow (\sim P) \vee Q \leftrightarrow \sim(P \wedge (\sim Q))$$

Other notations are e.g.: $P \rightarrow Q$ $P \supset Q$

(7) The Non-Implication - F12

The non-implication $P \nabla Q$ is the negation $\sim$ of the implication $P \supset Q$ ($P \nabla Q \leftrightarrow \sim(P \supset Q)$). The composite proposition is true when and only when P is true and Q is false.

$$P \nabla Q \leftrightarrow P \wedge (\sim Q) \leftrightarrow \sim((\sim P) \vee Q)$$

(8) The Inclusion or Converse Implication - F3

The inclusion $P \subset Q$ is the composite proposition which is false when and only when P is false and Q is true.

$$P \subset Q \leftrightarrow P \vee (\sim Q) \leftrightarrow \sim((\sim P) \wedge Q)$$

Other notations are e.g.: $P \leftarrow Q$ $P > Q$

(9) The Non-Inclusion - F14

The non-inclusion $P \not\subset Q$ is the negation $\sim$ of the inclusion $P \subset Q$ ($P \not\subset Q \leftrightarrow \sim(P \subset Q)$). The composite proposition is true when and only when P is false and Q is true.

$$P \not\subset Q \leftrightarrow (\sim P) \wedge Q \leftrightarrow \sim(P \vee (\sim Q))$$

(10) The Equivalence or Biconditional - F7

The equivalence $P \equiv Q$ of the propositions P and Q is the composite proposition which is true when and only when both P and Q are true or both P and Q are false.

$$\begin{aligned} P \equiv Q &\leftrightarrow (P \wedge Q) \vee ((\sim P) \wedge (\sim Q)) \\ &\quad (P \wedge Q) \vee (\sim(P \vee Q)) \\ &\quad (P \wedge Q) \vee (P \vee Q) \\ &\quad (P \vee (\sim Q)) \wedge ((\sim P) \vee Q) \end{aligned}$$

Other notations are e.g.: $P \leftrightarrow Q$ $P \phi Q$ $P \nabla Q$

(11) The Non-Equivalence or Exclusive Disjunction - F10

The non-equivalence $P \not\equiv Q$ of the propositions P and Q is the negation $\sim$ of the equivalence $P \equiv Q$ ($P \not\equiv Q \leftrightarrow \sim(P \equiv Q)$). The composite proposition is true when and only when P is true and Q is false or P is false and Q is true.

$$\begin{aligned} P \not\equiv Q &\leftrightarrow (P \vee Q) \wedge ((\sim P) \vee (\sim Q)) \\ &\quad (P \vee Q) \wedge (\sim(P \wedge Q)) \\ &\quad (P \vee Q) \wedge (P \nabla Q) \\ &\quad (P \wedge (\sim Q)) \vee ((\sim P) \wedge Q) \end{aligned}$$

Other notations are e.g.: $P \vee Q$ $P \perp Q$ $P \Delta Q$

As the equivalence $P \equiv Q$ is the conjunction $\wedge$ of the conditionals $P \supset Q$ and $P \subset Q$, it is also called the biconditional. As the non-equivalence $P \not\equiv Q$ is true when P or Q are true, but is false when both P and Q are true, it is also called the exclusive disjunction (this in contrast with the inclusive disjunction $P \vee Q$).

A set of functions is called functionally complete provided every propositional function can be expressed in terms of the functions in the set. As shown above, the set $\{\vee, \wedge, \sim\}$ is functionally complete. As however, the conjunction $\wedge$ may be expressed in terms of the negation $\sim$ and the disjunction $\vee$, and the disjunction $\vee$ may be expressed in terms of the negation $\sim$ and the conjunction $\wedge$, the sets $\{\vee, \sim\}$ and $\{\wedge, \sim\}$ are also functionally complete:

$$\begin{aligned} P \wedge Q &\leftrightarrow \sim((\sim P) \vee (\sim Q)) \\ P \vee Q &\leftrightarrow \sim((\sim P) \wedge (\sim Q)) \end{aligned}$$

NOT	P	Q	OR $P \vee Q$	AND $P \wedge Q$	NOR $P \nabla Q$	NAND $P \pi Q$
$\sim 1 \leftrightarrow 0$	1	1	1	1	0	0
$\sim 0 \leftrightarrow 1$	1	0	1	0	0	1
	0	1	1	0	0	1
	0	0	0	0	1	1

Table 3

Finally, the negation $\sim$, the disjunction $\vee$, and the conjunction $\wedge$ may be expressed each in terms of the non-disjunction ∇ only, or in terms of the non-conjunction π only. This means that the functions ∇ and π are both functionally complete:

$$\begin{array}{ll}
 \sim P \leftrightarrow P \nabla P & \text{AND} \quad \sim P \leftrightarrow P \pi P \\
 P \vee Q \leftrightarrow (P \nabla Q) \nabla (P \nabla Q) & P \vee Q \leftrightarrow (P \pi P) \pi (Q \pi Q) \\
 P \wedge Q \leftrightarrow (P \nabla P) \nabla (Q \nabla Q) & P \wedge Q \leftrightarrow (P \pi Q) \pi (P \pi Q)
 \end{array}$$

The non-disjunction ∇ and non-conjunction π are generally called the Pierce Stroke Function (3) and the Sheffer Stroke Function (4) respectively. They are especially useful for the design of logical circuits in which only one type of gates is to be used (05).

Logical Functions and APL 1 (ISO APL)

The initial implementation of the Iverson notation at IBM in 1965 only supported the negation $\sim$ the disjunction $\vee$ and the conjunction $\wedge$.

It did not include the nor ∇ and the nand π functions. E.E.McDonnell reports that the introduction was suggested by Peter Calingaert, then at Science Research Associates in Chicago, sometime in 1967 (7).

Finally, the following logical functions were implemented in APL360-1968 (8) and its successors APLSV-1973 and VS APL-1975 (9): the monadic $\sim$ and the dyadic $\vee$ $\wedge$ ∇ π .

The functions are respectively called the NOT-OR-AND-NOR-NAND and are defined through their truth tables as shown in the adapted Table 3. It should be noted that in the ISO-APL Second Draft Proposal DIS8485 (10), the same names are given and that the functions are informally defined as boolean instead of logical:

NOTas the "boolean complement"
 ORas the "boolean sum"
 ANDas the "boolean product"
 NORas the "boolean complement of the boolean sum"
 NANDas the "boolean complement of the boolean product"

Negation	$\sim P$	$P=0$	$1-P$
Disjunction	$P \vee Q$	$P \geq Q=0$	$P \uparrow Q$
Conjunction	$P \wedge Q$	$P > Q=0$	$P \downarrow Q$
Non-Disjunction	$P \nabla Q$	$P=0$	$1-P \uparrow Q$
Non-Conjunction	$P \pi Q$	$P \leq Q=0$	$1-P \downarrow Q$
Implication	$Q \vee \sim P$	$P \leq Q$	$Q \uparrow 1-P$
Non-Implication	$P \wedge \sim Q$	$P > Q$	$P \downarrow 1-Q$
Inclusion	$P \vee \sim Q$	$P \geq Q$	$P \uparrow 1-Q$
Non-Inclusion	$Q \wedge \sim P$	$P < Q$	$Q \downarrow 1-P$
Equivalence	$(P \wedge Q) \vee P \nabla Q$	$P=Q$	$(P \downarrow Q) \uparrow 1-P \uparrow Q$
Non-Equivalence	$(P \vee Q) \wedge P \pi Q$	$P \neq Q$	$(P \uparrow Q) \downarrow 1-P \downarrow Q$

Table 4

As shown in Table 4, the logical functions may be simulated by the relational functions $\leq$, $=$, $>$, $\geq$, $\neq$ as well as by the combination of the minus function ($-$) with the maximum and minimum functions ($\uparrow$ and $\downarrow$).

Column 1 gives the logical expressions as derived under Sections 1 and 2. Column 2 gives the corresponding expressions based on the relational functions. It should be noted that the negations $P=0$ and $Q=0$ may be replaced by the negations $P \neq 1$ and $Q \neq 1$. An advantage of the logical expressions is that their domain is restricted to the near-boolean integer arguments 0 and 1, while this is not the case for the expressions based on the relational functions.

Column 3 gives the expressions based on the combination of the minus function with the maximum and minimum functions. Here too, the domain of the arguments is not restricted to the near-boolean integers 0 and 1, but the advantage of these expressions is that they may be used e.g. for the manipulation of the characteristic or membership vectors V of fuzzy sets $F \in (11)(12)$:

- where the domain of the membership vectors is the interval $[0,1]$;
- $V1 \uparrow V2$ gives the membership vector of the union $FS1 \cup FS2$;
- $V1 \downarrow V2$ gives the membership vector of the intersection $FS1 \cap FS2$;
- and $1 - V$ gives the membership vector of the complementation of FS .

A	B	GCD
1	1	1
1	0	1
0	1	1
0	0	0

A	B	LCM
1	1	1
1	0	0
0	1	0
0	0	0

Table 5

As $0 \div 1 \leftrightarrow 0$ and $0 \div 0 \leftrightarrow 1$ and as far as the arguments are restricted to the near boolean integers 0 and 1, the negations $1 - P$ and $1 - Q$ may be replaced by the expressions $0 \div P$ and $0 \div Q$ in Column 3. However, this means that the expressions are no longer applicable for the manipulation of the characteristic or membership vectors of fuzzy sets.

Note:..... In fact, although different symbols are used, the maximum $\uparrow$ and minimum $\downarrow$ functions are consistent extensions of the logical disjunction $\vee$ and conjunction $\wedge$ respectively.

Dyadic $\vee$ and $\wedge$ in APL2 (Extended APL)

The study of the greatest common divisor GCD and the least common multiple LCM constitutes an important subject of number theory.

They are generally represented by the notations:

- (A,B) for the greatest common divisor group GCD of A and B;
- $[A,B]$ for the least common multiple LCM of A and B.

A number D is called a greatest common divisor GCD of the numbers A and B if D is a common divisor of A and B which is a multiple of every other common divisor C:

$$D|A \quad D|B \quad C|A \text{ and } C|B \quad \text{imply } C|D$$

A number M is called a least common multiple LCM of the numbers A and B if M is a common multiple of A and B which is a divisor of every other common multiple C :

$$A|M \quad B|M \quad A|C \text{ and } B|C \quad \text{imply } M|C$$

Here $A|B$ means that B is divisible by A , such that there is some integer X such that $A \times X \leftrightarrow B$. We call B a multiple of A , and A a divisor of B .

The numbers D and M are not unique. In the real number domain both definitions determine two numbers which are each others additional inverse. In the complex number domain both definitions determine four numbers with the same modulus or absolute value and which are mutually associated. To remove this multiplicity, the greatest common divisor GCD may be restricted to the first-quadrant-associate, and the least common multiple LCM may be restricted to the corresponding associate which satisfies the identity:

$$A \times B \leftrightarrow (A, B) \times [A, B]$$

Several authors, such as G. Birkhoff and S. Mac Lane (13) and W. Greub (14), have proposed to denote the greatest common divisor GCD and the least common multiple LCM by the notations $A \vee B$ and $A \wedge B$. The motivation is that their algebra is a boolean algebra and is a consistent extension of the proposition algebra.

Indeed, let $S \leftrightarrow \{0,1\}$. Based on the above given definitions and conventions, it may be shown that the composition tables of the greatest common divisor GCD or $A \vee B$ and the least common multiple LCM or $A \wedge B$ of the elements of S are identical with the truth tables of the disjunction $\vee$ and the conjunction $\wedge$ respectively (Table 5).

As an illustration of the extension, let N be a non-negative integer (e.g. 30) and S the set of non-negative integer divisors of N $\{1,2,3,5,6,10,15,30\}$. It may be shown that the greatest common divisor GCD or $A \vee B$ and the least common multiple LCM or $A \wedge B$ of the elements of S satisfy the above mentioned postulates of Huntington. As a consequence, the algebraic structure $S, \vee, \wedge$ is a boolean algebra, whereby:

- the greatest common divisor $A \vee B$ is the boolean sum;
- the least common multiple $A \wedge B$ is the boolean product;
- N and 1 are the identity elements versus the boolean addition and boolean multiplication respectively;
- and $N \div C$ is the boolean complementation of C .

In fact G. Birkhoff and S. Mac Lane proposed the notations $(A, B) \leftrightarrow A \wedge B$ and $[A, B] \leftrightarrow A \vee B$, while W. Greub proposed the notations $(A, B) \leftrightarrow A \vee B$ and $[A, B] \leftrightarrow A \wedge B$. It's

true that the functions $\vee$ and $\wedge$ satisfy the duality law, but to preserve the above described consistency with the algebra of propositions, the definitions $(A, B) \leftrightarrow A \vee B$ and $[A, B] \leftrightarrow A \wedge B$ may not be interchanged: the greatest common divisor is an extension of the disjunction $\vee$ and the least common multiple is an extension of the conjunction $\wedge$.

In a presentation at the Working Conference on Mechanical Language Structures, Princeton, New Jersey, August 1963, K.E. Iverson (15) proposed to add $A \sqcup B \leftrightarrow (A, B)$ and $A \sqcap B \leftrightarrow [A, B]$ to his notation. At APL 75 E.E. McDonnell, then at IBM in Palo Alto, now with I.P. Sharp Associates, proposed to use the notations $A \vee B \leftrightarrow (A, B)$ and $A \wedge B \leftrightarrow [A, B]$ in APL. The mathematics behind the proposal are thoroughly discussed in the paper and - though couched in terms of integral arguments - the extension to real and complex arguments is easy. The extension - as formulated by E.E. McDonnell - has been included in Iverson's Dictionary of APL (17).

The proposed extension $A \vee B \leftrightarrow (A, B)$ and $A \wedge B \leftrightarrow [A, B]$ has been implemented in SHARP APL (IPSA) (18). The implementation has not been retained in the commercial versions of APL2, but has been implemented recently in APL90 (19). It has been included in the list of topics to be explored by the ISO APL Working Group in their preparation of Extended APL (20) and a text to be used for further examination has been edited by E.E. McDonnell (21).

Extensions of APL and their standardisation are very important for the future of the language. Some authors insist - and with justice - on keeping them clear and plain. The proposed extension satisfies these requirements. If APL is to maintain this prestige as a better mathematical notation and as the ideal programming tool to support the teaching of mathematics, the proposed extension must be incorporated in the so-called Extended APL.

A	B	$A \vee B$	$A \wedge B$	$(A \times B) \div A \vee B$	$(A \times B) \div A \wedge B$
0	0	0	0	$0 \div 0$	$0 \div 0$
A	0	FQA:A	0	0	$0 \div 0$
0	B	FQA:B	0	0	$0 \div 0$

Table 6

Note:In his paper presented at APL 75, E.E. McDonnell claims "...that the value of $0 \div 0$ in APL should be 0 (not 1 as currently the case)". The proposal was further elaborated by the author in a paper presented at APL 76 (22). Arguments pleading in favour of keeping the value of $0 \div 0$ as 1 in APL were given at APL80 (23).

From the identity $A \times B \leftrightarrow (A \vee B) \div A \wedge B$ we may derive the formulas

- if $A \vee B$ is different from 0: $A \wedge B \leftrightarrow (A \times B) \div A \vee B$;
- if $A \wedge B$ is different from 0: $A \vee B \leftrightarrow (A \times B) \div A \wedge B$.

The question arises if those formulas are still valid when $A \vee B$ or $A \wedge B$ or both $A \vee B$ and $A \wedge B$ are 0 (Table 6).

If A and B are 0, both $A \vee B$ and $A \wedge B$ are 0. Both formulas reduce to the form $0 \div 0$ and give 1. This suggests indeed that $0 \div 0$ should be set to 0 instead of 1.

If A and B are 0, but not both, $A \vee B$ is the first-quadrant-associate of the non-zero argument and $A \wedge B$ is 0. The first formula gives the correct result 0, but the second formula reduces to the form $0 \div 0$ and gives 1, while it should give the first-quadrant-associate of the non-zero argument. This suggests that $0 \div 0$ should be set neither to 0 nor to 1, but that it must be possible to assign it the appropriate value.

The real problem is indeed that $0 \div 0$ is undefined and that APL should give a DOMAIN ERROR to be consistent with our mathematics. If the APL implementation supports event handling, the user may trap the error message and assign $0 \div 0$ its appropriate value e.g. with a system function like the IBM APL $\square EA$ (Execute Alternate). A more acceptable solution however may be to implement a system variable which implicitly sets the current value of $0 \div 0$ (such as $\square DIV$ - the default value being 1).

Further Extensions

ISO APL only supports the monadic form of the tilde function, the boolean complement $\sim B$. Several implementations, however, support the dyadic form $A \sim B$, the set difference of sets A and B, which in Iverson's APL Dictionary is called "less". It has been included in the list of topics to be explored by the ISO APL Working Group in their preparation of Extended APL and a text to be used by this APL Working Group for further examination has been edited by E.E. McDonnell (24). Set Functions and APL are discussed in a paper to be published in one of the next issues of the APL-CAM Journal (25).

So far, there are no proposals for the extension of the dyadic logical functions $A \vee B$ and $A \wedge B$, nor for the monadic use of the dyadic logical functions ($\vee B \wedge B \vee B \wedge B$). Perhaps inspiration may be found in the fact that some authors use the symbols $\exists$ and $\forall$ for the logical existential qualifier $\exists$ and the universal qualifier $\forall$ respectively.

References

- 1 E.V. Huntington; The Algebra of Logic;
Transactions of the American Mathematical Society, Vol. 5, 1904; pp. 288-309.
- 2 J.E. Whitesitt; Boolean Algebra and its Applications;
Addison-Wesley Series in the Engineering Sciences; Addison Wesley
Publishing Company, Reading, Massachusetts, 1961
- 3 C.S. Pierce; On the Algebra of Logic
Chapter 1: Syllogistic
Chapter 2: The logic of non-relative terms
Chapter 3: The logic of relatives;
American Journal of Mathematics, Vol.3, 1880, pp. 15-57
- 4 H.M. Sheffer; A Set of Five Independent Postulates For Boolean Algebra - With
Applications to Logic Constants;
Transactions of the American Mathematical Society, Vol. 14, 1913; pp. 481-488.
- 5 D.Zissos; Logic Design Algorithms;
Harwell Series: Oxford University Press, London, Great Britain, 1972.
- 6 A Abian; Boolean Rings;
Branden Press Publishers, Boston, Massachusetts, 1976
- 7 E.E. McDonnell; The Caret and the Stick Functions;
APL Quote Quad, Vol. 8, No. 4, June 1978, pp. 35-39
- 8 A.D. Falkoff and K.E. Iverson; APL 360\User's Manual;
Program Numbers 5734-XM1 and 5736-XXM1; Form No. GH20-0683-0; IBM
Corporation, Technical Publications Department, White Plains, New York,
September 1969.
- 9 A.D. Falkoff and K.E.Iverson; APL Language;
Form No. GC26-3847-0; IBM Corporation, Programming Publishing, San Jose,
California, March 1975.
- 10 Second Draft Proposal for Programming Language APL - DIS 84-85;
Edited by L.A. Morrow; Document Number ISO/TC97/SC22/WG3/N55;
AFNOR, Secretariat ISO/TC97/SC22/WG3, Paris, France, 25 February 1986.
- 11 L.A.Zadeh; Fuzzy Sets; Information and Control,
Vol. 8, No.3, June 1965, pp. 338-353.
- 12 H.J. Zimmerman; Fuzzy Set Theory - and Its Applications;
International Series in Management Science and Operations Research; Kluwer-
Nijhoff Publishing, Hingham, Massachusetts, 1985.

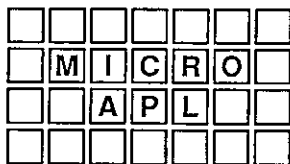
- 13 G. Birkoff and S. Mac Lane; A Survey of Modern Algebra (3rd Edition); The Macmillan Company, New York, 1965
- 14 W. Greub; Linear Algebra (Fourth Edition); Graduate Texts in Mathematics, Vol. 23; Springer-Verlag, Berlin-Heidelberg-New York, 1976.
- 15 K.E. Iverson; Formalism in Programming Languages; Communications of the ACM, Vol. 7, No2, February 1964, pp. 80-88. Reprint; A Source Book in APL; Edited by E.E. McDonnell; APL Press, Palo Alto, California, 1981; pp. 17-25.
- 16 E.E. McDonnell; A notation for the GCD and LCM Functions; APL 75 Congress Proceedings, University of Pisa, Italy, 11-13 June 1975; Edited by P.S. Abrams, G.H. Foster, H.R. Haegi, and S. Trumpy; Association for Computing Machinery ACM, New York, 1975; pp.240-243.
- 17 K.E. Iverson; A Dictionary of APL; I.P. Sharp Associates, Toronto, Canada, March 1987. Revised Edition; APL Quote Quad, Vol. 18, No.1, September 1987, pp. 5-40.
- 18 P. Berry; Greatest Common Divisor and Least Common Multiple now available as primitive functions; The I.P. Sharp Newsletter, Vol.9, No.1, January/February 1981, Technical Supplement 30, pp. T1-T4.
- 19 J.J. Girardot et P.B. Amade; APL90; Journees d'Etude "L'Univers d'APL2", Paris, France, 28-29 Janvier 1988, Association Francaise pour la Cybernetique Economique et Technique AFCET, Paris, France, 1988; pp. 54-70. Reprint: APL-CAM Journal, Vol. 10, No. 2, 16 April 1988, pp. 338-353
- 20 Draft Minutes of ISO-APL Working Group Meeting held at Mobil Research & Development, Farmer's Branch, Texas, 5-7 May 1987; Edited by E.E. McDonnell; Document ISO/TC97/SC22/WG3 N106 (20 May 1987).
- 21 Or/GCD and And/CM (Proposal for Extended APL) Edited by E.E. McDonnell; Document ISO/TC97/SC22/WG3 N108 (1 August 1987).
- 22 E.E. McDonnell; Zero Divided by Zero; APL 76 Conference Proceedings, Ottawa, Canada, 22-24 September 1976; Edited by G.T. Hunter; Association for Computing Machinery ACM, New York, 1976; pp. 295-296.
- 23 J.L.F. De Kerf; The Story of $0 \div 0$; APL80 International Conference on APL, Noordwijkerhout, The Netherlands, 24-26 June 1980; Edited by G.A. van der Linden; North-Holland Publishing Company, Amsterdam, The Netherlands, 1980; pp. 133-135.

- 24 Without (Proposal for Extended APL);
Edited by E.E. McDonnell; Document ISO/TC97/SC22/WG3 N107 (23 June 1987).
- 25 J.L.F. De Kerf; Set Functions and APL;
In Preparation - to be published in the APL CAM Journal.
- 26 P.Lorenzen; Formale Logik; Sammlung Goschen Band 1176/1176a;
Walter De Gruyter & Co., Berlin, 1962.

MicroAPL for ...

APL and networking consultancy

MicroAPL has 10 year's experience with small-computer APL systems. We have pioneered the usage of networked microcomputers and can plan, specify, install and maintain PC networks of all sizes. We have experience with both Ethernet and Token-ring networks and their mainframe gateways. Our programming skills cover all areas of APL and systems programming. MicroAPL can also train and supervise your APL staff on your behalf. If you would like to review your systems requirements, contact David Eastwood at:



MicroAPL Ltd.
South Bank Technopark
LONDON, SE1 6LN
Telephone: 01 922 8866

Access and Barclaycard
Orders accepted

Parallel Computing and APL

by David Piper

Abstract

Parallel computing is a subject rapidly increasing in importance. Parallel hardware has been available for some time. With the advent of chips such as the Transputer some of the hardware is beginning to demonstrate a level of technical maturity. Development of software has not progressed at the same speed. Useable operating systems for transputer based machines are only now appearing. Programming languages and the development of applications lag further behind. Much of the historical effort has concentrated on the development of parallelizing compilers for existing sequential language code. This paper shows how APL is already a highly developed parallel language and with some functional extensions can be made even more effective as a parallel language.

Introduction

The speed of development of computer hardware is such that the physical limits of design are rapidly being approached. The essential limits are in terms of:

SIZE pathways only a few atoms wide
SPEED close juxtaposition to shorten pathways
COMPLEXITY packing densities and connection patterns

Even with the development of quantum well devices [BATE88], the gains to be made are less than those already made in the development of semi-conductor devices.

Once physical limits have been reached, gains in speed will have to be derived by executing more than one instruction at a time - parallel processing. The performance gains that can be obtained by the use of parallel computing are almost limitless. The problem that remains is making use of the existing parallel hardware; parallel software is still in a relatively primitive state.

Classification of Parallelism

The range of parallel hardware already available is quite extensive. A difficulty arises over the classification of exactly what is parallel and what is not. The definitions of distributed and parallel machines certainly overlap. A range of taxonomies have been defined.

The simplest of these, by Flynn [SHARP87], describes the following classes:

SISD	Single instruction, single data (normal sequential processing)
SIMD	Single instruction, multiple data (vector processors)
MISD	Multiple instruction, single data (very rare, pipe-lines perhaps)
MIMD	Multiple instruction, multiple data (full parallel processing)

This classification is good for distinguishing between more traditional machine types (e.g. sequential and vector processors) and the truly parallel processors. It achieves little in terms of classifying the types of parallel machine. Sharp [SHARP87] refines Flynn's classification scheme by sub-dividing the MIMD class into two. The MISD class can be regarded as a special case of the first of these new classes. Sharp's full scheme is as follows:

SPS	Single processor executing scalar data (SISD)
SPA	Single processor executing array data (SIMD)
MPS	Multi-processor executing scalar data (MISD + MIMD)
MPA	Multi-processor executing array data (MIMD)

Classification of Software

The classification of hardware is a complex subject because there are many potential mechanisms for implementing parallelism. As far as software is concerned, the problem is much simpler. We will want to:

- Execute a single process on a scalar or array (SPS, SPA)
- Execute multiple processes on a scalar or array (MPS, MPA)
- Execute multiple processes on multiple arrays (MPA)

The divide is between function and data, the classification made by Sharp is ideal.

Parallel Software

Much of the research into parallel software has been dedicated to the conversion of sequential software (i.e. existing applications) into parallel software. This is to facilitate the execution of existing libraries of code on faster and more powerful parallel hardware.

The research has fallen into 3 main categories:

- Conversion of multi-tasking to parallel-tasking
- Parallelizing compilers
- New parallel languages or parallel extensions to existing languages

Multi-tasking to Parallel-tasking

Many uni-processor systems have emulated a form of parallel processing by the use of multi-tasking systems software. Even relatively simple single user operating systems have rudimentary multi-tasking facilities (e.g. the MS/DOS PRINT command).

The conversion of such multi-tasking systems to true parallel processing is an obvious, and fairly trivial task. A number of problems with the conversion concept can be identified, in particular there may be a failure to take advantage of all the opportunities for parallel processing. Since multi-tasking is a special feature implemented on uni-processor systems via carefully written systems software, not all the available opportunities for multi-tasking will have been taken advantage of. The cost involved in providing more and more multi-tasking capabilities is too great. The literal conversion from single processor to parallel processing systems will similarly fail to take advantage of all available opportunities for parallelism.

Much of the cost advantage provided by a conversion-only exercise will be lost if a more substantial process of re-writing is undertaken. This may, however, be the only realistic way to gain more significant benefits from parallel processing.

Even if operating system software is fully converted to parallel processing, all of the application software still has to be converted. Existing applications will continue to be non-parallel, many of the potential performance benefits remaining unrealised.

Parallelizing Compilers

Another approach to the introduction of parallelism is the creation of parallelizing compilers. These take sequential source code and attempt to detect, and take advantage of, the opportunities for parallel processing inherent in all such code.

The disadvantages of this approach are similar to those discussed above. In particular a parallelizing compiler can only detect parallelism in the code as it is currently written. There may exist other, explicitly parallel, algorithms which are more efficient. Perhaps knowledge based systems offer opportunities here - an intelligent parallelizing compiler that can recognise sequential algorithms and can substitute more efficient parallel ones.

There is however an additional cost. A great deal of effort may have been put into optimising the sequential algorithm, this will be wasted in the conversion to parallel processing. In addition, the optimised algorithm may be more difficult to translate into parallel code than a naive sequential algorithm.

The technologies discussed in this and the previous sections are essentially linked to migration/conversion exercises. The importance of this issue should not be under-

estimated; it is, however, a technology with a finite lifetime. Eventually the development of new parallel code will become the dominant issue and a key task in the search for increasing speed of execution.

Languages with Parallel Constructs

By far the best approach to the creation of parallel code is the use of explicitly parallel languages. These come in two major forms:

- parallel versions of existing languages (e.g. DAP-FORTRAN, PARLOG)
- new languages with explicit parallel constructs (e.g. OCCAM)

A major criticism can be levelled at both the above approaches - the parallelism still has to be programmed in; it does not arise as a natural consequence of the language syntax and algorithms used. The programmer is responsible for administering the parallelism, deciding which sections can be executed in parallel and which must be performed sequentially. As well as being difficult, this introduces a new potential source of errors into the programming process.

Even new parallel languages such as OCCAM are essentially enriched sequential languages, relying on constructs such as FOR-NEXT loops (though these may be executed in parallel). The code produced by the programmer is essentially sequential with some instructions to permit parallel execution. There is no encouragement to think in terms of parallel processes, and the instructions to permit parallelism may be omitted without penalty (until execution time).

Functional Languages

The functional programming paradigm has been proclaimed as being of great value for parallel processing. Many functional languages have been implemented, but few have achieved much commercial success. Perhaps the major exception is LISP. [MICH89] describes functional languages as having the following properties:

- a name is only ever associated with one value
- there is no fixed execution order
- new values are associated with new names via recursion or function calls
- functions can be treated as values

Following from the second property, functional languages provide many opportunities for parallel execution without formal language constructs. If there is no fixed execution

order, then the constituent parts of a program may be executed in parallel. Since the parallelism is derived from functional independence, functional languages are best suited to the MPS form of parallelism.

APL as a Parallel Language

APL has some significant advantages for the implementation of parallel processing systems:

- parallelism is inherent in the language.
- many of the features of functional languages can also be found in APL.

In addition, by adding a single consistent extension to APL, the opportunities for parallelism can be significantly increased.

SPA Parallelism and APL

Single Process, Array data (SPA) parallelism is the form easiest to see in APL, it is apparent in even the simplest APL statements. Conversely, it is frequently the most difficult to implement in other forms of parallel language; this is especially true for functional programming languages. The difficulty arises from the recursive nature of such languages. Parallelizing compilers normally try to pick out this form of parallelism by looking closely at loops or recursion and attempting to "untangle" them in some way.

The difficulties of implementing SPA parallelism are discussed in detail in [SHARP87], chapter 9. On the last page of this chapter, the following appears:

"This chapter has, hopefully, indicated that some problems which initially do not appear suited to array processing may indeed be amenable to such methods. Any APL programmer will need little convincing of that, for many APL one-liners rely on manipulating vectors and matrices as whole objects. It might be argued that rather than adapting strictly sequential languages such as FORTRAN, we ought to have looked at compiling APL for array processes....."

Consider the following APL statement:

$R \leftarrow A + B$

If A and B are arrays, SPA parallelism is immediately apparent. This simplicity of implementation is the basis of IBM's VECTOR processor version of APL2. The compilation of such code is discussed in [CHING88], which includes examples of the ease of translation into vector processor code. These points are also discussed in [BROWN88], and other examples are given in [BUCHHOLZ86].

Opportunities for SPA parallelism are increased in APL implementations with nested arrays. The EACH operator applies a function to each of the items in its argument(s). Thus if C is a nested array:

```
R←p" C
```

returns the shape of each item of C; perhaps the EACH operator should be renamed the PARALLEL operator! Using EACH, SPA parallelism can be applied to non-scalar (and thus, in APL2 nomenclature, non-pervasive) functions.

MPS/MPA Parallelism in APL

APL presents natural opportunities for Multi-Process parallelism (on scalar or array data). These are founded on the same basis as the implementation of MPS parallelism in functional languages. Consider the following fragment of function code:

```
[...] C←A×B
[...] D←I / C
[...] E←C>1000
```

Since the following points are all true:

- C is not changed after its initial assignment
- the result of each line is assigned to a different variable
- the third line makes no use of D

the order of execution of the last two lines of code is immaterial; they may, therefore, be executed in parallel. The same argument can be applied to the parallel execution of defined functions:

```
[...] C←A CALC1 B
[...] D←A CALC2 B
[...] E←C REPORT D
```

Assuming CALC1 and CALC2 have no global side effects, then parallel processing is possible for the first two lines. Note this is an additional argument for the avoidance of global variables. Note also that coalescing the lines of code has no effect on the potential for parallelism:

```
[...] E←(A CALC1 B)REPORT A CALC2 B
```

Whilst functional languages limit this form of parallelism to MPS (scalar data), APL allows both MPS and MPA parallelism. This results from APL's ability to operate independently of the rank and shape of its data (subject to the constraints of conformability).

Extending APL for more Parallelism

One frequent proposal for the extension of APL is the implementation of function arrays. This has been discussed elsewhere in VECTOR (see for example [PIPER89] and the references given therein).

The case for function arrays is best illustrated by some simple examples:

```
1 2 3 + 4 5 6
```

can (in versions of APL that support strand notation) also be written:

```
(1+4)(2+5)(3+6)
```

a very similar expression:

```
(1+4)(2×5)(3÷6)
```

has no equivalent compact form. The implementation of function arrays provides this compact form:

```
1 2 3(+×÷)4 5 6
```

the expression:

```
(+×÷)
```

is a function vector.

The applicability of such function arrays to parallel processing should be immediately apparent. User defined functions can also be stranded, so we can re-write the above example as follows:

```
[...] (C D)+(←A)(CALC1 CALC2)(←B)
[...] E←C REPORT D
```

Note that A and B have to be enclosed so that scalar extension is applied.

Pipe-lining in APL

One technique provided by the ability to strand functions is that of pipe-lining. For example, to read a file, format the resulting data and write an output file, code like the following might be written:

```

[.] EOF←0
[.] START:→EOF/END
[.] (EOF DATA)←READ_INP FILE
[.] RPT←FORMAT DATA
[.] WRITE_RPT REPORT
[.] →START
[.] END:

```

This may not be executed in parallel since it violates one of the rules listed in section 3.2 - the execution of each line depends on the result of the previous line. If function arrays can be used, however, a more concise formulation can be written:

```

[.] (DATA RPT)←↑''
[.] EOF←0
[.] START:→EOF/END
[.] (RPT EOF DATA)←(WRITE_RPT FORMAT READ_INP)RPT DATA FILE
[.] →START
[.] END:

```

The pipe-line construct in the above example permits the re-introduction of parallelism at least to a limited degree. All the functions execute once in parallel then wait for the rest to catch up, before iterating around the loop and commencing again. Note that at the end of file, additional processing is required to allow the FORMAT and WRITE__RPT functions to catch up and process the last couple of records.

Are Function Arrays Needed?

If we look at the above example carefully, we can see that by placing the 3 processing statements inside a subsidiary function we can achieve an identical result without function arrays:

```

[.] (DATA RPT)←↑''
[.] EOF←0
[.] START:→EOF/END
[.] (RPT EOF DATA)←PROC_REPORT RPT DATA FILE
[.] →START
[.] END:

```

```

VRES←PROC_REPORT PARMS;DATA;FILE;RPT
[1] (RPT DATA FILE)←PARMS
[2] WRITE_RPT RPT
[3] RPT←FORMAT DATA
[4] (EOF DATA)←READ_INP FILE
[5] RES←(RPT EOF DATA)
V

```

Despite this, I believe the explicit pipeline using function arrays is much clearer and more easily understood. If (under APL2) we implement an UNTIL operator, we can eliminate the coded loop from our function:

```
[...] (DATA RPT)+c''  
[...] EOF+((WRITE_RPT FORMAT READ_INP)UNTIL 2)RPT DATA FILE
```

Note the right operand indicates which item of the explicit result is to be tested for the end condition. Also the left operand is the entire function vector, an array of functions being regarded as a function in its own right.

Compiling Parallel APL

The idea of compiling APL is now widely accepted. Several commercial products have already reached the market place. A compiler for a parallel APL could easily match the capabilities and performance of compilers for other parallel languages.

For badly written scalar-style APL code, the compiler could make use of all the technique in parallelizing compilers for other languages to optimise the APL code (perhaps this could even be de-compiled to show how the code could have been better written). At the very minimum, a parallel APL compiler could produce performance equivalent to a parallelizing compiler for a sequential language. If the full richness of the parallel constructs is utilised, better performance should be available.

Conclusions

Parallel computing is becoming more and more important as a technology. Hardware products are becoming increasingly commercially available, machines such as the Transputer have achieved a certain degree of acceptance in the market place. IBM's entry into the commercial world with the 3090 Vector Processor signals another stage in the development of commercial parallel hardware.

Currently the weakness in parallel technology is the lack of effective parallel programming tools. APL represents an excellent parallel programming tool of unparalleled (sic) power. The parallelism is inherent in the APL syntax. Unlike functional programming languages, APL presents opportunities for SPA, MPS and MPA form of parallelism. All these forms are available without any additional language constructs.

The possibilities for parallelism can be further enriched in APL by the addition of arrays of functions. These do not necessarily provide additional opportunities for parallelism, they do, however, make existing opportunities easier to exploit.

References

- [BATE88] The Quantum Effect Device: Tomorrows Transistor R.T. Bate, Scientific American Trends in Computing Special Issue (Scientific American, March 1988)
- [BROWN88] An APL2 Description of the IBM 3090 Vector Facility Dr J. Brown, Proceedings APL88.
- [BUCHHOLZ86] The IBM S/370 Vector Architecture W. Buchholz, IBM Systems Journal, Vol 25, No 1. 1986
- [CHING88] A Vector Code Back End of the APL370 Compiler on IBM3090 Wai-Mee Ching and Andrew Xu, Proceedings APL88.
- [MICHEALSON89] An Introduction to Functional Programming through Lambda Calculus G. Michealson, Addison-Wesley ISBN 0-201-17812-5
- [PIPER89] Some Notes on Arrays of Functions D.B. Piper, Vector, Vol 6, No 1. July 1989
- [SHARP87] An Introduction to Parallel and Distributed Processing J.A. Sharp, Blackwell Scientific Publications ISBN 0-632-01462-8

Other Sources of Information

- Function Assignment and Arrays of Functions Dr J. Brown, Proceedings APL84.
- Chips for Advanced Computing James Mundi, Scientific American Trends in Computing Special Issue (Reprinted from Scientific American, October 1987)
- Programming for Advanced Computing D. Gelenter, Scientific American Trends in Computing Special Issue (Reprinted from Scientific American, October 1987)
- Science Now, BBC Radio 4, 2nd January 1989
- Private Communications R.J. Cocking, 1988
- Private Communications D.M. Ziemann, 1988

**British APL Association
Software Library**

New Entries to the List

Compiled by Alison Chatterton

Apologies to all those people who have sent in requests for disks. Although the intention was that I should take over the software library from Dave Ziemann in January, other commitments and a missing piece of equipment, have meant that this has only just occurred - mid-April '89.

I hope that by the time you read this, many of your requests will have been cleared.

A short catalogue of all available disks including I-APL, Morten Kromberg's GRAF, the FinnAPL statistics and Zdenek V. Jizba's tutorials will be printed in Vector 6.1

In addition, a number of new disks, which may be of interest, will have been added to the library. These include:

APL tools for building expert systems. Donor: Z.Jizba

APL kits. Donor: F.Espinasse

Lotus menus. Donor: A.Wiggins

APL migrate. Donor: R.Naugle

PortaAPL demo. Donor: R.Naugle

**British APL Association Software Library
Software Submission Form - Page 1 of 2**

Please copy and fill in this form for EACH disk you submit.

Details corresponding to items flagged (*) will NOT be made publically available, but are for our records only. Please Use BLOCK CAPITALS for all items except number 18.

0. Submission date: _____
1. Name: _____ 2. Daytime phone(*): _____
3. FULL address(*): _____

4. Electronic mail addresses(*): _____
5. Short disk title: _____
6. Brief description of disk: _____

7. List target machine: _____
8. List additional software required: _____
9. Indicate special hardware requirements: _____
10. Is user documentation provided on the disk?(Y/N): _____
11. List titles of any paper documentation included with your submission: _____

12. Is this documentation, or any other, available to users upon application to you? (Y/N): _____ Please give details: _____

13. Does the disk include any form of payment request from users of the software?(Y/N): _____ Please give details: _____

14. Does your submission constitute a 'demonstration disk' in that it demonstrates software that is available for purchase?(Y/N): _____

British APL Association Software Library
Software Submission Form - Page 2 of 2

15. If the TARGET machine is NOT a DOS-based PC, does the disk include instructions for transfer to the target machine?(Y/N): _____
16. File names and descriptions. This information will be made publically available in the software library catalogue. Please save us some work by including these details on a file named <CAT> (no file extension) on your submitted disk. Please enter these details for all files on the disk, except <CAT> itself. You can affix a print of <CAT> below.

For APL workspaces, please use the space below to document functions.

[illegible]

17. Use any extra space above for comments you wish to appear in the software library catalogue.
18. Your signature is necessary. It declares your legal right to make the disk freely available for copying and use, and grants that right to the British APL Association.

Signature: _____

Mail your submission to: The BAA Software Library, c/o Alison Chatterton, 9 Oak Grove,
Hertford, Herts SG13 8AT, ENGLAND

VECTOR 6.1 at New York

**1000 extra copies of VECTOR 6.1 are scheduled for
inclusion in the Conference Wallets at APL89!**

PROVISIONAL CONTENTS**EDITORIAL:**

Peter Donnelly

APL NEWS

Quick Reference Diary
APL Course Diary
APL90 Call for Papers
British APL Association News
The Education VECTOR

Graham Parkhouse
Alan Sykes**REVIEWS**

APL Product Guide
The Toronto APL Toolkit
2nd Generation APLs on Micros ...
IBM APL2 for the PC and PS/2
Impressions: Dyalog APL for MS DOS

Alison Chatterton
Jonathan BarmanDick Bowman
Jonathan Martin**GENERAL ARTICLES**

Printing APL matrices
Personal Constructs and Rule Induction
Case study: The Camacho Maze Program

Jerry Brennan
Mike Williams
Anthony Camacho**TECHNICAL SECTION: Error Trapping**

Hackers Corner
Error Trapping in APL68000
Error Trapping in on Dyalog APL
Semantic Class and Arrays of Functions

Adrian Smith
David Eastwood
Pauline Brand
David Piper**British APL Association Software Library**

Full Comprehensive Catalogue
Index to Advertisers

Alison Chatterton
This means YOU!!

Index to Advertisers

APL People ()	18
British Airways	84
British APL Association	2
Cocking and Drury	16
Dyadic Systems Ltd	6
MicroAPL	60,113
VECTOR back numbers	92
APL Booklist (Renaissance Data Systems)	76

All queries regarding advertising in VECTOR should be made to the BAA administrator, Alison Chatterton, at the address shown on the inside back cover.

BRITISH APL ASSOCIATION

Membership Application / Renewal

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
 Department: _____
 Organisation: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please ring):88/89 89/90

Non-voting student membership (UK only)	£5	£5
UK private membership	£10	£10
Overseas private membership	£18	£18
Airmail supplement (not needed for Europe)	£8	£8
Corporate membership	£85	£85
Corporate membership overseas	£140	£140
Sustaining membership	£360	£360

For student applicants:

Name of course: _____
 Name and title of supervisor: _____
 Signature of supervisor: _____

If you would like to pay by credit card ...

Card no (Access/Visa): _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque "The British APL Association", or you may quote your Access or Visa number. Please send the completed form to:

BAA Administration,
 Alison Chatterton, 9 Oak Grove,
 HERTFORD SG13 8AT
 England

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1988/89 Committee

Chairman:	Peter Donnelly 0420-87024	Dyadic Systems Ltd., Park House, The High Street, ALTON, Hants GU34 1EN
Secretary:	Graham Parkhouse 0483-571281 (x2379)	Dept. of Mech. Eng., University of Surrey, GUILDFORD GU2 5XH.
Treasurer:	John Sullivan 01 462-3628	24 Mounthurst Road, Hayes, Kent BR2 7QN
Journal Editor:	Adrian Smith 04393-385	Brook House, Gilling East, YORK YO6 4JJ
Education:	Dr Alan Sykes 0792-295296	Dept of Management Science University College of Swansea, Singleton Park, SWANSEA SA2 8PP
Publicity:	Jonathan Martin 01 562-5697	(S499) British Airways, PO Box 10, Heathrow Airport, HOUNSLOW, Middlesex TW6 2JA
Technical:	David Eastwood 01-922 8866	MicroAPL Ltd South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Jill Moss 0225-462602	APL People Ltd, The Old Malthouse Clarence St., BATH, Avon BA1 5NS
Projects:	John Searle 01 948-6737	13A Mount Ararat Road, Richmond, Surrey TW10 6PQ
Administration:	Alison Chatterton 0992-552489	9, Oak Grove, HERTFORD, SG13 8AT

Journal Working Group

Jonathan Barman	048839-575
Anthony Camacho	0727-60130
Cathy Dargue	0895-31313
Val Lusmore	0225-448501
Adrian Smith	04393-385
David Ziemann	01-436 9481

Activities Working Group

Phil Goucher	03727-21282
Maurice Jordan	01-562 3090
David Parker	01-834 2333

Typeset using MS Word 4.0 on Xerox 3700 (Covers on Apple Laserwriter)

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
The Old Mailhouse
Clarence St. BA11H, BA1 5NS
Tel:0225-462602

Cocking & Drury Ltd
180 Tottenham Court Rd
LONDON, W1P 9LE
Tel:01-436 9481

HMW Computing Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4A 0HA
Tel:01-353 4212
Fax:01-353 3325

Inner Product Ltd
Earle House
73 Clapham Common Southside
LONDON SW4 9DG
Tel:01-673-3354

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel:01-922 8866

Peter Gynax Systems
213 Goldhurst Terrace
LONDON NW6 3ER
Tel:01-624-7013
Mobile:0860-377963

APL Software Tech,
14 Rosewood Avenue
Alveston, BRISTOL BS12 2PP
Tel:0454-415737

Dyadic Systems Ltd
Park House, The High Street
ALTON, Hants
Tel:0420-87024

APL Impetus Ltd
Rusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent, TN15 0PD
Tel:0732-885126

MetaTechnics Systems Ltd
356 West Barnes Lane
Molespur Park, Surrey
Tel:01-948-6236
Fax:01-336-1062

I. P. Sharp Associates
10 Dean Farrar St.
LONDON SW1H 0DX
Tel:01-222-7033



The British Computer Society, 13 Mansfield Street, London W1M 0BD.